Indira Gandhi
National Open University
School of Computer and
Information Sciences

MCS-202
Computer Organisation

Block

1

Data Representation and Logic Circuits

UNIT 1
A Computer System

UNIT 2
Data Representation

UNIT 3
Logic Circuits – Introduction

UNIT 4
Logic Circuits – Sequential Circuits

FACULTY OF THE SCHOOL

Prof P. V. Suresh, Director
Dr Shashi Bhushan
Mr M. P. Mishra
Prof. V. V. Subrahmanyam
Mr Akshay Kumar
Dr Sudhansh Sharma

PROGRAMME/COURSE DESIGN COMMITTEE

Shri Sanjeev Thakur
Amity School of Computer Sciences,
Noida Shri Amrit Nath Thulal
Amity School of Engineering and Technology
New Delhi
Dr. Om Vikas(Retd),
Ministry of ICT, Delhi
Shri Vishwakarma
Amity School of Engineering and
Technology New Delhi
Prof (Retd) S. K. Gupta, IIT Delhi
Prof. T.V. Vijay Kumar, SC&SS, JNU,
New Delhi
Prof. Ela Kumar, CSE, IGDTUW, Delhi
Prof. Gayatri Dhingra, GVMITM, Sonipat
Sh. Milind Mahajani
Impressico Business Solutions, Noida, UP
Prof. V. V. Subrahmanyam,
SOCIS, New Delhi
Prof. P. V. Suresh,
SOCIS, IGNOU, New Delhi
Dr. Shashi Bhushan
SOCIS, IGNOU, New Delhi
Shri Akshay Kumar,
SOCIS, IGNOU, New Delhi
Shri M. P. Mishra,
SOCIS, IGNOU, New Delhi
Dr. Sudhansh Sharma
SOCIS, IGNOU, New Delhi

BLOCK PREPARATION TEAM

Dr Zahid Raja *(Content Editor)*
Jawaharlal Nehru University
New Delhi

Prof. (Dr.) Seema Verma *(Course Writer – Unit 1)*
Banasthali Vidyapith
New Delhi

Mr Akshay Kumar (*Course Writer – Units 2 to 4)*
SOCIS, IGNOU

(Language Editor) School of Humanities
IGNOU

Course Coordinator: Mr Akshay Kumar

PRINT PRODUCTION

March, 2021

© *Indira Gandhi National Open University, 2021*

All rights reserved. No part of this work may be reproduced in any form, by mimeograph or any other means, without permission in writing from the Indira Gandhi National Open University.

Further information on the Indira Gandhi National Open University courses may be obtained from the University's office at Maidan Garhi, New Delhi-110068.

Printed and published on behalf of the Indira Gandhi National Open University, New Delhi by the MPDD, IGNOU, New Delhi

Laser Typesetting : Akashdeep Printers, 20-Ansari Road, Daryaganj, New Delhi-110002

Printed at :

COURSE INTRODUCTION

The objective of this course is to understand the basic components of the computer systems. The course includes ways of data representation in computers, interconnection structures connecting various components together, description of memory system, input-output system, and the Central Processing Unit of a typical computer system. Going ahead, the course presents an insight into the digital logic circuits responsible for realizing the computer, microprocessors as core of the computing, assembly language programming for understanding the interaction with the microprocessor system and interfacing of some of the important peripheral devices. Some portion from advance computer architecture has also been included to enhance the domain knowledge of the readers.

The first block of the course explains Data Representation, Instruction Execution, Interrupts, Buses, Boolean algebra, Design of Logic Circuits, etc. The second block deals with the Memory System, The Memory Hierarchy, Secondary Storage technologies, the concepts of high speed memory, Cache Organization , Input Output interfaces, Input Output techniques, DMA, Input Output processors, External Communication Interfaces, Interrupt Processing, BUS arbitration, etc. The third block deals with the Central Processing Unit. It includes the Instruction Set, the Instruction format, the Instruction Set Architecture, Micro-Operations, the organization of Arithmetic logic unit, Design of simple units of ALU, the Control Unit, The hardwired control, Wilkes control, the Micro-programmed control etc. The fourth block deals with the Assembly Language Programming, Microprocessor, RISC, and various types of multiprocessor technologies.

The List of Facebook Recorded Sessions available on IGNOU Facebook page (They were recorded for MCS012 but are very useful for MCS-202. You are advised to watch them for MCS-202): These Videos are also available through eGyankosh link http://egyankosh.ac.in/handle/123456789/60684

1.**The Basic Computer and Fixed Point Numbers https://www.facebook.com/Official.../videos/567832200754534/**

2.**Floating Point Number representation and Error Detection Codes https://www.facebook.com/Official.../videos/179423369719644/**

3.**Combinational Circuits https://www.facebook.com/Officia.../videos/2639701279681674/**

4.**Sequential Circuits https://www.facebook.com/Officia.../videos/2997470290273216/**

5.**Memory Organisation https://www.facebook.com/Officia.../videos/2621991574793648/**

6.**Cache Mapping and I/O Organisation https://www.facebook.com/Official.../videos/265746954469764/**

7.**Assembly Language Programming for 8086 Microprocessor https://www.facebook.com/Officia.../videos/3835800186493405/**

8. **Discussion on 8086 Assembly Language Programs https://www.facebook.com/Officia.../videos/3835800186493405/**

9. Discussion on MCS012 - Block 3: The Central Processing Unit https://www.facebook.com/Official.../videos/568476564051914/

10. MCS012: Computer Organisation and Assembly Language Programming – An Overview https://www.facebook.com/Official.../videos/357526481886539/

11. Reduced Instruction Set Computer (RISC) Architecture https://www.facebook.com/Official.../videos/265809814701761/

12. Procedure calls in 8086 Assembly Language Programming https://www.facebook.com/Official.../videos/292194305388266/

BLOCK INTRODUCTION

The first block of the course introduces you to some of the basic concepts relation to a computer. The Block is divided into four units.

Unit 1 explains some of the basic aspects of instruction execution and some of the architectures that has been employed for design of a computer. This unit also introduces you to brief history of computers and interrupt mechanism of a computer system

Unit 2 explains the Data Representation in details. It introduces you to conversions among the basic number systems, such as Binary, Decimal, Octal, Hexadecimal. In addition, the unit discusses about the character representation including ASCII and Unicode. This Unit also introduces you to error detection and correction mechanism in the data units of a computer.

Unit 3 introduces you to the concepts of basic logic circuits used in making of a computer. It introduces the concept of Logic Gates, Boolean algebra, Combinational circuits. It also explains cetrain examples of combinational circuits such as Adders, Decoders, Multiplexers, ROM etc.

Unit 4 introduces you to concept of Sequential circuits. It explains the functioning of basic latches and introduces you to Flip flops, Excitation tables, Master-Slave flip flops etc. It also presents certain examples of sequential circuits such as Counters, Registers, RAM, etc.

A course on computers can never be complete because of the existing diversities of the computer systems. Therefore, you are advised to read through further readings to enhance the basic understanding that you will acquire from the block.

Further Readings For The Block

1) Mano M Morris, *Computer System Architecture*, 3rd Edition/Latest Edition, Prentice Hall of India Publication, Pearson Education Asia

2) Stallings W., *Computer Organization & Architecture: Designing For Performance*, 10th/11th Edition, Pearson Education Asia

3) Hennessy/Patterson, *Computer Organization and Design : The Hardware/ Software Interface*; 5th/6th Edition, Morgan Kaufmann.

UNIT 1 Computer System

Structure **Page Nos.**

1.0 Introduction
1.1 Objectives
1.2 A Brief History
1.3 Structure of a Computer
  1.3.1 The CPU
  1.3.2 Register Sets
  1.3.3 Datapath
  1.3.4 Control Unit
  1.3.5 Memory Unit and I/O Devices
  1.3.6 What is an Instruction?
1.4 How are Instructions Executed?
1.5 Instruction Cycle
1.6 Various Computer Architectures
  1.6.1 von Neumann Architecture
  1.6.2 Harvard Architecture
  1.6.3 Instruction Set Architecture (ISA)
  1.6.4 RISC
  1.6.5 Multiprocessor and multicore Architectures
  1.6.6 Mobile Architecture
1.7 Summary
1.8 Solutions/Answers

1.0 INTRODUCTION

In the present competitive world, a business can survive if it uses most advanced Information technology to support various businesses processes. In this digital world, computers are an important part your daily life. You use computer to use health services, banking services, teaching and learning services, online services made available by the Government and many more such services. Computer technology can be used to make all these processes more efficient and user friendly.

This Unit introduces you to some of the basic terminologies used to define computer system of today. In addition, the breakthrough in the history of computer systems has also been included in this Unit. This Unit also introduces you to some of the popular computer architectures, like von Neumann Architecture, which was one of the first computer architecture, and other contemporary architectures of Computer System. Also, a simple novel idea for execution of instructions has also been introduced. This basic process of instruction execution will be explained in more details in the later Units of the course. You can get more information on these principles from the further readings.

1.1 OBJECTIVES

After going through this unit, you will be able to:

- explain the basic structure of computer system
- list and explain the features of the von Neumann architecture of the computer;
- identify some of the important breakthroughs in history of computers;
- identify the process of instruction execution;
- define some of the contemporary computer organization

1.2 A Brief History

This section traces a brief historical background of computer. You should be aware of that the push to construct Computer has not come from one person or group or organisation. There were many attempts to develop an automatic and programmable computing device. In 1944, the University of Pennsylvania developed the first automatic computing device, which was called Electronic Numerical Integrator and Calculator (ENIAC). It was a general-purpose machine fabricated utilizing vacuum tubes. This machine was planned basically to compute the shooting scope of weapons during World War II. It was arranged by manual setting of exchanging and associating links. An improved ENIAC model was called Electronic Discrete Variable Automatic Computer (EDVAC), which was completed in 1952. Meanwhile, the specialists at the Institute for Advanced Study (IAS) in Princeton assembled (1946) an IAS machine, which was multiple times quicker than ENIAC.

A similar project was started in 1946 at Cambridge University. The objective of this project was to design a computer that stores instruction and data in the computer memory. This proposed computer was known as the Electronic Delay Storage Automatic Calculator (EDSAC). In 1949, EDSAC was completed and became the first computer of the world that stored the program instructions and related data in the memory of the computer. The architecture of these machines is discussed in section 1.6. Thereafter, Harvard launched a series of computes named Harvard Mark I, Mark II, Mark III, and Mark IV. The machines have different architecture than that of other contemporary machines. This architecture was called the Harvard Architecture. The Harvard architecture is discussed in section 1.6. The term Harvard architecture, now, used for machines, which have separate instruction and data cache memory. The term cache memory is explained in Block 2.

In 1951, UNIVersal Automatic Computer (UNIVAC I) was developed. This can be called the first commercial computer. In 1952, IBM announced its first computer, the IBM701. Later in 1964, IBM announced a family of computers, called IBM 360 series. The basic concept of family of computers was to keep the similar instruction set. However, the performance and price of the family increased with higher models. This allowed businesses to migrate to higher performance computer while retaining their investments in software. Another important computer series of this time was PDP-8 by Digital Equipment Corporation (DEC).

Advancement of technology lead to development of microprocessor on a single silicon chip. In 1971, Intel produced its first microprocess, called Intel 4004. The first personal computer (PC) was introduced by the Apple computer in 1977. Further, in this year itself world again saw the installation of a mainframe computer. A mainframe computer had many user terminal that used to solve the processing power of a single mainframe computer VAX-11/780 by the DEC. The year 1977 may be considered as an important year for computer industry, as in this year in addition to the developments as given above, one of the important microprocessor 8086, which you will study in Block 4, was introduced. This led to development of micro-computers. Different organisations launched their micro-computers based on Intel 8088/8086 microprocessors or many other microprocessors. In the subsequent years, the computers developed by Compaq, Apple, IBM, Dell, and many others became popular in the academia and

industry. A powerful computer that could perform millions of computation, called super computers were developed. The first such computer, the CDC 6600, was introduced in 1961 by the Control Data Corporation. Cray Research Corporation introduced the most expensive but most efficient super computer Cray-1, in 1976. During the decades of 1980s and 1990s, more powerful super computers were launched. These super computers had larger number of more powerful processors. These super computers were categorised on the basis of the sharing of memory by various processors, viz. shared memory and distributed memory super computers. A super computer differs from a simple multi-processor system, as it can have hundreds or even more processors. Examples of computers at this time included Intel iPSC, nCUBE, Imaging Equipment (CM-2, CM-5), and many others. The perfect design of computer is to install central servers through computer networks. These networks connect cheap desktop machines that have the ability to compensate for unmatched computing power. However, in late 1990s, with the popularity of Local area networks (LANs), which allowed sharing of resources, mainframes and minicomputers were replaced by network computers or personal computers, called desktop computers. These individual desktop computers were then connected to large computers over broadband (WAN) networks. The inclusiveness of the Internet has developed keen attention in the areas like network and grid computing. "Grids are geographically distributed platforms of computation". Grid computing provides reliable, consistent, persistent, and low-cost access to high-end computational facilities.

Table 1.1 highlights the development of Computers, in terms of type of circuitry used, the size of Random Access Memory (RAM), the speed in terms of Instruction execution per second and Programming languages of those era. The Table also lists some of important computers of those days.

Subject	1st generation	2nd generation	3rd generation	4th generation	5th generation
Period	1940-1956	1956-1963	1964-1971	1971-present	present & beyond
Circuitry	Vacuum tube	Transistor	Integrated chips (IC)	Microprocessor (VLSI)	ULSI (Ultra Large Scale Integration) technology
Memory Capacity	20 KB	128KB	1MB	Magnetic core memory, LSI and VLSI. High Capacity	ULSI
Processing Speed	300 IPS instructions Per sec.	300 IPS	1MIPS (1 million inst. Per sec.)	Faster than 3rd generation	Very fast
Programming Language	Machine, Language	Assembly language & early high-level languages(FORTRAN, COBOL, ALGOL)	C,C++	Higher level languages,C,C++,Java	All the Higher level languages,,Neural networks,
Example of computers	UNIVAC, EDVAC	IBM 1401, IBM 7094, CDC 3600,D UNIVAC 1108	IBM 360 series, 1900 series	Pentium series,Multimedia,	Artificial Intelligence, Robotics

TABLE 1.1 Decades of Computing
["Ref: https://www.discovertips.in/2017/06/computer-study-notes-history-and.html"]

Semi-Conductor Chips and Computer

A computer system consists of number of components, which are fabricated using semicoductor materials. The basic unit of these semi-conductor materials is an electronic transisters, which are used to integrate large computer circuitry on a single semi-conductor chip. Over the last five decades computer has shown an exponential rate of improvement due to growth and advancement in circuit integration technology on a semi-conductor chip; from a handful of transistors to millions of transistors on a single integrated chip. This resulted in multiple

fold increase in the size of computer memory and very large increase in processing capacities of computer processor. The integration technology has shown a rapid growth. Initially, only a few transistors were fabricated on a computer chip. This was called small-scale integration (SSI). Subsequently, with the advancements of technology, computer chips were made using the medium-scale integration (MSI), then using the large-scale integration (LSI), then using the very large-scale integration (VLSI), and currently to ultra large-scale integration (ULSI). Figure 1.2 includes various technologies and number of transistors per chip, also called as chip complexity. The integration growth can be more understandable in terms of "feature size". Feature size is that dimension of transistor that is required to be minimized without changing the functionality of the device. So, a decrease in feature size will eventually increase the number of transistors per chip which in turn, will result in development of new advanced chip having advanced functionality. This has resulted in growth of semiconductor memories (RAM memories) so that now, designers can trade off speed with memory size.

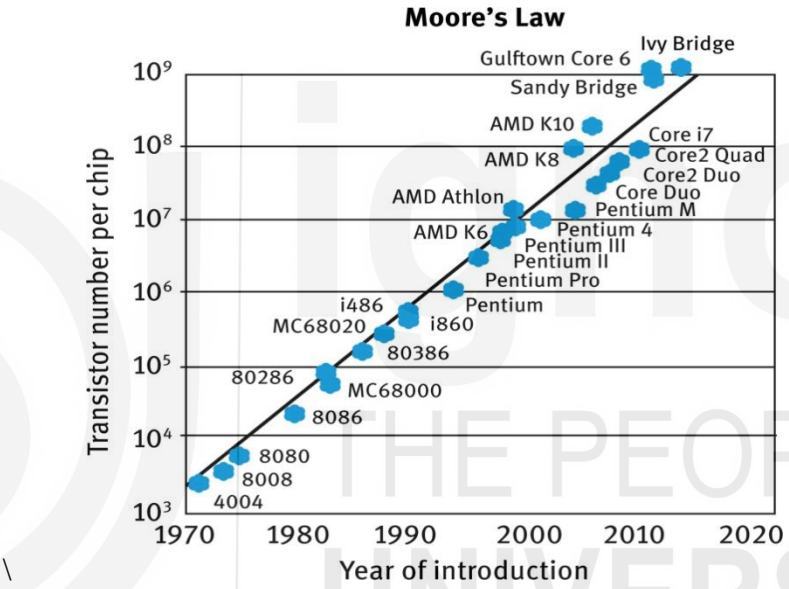


Figure 1.2: Numbers of Transistors per Chip
["Ref: https://www.ncbi.nlm.nih.gov/books/NBK321721/figure/oin_tutorial.F3/"]

1.3 STRUCTURE OF A COMPUTER

As discussed in the previous section, computer is fabricated using semi-conductor chips. This technology primarily have two stable states of representation, such as presence or absence of output. These two states are used to denote bits 0 and bit 1 respectively. Thus, the computer is a digital device, which performs all the operations using binary digits (bits). Therefore, computer instructions, character set, data in computation etc everything is to be represented and processed using binary codes or binary number system. The minimal processing requirements of a computers system are:

- It should allow input of instructions or commands and data on which these comamands are to perfomed.
- There can be large number of instructions that may be required for data manipulation. Some of these instructions may involve decision making and/or repetitions, therefore, it may be a good idea to store instruction and data. Thus, a computer should have memory.

- The computer should be able to process data as per the command/instruction. Thus, it will require components, which may process data, store temporary results and transfer data among different units.

- It should be able to store the results of the processed data using some output unit.

Therefore, a computer system should have a processor to perform computations as per the instructions, a memory for data storage/instructions, Input/Output units to input the data and insturction and output the results and a data path to transfer data. In order to define a simple structure of a computer system, the role and functions of its basic components should be studied. This section defines the basic components of the computer, like CPU, the Memory, the Input/output devices, data paths etc.

1.3.1 The CPU

The CPU is responsible for executing a sequence of instructions, which are part of a computer program. A computer program along with the data is stored in the main memory of a computer system. The CPU consists of following components: (1) registers, (2) an arithmetic logic unit (ALU), and (3) a control unit (CU). The role of each component of CPU is well defined. The registers are used to store - (1) commands or instruction which is presently being executed by the Arithmetic logic unit, (2) they can also store addresses such as address of operands /data, "address of the next instruction to executed", (3) the data or operands itself. Arithmetic logic unit is designed to perform binary computations.

The control unit (CU) controls the execution of instructions by the computer. CU controls the fetching, interpreting and execution of the commands or instructions on a computer, which primarily results in processing of the data stored in registers or the main memory. Figure 1.3 shows the structure of the CPU and its interaction with the memory system and input / output devices. The CPU "reads commands form the memory, reads and writes data from and to the memory, and transfers the data to the input / output devices". The most common and simple commands/instruction processing can be summarized as follows:

1. Get the next instruction or command to be executed from the main memory to the CPU registers
    a. In general, the address of the next instruction is stored in the program counter register (PC).
    b. The CU causes the instruction fetch operation using the PC.
    c. The fetched instruction is stored in the CPU, in general, in an instruction Register (IR)
2. The instruction in the IR is interpreted by the control unit.
3. In addition to step 2, the operands are brought from the main memory to CPU registers.
4. The operation as interpreted at point 2, is performed on the data obtained in step 3.
5. Results of the operation in step 4 are stored in the CPU registers. In case the instruction specifies that the result of the operation is to be stored in the memory, then the result data in CPU registers are transferred to the specified memory locations.

The execution cycle is repeated as long as there are more commands to be executed. Sometimes the execution of a program is required to be terminated abnormally due to occurrence of certain error or other conditions, like division by zero. Thus, there may be need of a mechanism that can interrupt the execution

of a sequence of instruction. This mechanism is known as an interrupt mechanism, which uses an interrupt signal. On occurrence of an interrupt signal, CPU suspends the execution of next instruction to be executed, though it completes the execution of the current instruction. More specifically, when a request for interruption occurs, a move to an interrupt management process occurs. Interrupt management systems are the set of programs that are used to address the cause of the interruption and restores the system to the last instruction, where interruption was acknowledged.

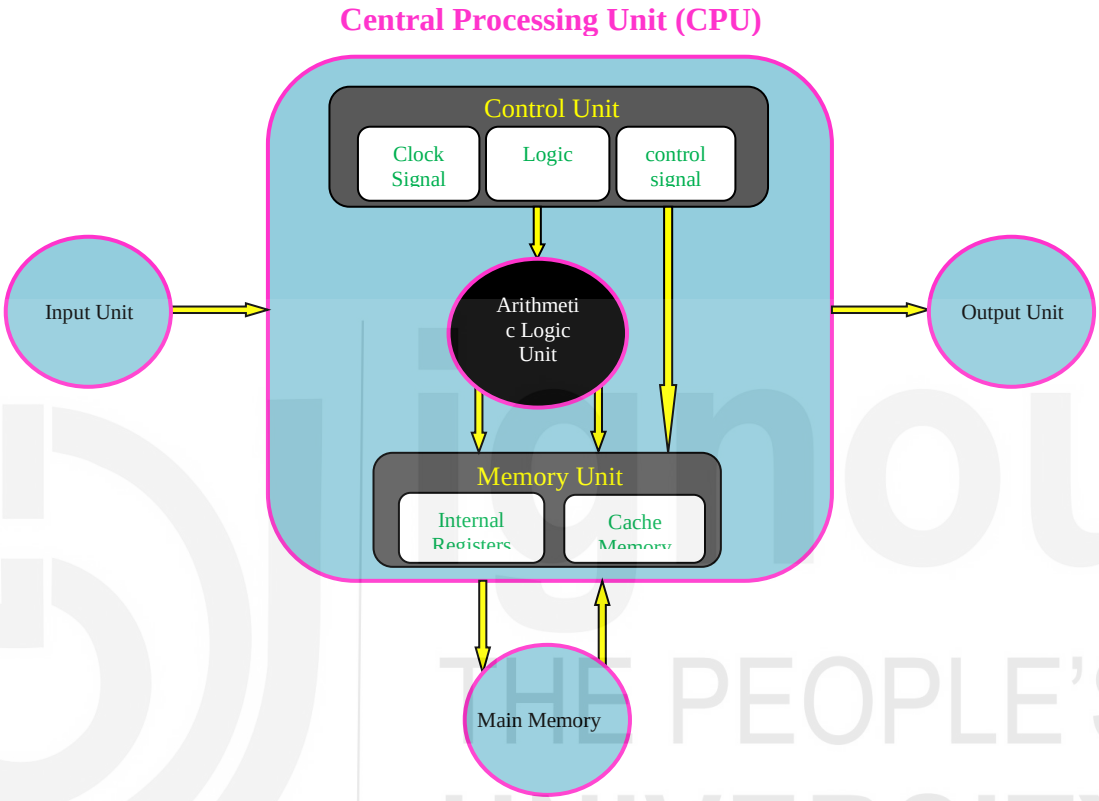


Figure 1.3: Central processing Unit and its components

1.3.2. Register Set

Registers are extremely fast, but temporary storage locations, within the CPU. Registers store the intermediate results, therefore, are used to enhance the CPU performance for performing arithmetic; logical or other operations. The names, number, type, and length of registers are different for different computers. In general, general purpose registers can be used for storing the data for an operation or address of an operand or any other purpose of various tasks in a program. Special purpose registers are limited to designated functionalities only. There are cases when some registers are used only for performing operations on data and cannot be used for operand address computations. These data registers length should be long enough to hold values for most types of data. Some machines allow two integrated registers to hold longer length data. Address registers are used for computing the address of an operand in the memory. This task may be assigned to specific types of register or the general address purpose register may be used for this purpose too. Address registers should be long enough to handle a very large address, which in general depends on the size of the memory. The number of registers in a computer affects the size of the instructions. Why? you will find answer to this question in Block 3. Some important registers are given below:

Register for Fetching and storing Instructions

In a computer system the instructions of a program are stored in the main memory. Two main registers, which are involved for transferring instructions from main memory to CPU are: Program counter (PC) and instruction/command or instruction register (IR). PC contains the address of the memory location from which the next command may be executed. Instructions are fetched to IR, so that it can be interpreted and executed.

Condition Registers and Status registers

In some computers, status registers or flag registers are used to store status information of various operations. Some computers have a special program status register (PSW). PSW contains the present status of the processor flags. A detailed discussion on flags is given in Block 3 and Block 4.

1.3.3. Datapath

As discussed earlier, a computer performs instruction execution using differnet components. But, how does the data and instructions get communicated from one component to other. This is achieved by datapaths. There can be two diferent types of datapaths:

(i) The datapaths which are internal to CPU: Such datapaths transfer two different categories of data. Data category, which includes data contained in different registers of ALU. The control data, which essentially pulls control signals from the datapaths for the use of control unit. In addition, the data is moved between two registers or between ALU and a register. This internal data migration is done by local buses, which may carry control information, instructions and addresses.

(ii) Externally, data may be transferred to and from registers to memory and I / O devices, usually using a special set of circuit called the **system bus**.

Internal data transfer between registers and between ALUs and registers can be done through various organizations including one, or more buses. Dedicated datapaths can also be used for data transfer devices between the CPU components on a regular basis. For example, Program counter register (PC) content is transferred to Memory Address Register (MAR) to fetch new commands at the commencement of each command cycle. Therefore, dedicated datapaths from PC to MAR can help speed up this part of command execution.

1.3.4. Control Unit

The control unit is primary unit which commands operations of the system by giving control signals to various units of computer. The control unit is also responsible to regulate internal and external flow of data from CPU. The data transfer from CPU to/from memory and CPU to/from I/O is also controlled by these signals. A continuous pulse sequence is generated by a system clock over a set period of time. This sequence of steps, identified as t0, t1, t2, ..., ($t0 \leq t1 \leq t2$, ...), is used to perform a specific command by enabling control signals in a specific order. The details on various types of control units and their operation are discussed in Block 3 of this course.

1.3.5 Memory Unit and I/O Devices

An interesting part of computer is the memory of a computer, which stores the data as well as instructions. Computer stores binary digits 0 and 1 called bits. However, bit is a very elementary, therefore, a meaningful combination of bits is generally, required to be stored in memory. A group of 8 bits is traditionally called a “Byte”. However present day data may include 16 bits (2 bytes), 32 bits (4 bytes), 64 bits (8 bytes) and so on.

Interestingly the size of the memory is represented as a byte in many situations, which was equal to one character in earlier data representation (Please refer to Unit 2 for data representation). Therefore, higher units are needs to measure the size of the memory. As computer is a binary device, an interesting combination as given in the following table is used to measure the memory capacity.

Unit	Equivalent to
1 Kilobyte (KB)	2^{10} Byte = 1024 bytes ≈ 1000 bytes
1 Megabyte (MB)	2^{10} KB = 20^{20} byte
1 Gigabyte (GB)	2^{10} MB = 20^{30} byte
1 Terabyte (TB)	2^{10} GB = 20^{40} byte
1 Petabyte (PB)	2^{10} TB = 20^{50} byte
1 Exabyte (EB)	2^{10} PB = 20^{60} byte
1 Zettabyte (ZB)	2^{10} EB = 20^{70} byte
1 Yottabyte (YB)	2^{10} ZB = 20^{70} byte
and beyond	

As stated earlier, the purpose of memory in computer is to store the instructions of a program and the related data. These instructions and related data to these instructions are fetched by the CPU, which then executes theses instructions. This memory is also called the Random Access Memory (RAM) as any location of the memory can be addressed randomly. Details on memory system are given in Block 2.

I/O devices are used to input data and programs, e.g. a keyboard can be used to input a program. Present day computer use pointing devices and screens to select options from Graphical user Interfaces (GUI) like Microsoft Windows. The output devices like printer, monitor etc. display the output either in printed (also called hard copy) form or are displayed on the details on I/O devices are also given in Block 2.

1.3.6 What is an Instruction?

In the entire discussion on CPU, one term was used consistently - an instruction. What is an instruction in the context of a computer system?

In general, you write programs in a high level computer language like C, Python, JAVA, C++ and so on. Most of these programming language requires you to compile the program into an object program, which is linked with library programs and loaded in the memory of the computer. A program also include some basic data and I/O from keyboard or files. These loaded programs contains binary instructions, which operate on binary data.

Some of the common high level statements include arithmetic instructions like addition, subtraction; decision like if-then-else; repetitive loops which also involves decisions like while, for etc.; and procedure/function calls. In all these statements few common characteristics stand out, which are made part of binary instructions as given below:

(1) Each binary instruction may define one operation using a binary operation code also called opcode.

(2) Each instruction may have one or more operands which may be data itself or can be used to compute the address of operand. A computer instructions, depending on machines, may consist of one to three operands.

(3) The result of operation of machine instruction can be stored in some machine register.

(4) Some instructions, like branch, function call etc., result in transfer of execution to a new instruction, which may be at a different address in the program. This, can be achieved by changing the value in program counter register, which stores the next instruction to be executed.

Binary instruction design of a computer system is a very complex task. In fact the machine instructions of different computers are different, that is why you require a separate compiler for a separate type of computing machine. A detailed discussion on computer instructions is given in Block 3.

Check Your Progress 1

1) State True or False

		T/F
a)	Byte consists of 8 bits, and it may represent a character.	☐
b)	A character representation requires at least one byte.	☐
c)	One bit is an independent unit and can be accessed independently.	☐
d)	A machine can have two paths called internal and external.	☐

2) Explain the concept of an Instruction.

……

……

……

3) Why does a computer need CU, ALU, memory and I/O devices?

…… ……

……..

……..

1.4 HOW ARE INSTRUCTION EXECUTED?

After getting a fair idea about basic structure of computer system, let us now understand about execution of a program. To learn about how a computer executes a program, let us take an example from high level language.

Consider the following program segment of a high level programming language:

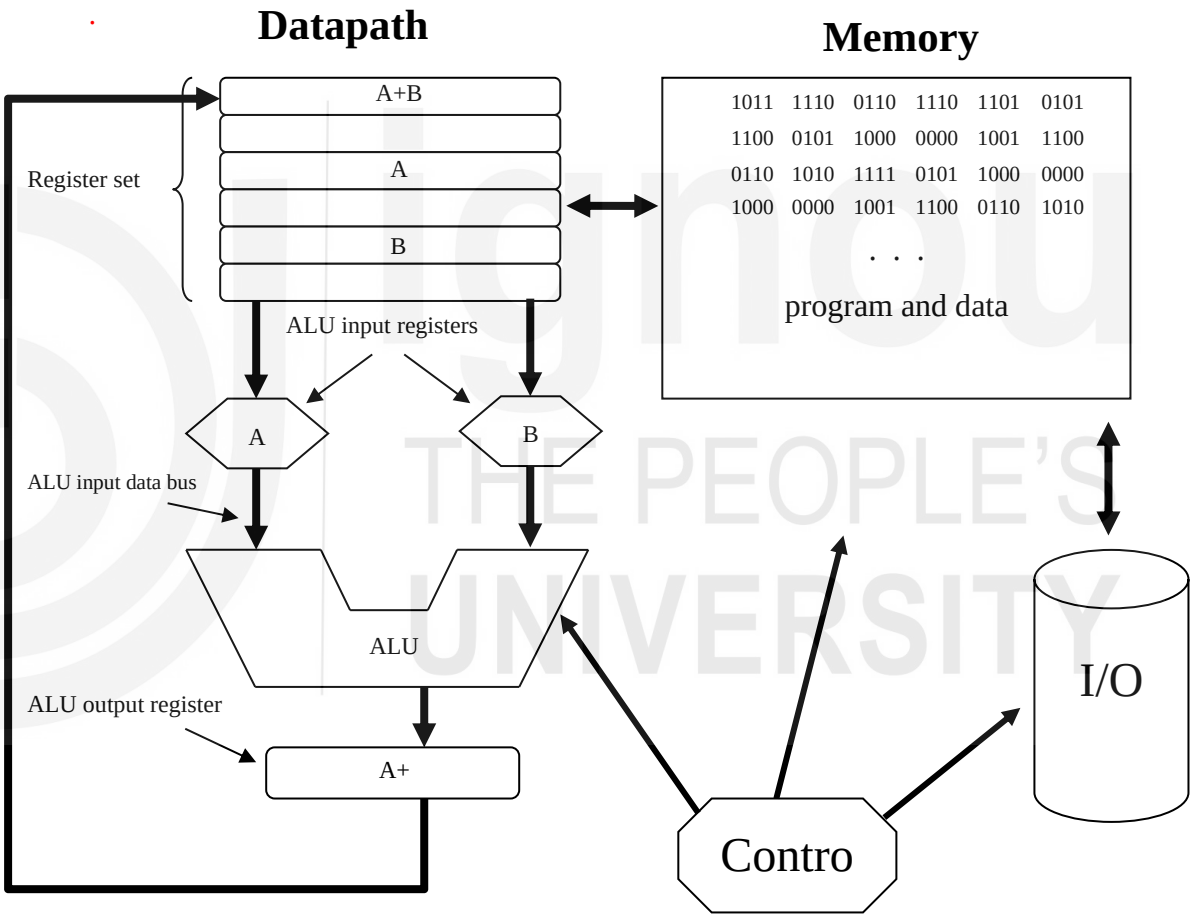
```
int x=10, y=5, z;
z=x+y;
```

Please note that the instruction *int x=10, y=5, z;* will allocate three memory locations of type integer (how big will be one integer). The *x, y* and *z,* in the context of programming languages, are called variables. This instruction will also assign integer value 10 in first location, identified by variable name *x* and integer value 5 in second location identified by variable name *y*.

The instruction will also allocate a third memory location named *z*. Thus, first instruction, technically will be translated to create three integer location named *x*, *y* and *z*. Please note that these locations, when loaded in the memory will be identified as three separate addresses. The second instruction *z=x+y;* will be executed by the CPU to produce the desired results. But, how does these set of instructions be executed by computer? As a first step, a compiler program will be executed by computer and all the High level programming statements will be translated to a machine language program consisting of data in the form of variable locations and values, and instructions as binary operation codes and operand addresses.

Please note in the C program segment, the declaration results in creation of variable locations, with data values.

It is the instruction *z=x+y;* which gets converted to one or more machine instructions, which will have binary codes indicating addition operation, opened address on which this addition is to be performed, and where the result will be stored. Assuming a typical machine is shown in Figure 1.4



Fetch – Decode – Execute Cycle

Figure 1.4: "Instruction and data format of an assumed machine[https://www.cise.ufl.edu/~mssz/CompOrg/CDA-lang.html"]

Please notice the role of ALU registers. They get values of locations *x* and *y* first to be added by the ALU and the answer of this addition is stored in a register, which is sent back to the memory location *z*.

In general, a machine consists of several registers as shown in figure 1.5. The details on these registers will be discussed in Block 3 of this course.

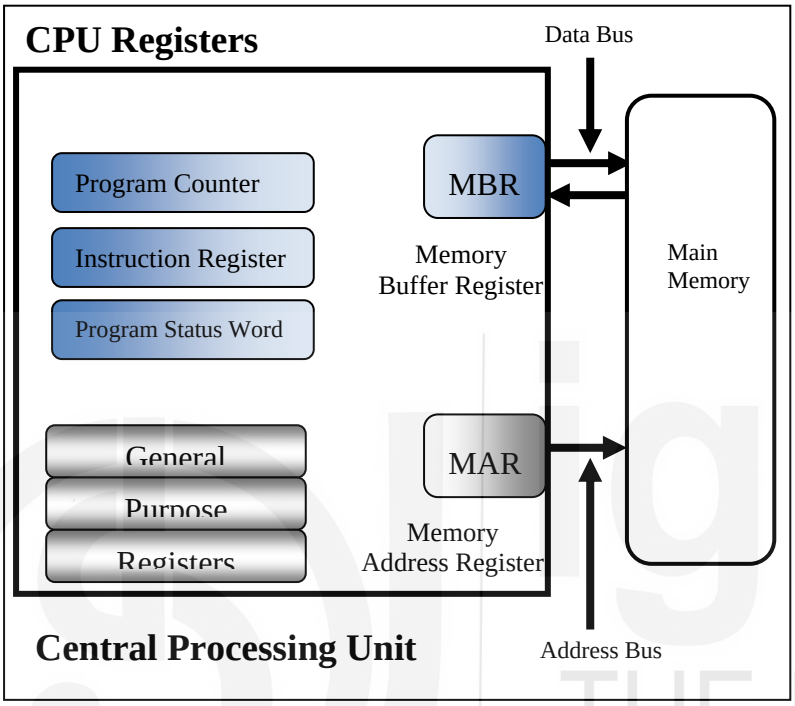


Figure 1.5: "Central Processing Unit with registers [http://digitalthinkerhelp.com/what-is-cpu-processor-register-in-computer-its-types-with-functions/]"

1.5 INSTRUCTION CYCLE

A program is having a set of instruction which are stored in computer's memory. In general, the instructions of a program are executed sequentially by the processor. A computer system executes an instruction in the following four phases:

1. Fetching.
2. Decoding
3. Read the operands from the memory.
4. Execution.

These steps are explained in details in Block 3. In this unit they are just being introduced.

Fetching the instruction: An instruction is placed in the memory location or locations in RAM. Instruction fetch will bring this instruction into the Instruction register. This step requires the information about where the instruction is in the memory. In general, PC register contains this information.

Decode the Instruction: Decoding the instruction requires to interpret the operation code of the instruction, this will be followed by finding locations of the operands in the memory.

Read the operands from memory: Once the addresses of the operands are known, they are brought into the CPU registers, such that the required operation may be performed.

Execute the Instruction: Finally the instructions are executed and results are stored in the local temporary locations.

The process is shown in figure 1.6

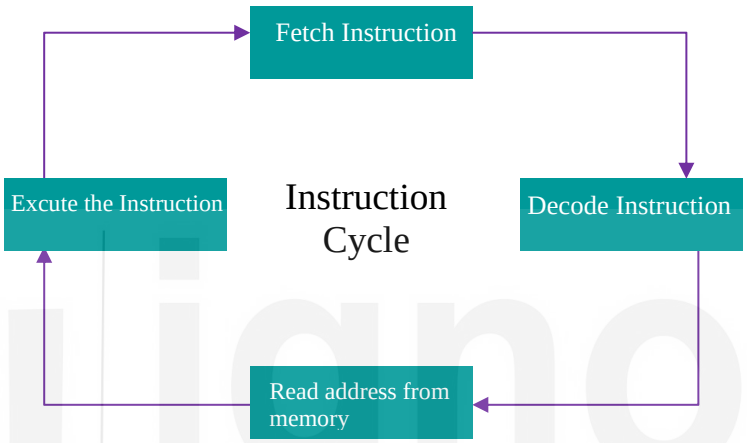


Figure 1.6: Instruction Cycle [https://www.javatpoint.com/instruction-cycle].

Can a program be interrupted while it is getting executed? A program may be required to be stopped, during the execution due to occurrence of errors or occurrence of completion of certain other important – I/O events. This process has been explained in the next subsection:

Interrupts

The meaning of word interrupt is to stop the ongoing activity. In computer, an interrupt, stops execution of next instruction in the instruction cycle, once the execution of ongoing instruction is completed. Why? Your will get an answer with this question in Block 3.

Why does an interrupt occurs in a computer?

Interrupt occurs due to:	**Example**
Program instruction execution itself. Causes an interrupt.	❑ Division by Zero occurs ❑ The result exceeds the limit of the number allowed. ❑ Security violations.
Clock initiated.	• Expiry of time limit of a program.
I/O devices	❑ Input/Output starting or completion ❑ Error due to an I/O device
Hardware generated.	❑ Memory errors

Figure 1.7: Example of Interrupts in a Computer.

Interrupt mechanism is a very useful mechanism for increasing the efficiency of program execution. The instruction cycle many continue, till the time an interrupt occurs. As a result of an interrupt, the CPU knows that some event has occurred and then CPU stops the execution of the current program and goes on to process that event. CPU then uses the following steps to process the interrupt:

- The CPU identifies the source of the interrupt.
- CPU executes and Interrupt Servicing Routine (ISR), which services the event that has occurred.
- Meanwhile, the program which was being executed is moved to a hold state.
- Once the CPU completes the ISR, i.e. executes it till its completion, it resumes the execution of program it has put on hold.

Interrupts and Instruction Cycle

The interrupt process is summarised below:

- CPU is executing a program say "X". and is in the decode stage of ith instruction of the program X.
- Assume an interrupt due to an event occurred at this time.
- CPU waits till the ith instruction execution is complete, and then stores the register values & PC, which has address of $(i+1)^{th}$ instructions into memory or special storage area.
- CPU indentifies the interrupt and executes the interrupt servicing program till it is complete.
- CPU then return to execution of the $(i+1)^{th}$ instruction by restoring the register and PC.

Thus, after interrupt processing, the execution of the interrupt program is resumed. Figure 1.8 shows Interrupt cycle.

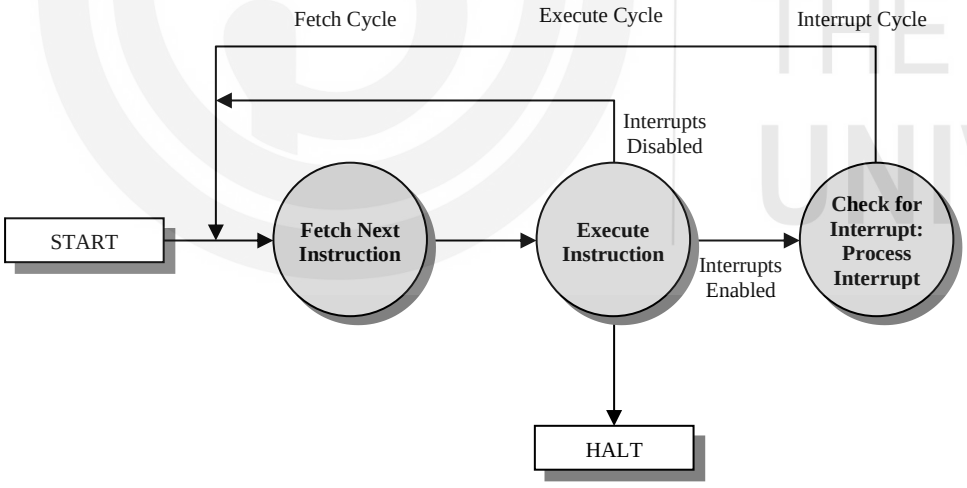


Figure 1.8: "Instruction Cycle with Interrupt Cycle [https://www.ece.ucdavis.edu/~vojin/CLASSES/EEC70/W2004/presentations/Ch_03.pdf"]

Please note an important point in the figure 1.8, which is - interrupt enable and interrupt disable conditions. If CPU is executing very important instruction, the response to an interrupt can be disabled. In this situation interrupts will not be processed or acknowledged by the CPU till they are enabled again.

Check Your Progress 2

1) **State True or False** T/F

i) Assume a computer has one-byte long instructions, its PC contains a decimal value 205 and only one byte is fetched from the memory at a time. For this machine, after fetching an instruction, the value of PC will become 206.

ii) PC register is needed to fetch the data from memory. ☐

iii) A clock signal cannot cause as interrupt. ☐

iv) The interrupt are answered by the hardware. ☐

v) The processor processes one interrupt, even if, multiple interrupts may occur at a time. ☐

2) Explain the term interrupt and its cause.

.....

.....

.....

3) Explain the interrupt processing in a computer.

.....

.....

.....

1.6 VARIOUS COMPUTER ARCHITECTURE

Architecture of a computer contains procedures or processes to explain the user about the functionality of a computer system. This is analogous to design of a building, i.e. the construction of buildings is tailored to the needs of the user by taking into account the cost threshold. The original design is designed on paper. Likewise the computer, after it was constructed using the logic of transistors and integrated circuits, the construction was tested and built in a hardware way. Computers can be evaluated based on their performance, efficiency, reliability, and cost of computer software, which works with software and computer technology standards. In this section, you will learn about various computer architectures.

1.6.1 von-Neumann Architecture

John von – Neumann has invented this architecture. Several computers are based on the construction of this architecture and other improvements on this architecture. The basic building blocks of this architecture are-

1. CPU
2. Memory Unit
3. I/O Devices
4. Connection Structure

Each memory unit having multiple locations will have different addresses. Same memory is having instructions and data, which has multiple locations with each location having unique address. A computer program consists of instructions and data. CPU process the data stored in the memory or obtained term input devices as per the instruction the program. The results are stored in the memory or output devices. The data processing operation in CPU use registers.

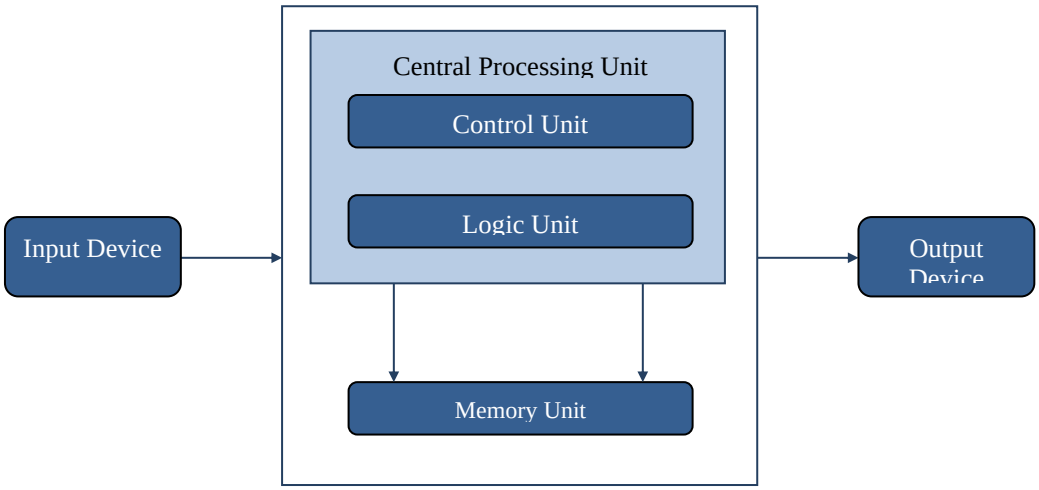


Figure 1.9: Von Neumann Machine Ref: www.educba.com

Buses (address bus / data bus / control bus) are used for communicating the address, data and control signals. The input device fetches data or commands and the Central processing unit (CPU) executes it. When the task is completed, the result is stored on output device.

As discussed in the previous section, a computer system executes instruction/commands using processor and memory while the registers provide the processor's temporary storage requirements, whereas memory works with medium-term and long-term data storage requirements. Any data processing system consists of a sequence of instructions that enable the processor to perform the functionality you want. These instructions operate on data, which may be input from input devices. The processed data is the output of the data processing systems. We may store instruction and data together or separately. In the Princeton architecture, data and instruction share the same memory space. In Harvard architecture, system or instruction memory space differs from data memory space. Princeton configuration can lead to easy hardware connection to memory, as only one connection is required. Harvard configuration requires, dual connections, can simultaneously read instruction and operand data, so it can lead to improved performance. Most machines have a Princeton design. Intel 8051 is a well-known Harvard architecture. Figure 1.10 is a diagrammatic representation of Harvard architecture, whereas figure 1.11 represents the Princeton architecture of a computer.

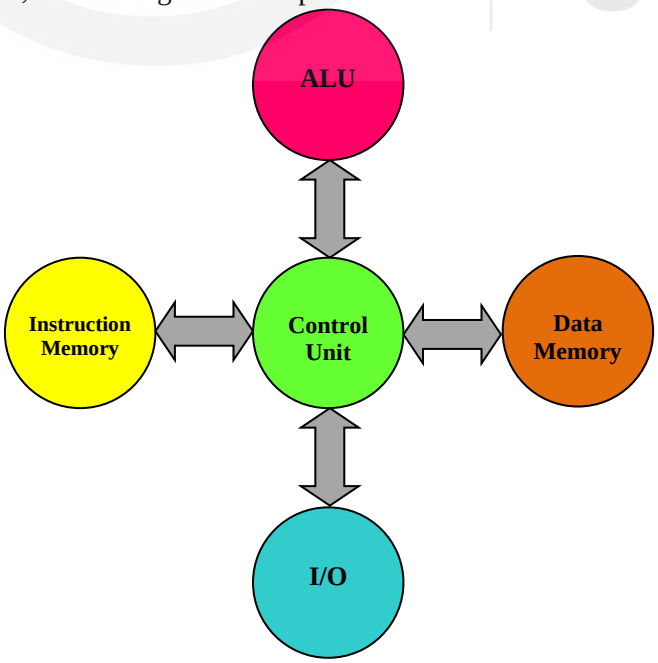


Figure 1.10: Harvard Architecture

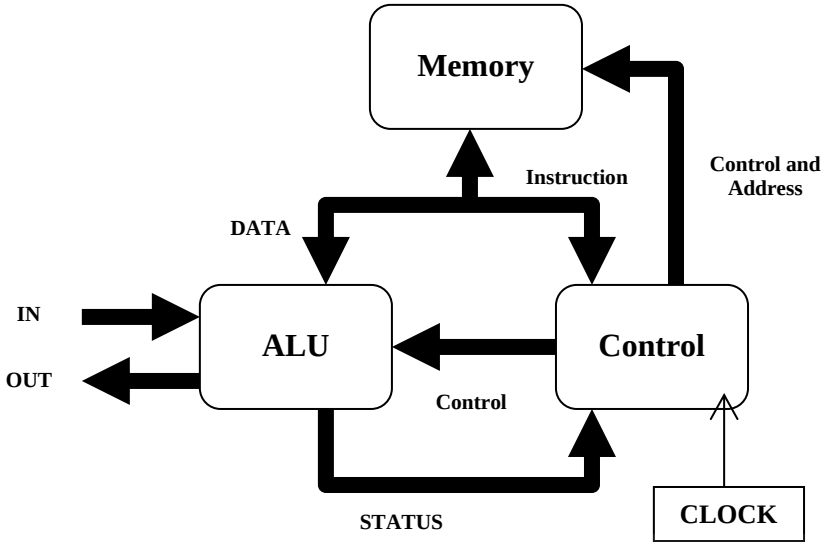


Figure 1.11: Princeton (von Neumann) Architecture

Stored System Computers –

Such computers can be programmed to do various tasks and also, applications are kept on them that is why they are named as Stored System Computers. This concept of a stored system was presented by John von Neumann. In such computer systems, a single memory is used to store programs and data and is treated in the same way. A computer built in this way will be easy to imitate.

The von Neumann architecture, also called as Princeton model, is a computer architecture designed by the 1945 definition of John von Neumann and others in the First Draft of a Report on the EDVAC. The draft defines the design of a digital computer. As per this document a general purpose computer should contain:

- a Processing unit comprising of arithmetic unit, logic unit and processor registers
- A control unit comprising of command register and a program counter register
- Data storage in data memory
- Limited external storage
- Input and output methods

The name "von Neumann architecture" has been attributed to any computer where the stored program and data are not transmitted simultaneously because they share the same medium of data transfer, called system bus. This is called a von Neumann bottleneck, as it restricts simultaneous access of data and instructions, thus, may result in reduction in the performance of the system. However, the single path simplifies the design of the von Neumann machine. In comparison, a Harvard architecture machine uses one dedicated bus each for address memory and data memory respectively, therefore, is more complex.

The basic characteristics of Von Neumann architecture are summarized below.

(1) A computer has the following operational units:

- The control unit (CU), which interprets commands to be executed, and generates control signal which instructs other units about how to perform the operation.

- Registers and other circuitry.
- Input/output system, which are used for input of instructions and output of results.
- A memory, which stores instructions as well as data (both).
- The inter connection structures, which allow communication between various components.

(2) von Neumann machine works on a concept of stored program, which requires that data and commands both must be loaded into the memory of the main computer prior to execution &called as Random Access Memory (RAM).

(3) The instruction/commands in a Von Neumann machine are executed in a sequence, therefore, to change this order of execution, a special instruction may be used.

(4) A Von Neumann machine has a single path from control unit to main memory so only one of two-data or instruction can be transferred between the two at a time.

1.6.2 Harvard Architecture

When data and instructions are kept in separate memory then it is Harvard architecture. Data is fetched from one memory location and instruction is retrieved from a separate location. Pipelining can be done in this architecture. It is complicated to design this architecture. The CPU can fetch, decode and execute instructions and data. The construction of this architecture has separate access for codes and data address.

The modified Harvard architecture is similar to the Harvard architecture and has a standard address space for a separate data and instruction cache. It has digital signal processors that can handle audio and video data efficiently. It also has microcontrollers, the processing circuits that process small number of applications and has small data memory and speed up processing by performing the commands and data access simultaneously.

Figure 1.12 shows different connection paths for modified Harvard architecture. All these four units are contained in CPU. It can perform simultaneous input / output operations and has a separate mathematical and logic component.

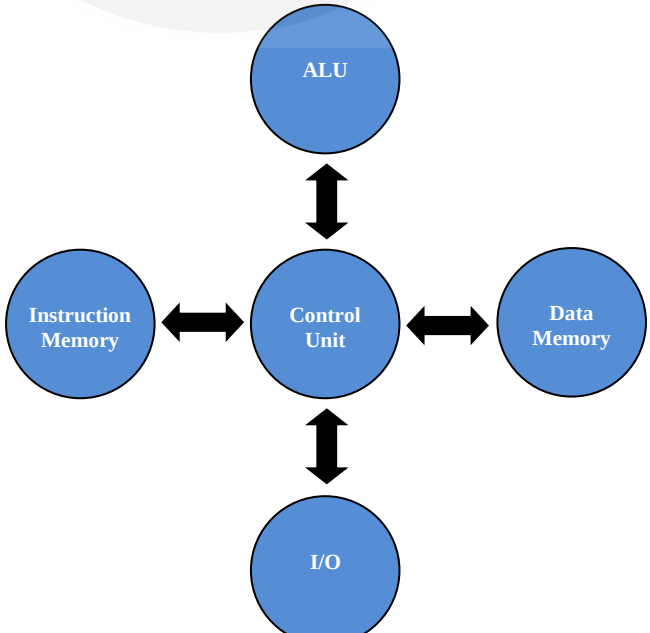


Figure 1.12: The modified Harvard architecture.

1.6.3 Instruction Set Architecture (ISA)

Instruction set architecture (ISA) defines a set of instructions that are to be supported by a processor. From system point of view, ISA does not care about certain computer implementation details. It is only about setting up or collecting the basic functions that a computer should support. Some of the common examples of ISA are Intel upgraded x86, ARM, MIPS, and AMD.

An ISA includes different kinds of instructions, sizes of differ instruction formats. These concepts will be explained in more details in the Block 3. The following example of MIPS ISA very briefly defines the description that should be supported by an ISA. You may refer to further readings for more details in this ISA.

1. ISA defines the types of commands that the processor will support.

Depending on class of work they are doing MIPS Instructions are divided into three types:

- Arithmetic / Logic Instructions
- Data transfer instructions
- Branch and Jump Instructions

2. Maximum length of each type of instruction has been defined in ISA.

3. I*nstruction format for each type of instruction has been defined in ISA.*

Various formats in MIPS ISA:

- R-Instruction format
- I-Instruction Format
- J-Instruction Format

As every format is having separate command coding schemes in terms of operation code, number of operands etc., so they are required to be understood differently by the processor.

The following diagram shows the hierarchy of abstraction of Architecture:

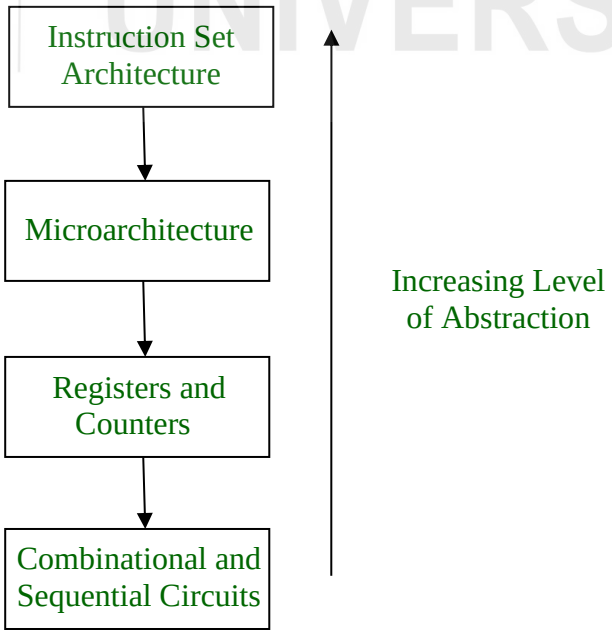


Figure 1.13: The Abstraction Hierarchy

As Micro-architectural standard is placed just below the ISA standard and is therefore deals with the implementation of computer-based core functions as suggested by ISA.

What is the need to differentiate between Micro-architecture and ISA? It is required to establish and maintain system compliance across all ISA-based hardware applications. Adapting different machines to the same set of basic commands (ISA) allows the same system to run smoothly on multiple different machines, thus, making it easier for editors to write and store code for many different machines simultaneously and efficiently.

This abstraction hierarchy supports flexibility, which is why first the ISA is developed and then various micro architectures are created that are compatible with this machine-operated ISA. The micro-architecture is implemented using various logic circuits.

1.6.4 RISC

RISC formulation is being used by ARM core. RISC is a strong design and it delivers simple commands in a single cycle with high clock speeds. RISC targets to reduce the hardware complexity of instructions because hardware has less flexibility in comparison to software. As a result, the structure of the RISC places expectations on the compiler. Conversely, complex instructions (CISC) rely heavily on hardware for operational performance, and as a result CISC commands are complex. Figure 1.14 shows this major difference.

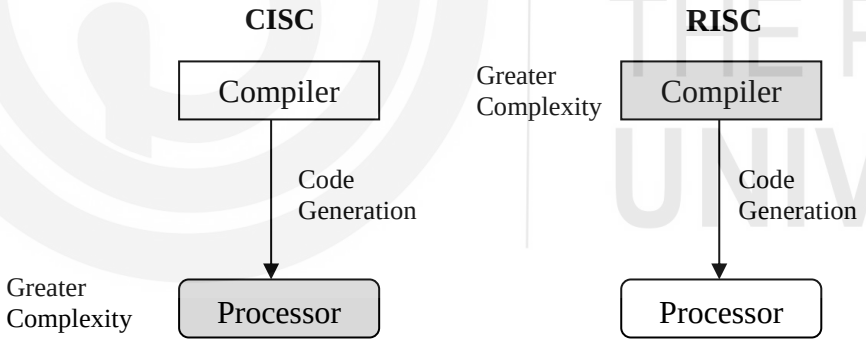


Figure 1.14: RISC vs. CISC

CISC emphasizes the complexity of hardware. The RISC emphasizes the complexity of the compiler.

The RISC philosophy is based on four main building codes:

1. *Instructions* - Number of instructions has been reduced in RISC. An instruction of RISC is executed in several segments. Instruction execution is overlapped in these segments (called pipeline segment). Each segment takes about one clock cycle time. For example, a processor performs complex tasks by combining several simple commands. Each command is of a limited length to allow the “pipeline to pick up future commands before deciding on current commands”. CISC instructions are usually of varied length.

2. *Pipes* - The processing of the instructions is divided into smaller units that can be made similar to pipes. Appropriately the pipe continues one step in each cycle of execution.
3. *Registers* – Large registers for general purposes have been included in RISC which may contain data or address. CISC processors have been provided with specific registers for specific tasks.
4. *Load-store creation* - The processor operates on data managed in registers. Memory access is expensive, so separating memory access to data processing provides an opportunity. With CISC design the data processing functionality can work in direct memory.

The RISC is explained in more details in Block 3.

1.6.5 Multiprocessor and Multicore Architecture

One of the ways to improve computer system efficiency is to have processors with faster clock speeds. But, owing to the thermal wall problem, when the clock speeds started hitting the thermal barrier, focus was on to extract maximum work out of the available system as an alternative of increasing processor speed.\

One of the ways to introduce parallelism in computers is to have more than one processor available in the single computer system referred to as multiprocessor system. It is very likely that at times the job (application) to be executed on a processor can be divided into various tasks with two or more tasks being capable of running independent of each other. Multiprocessor system corresponds to the architecture in which rather than having only one processor we have more than one processor available on the chip. Thus, if the job demanding execution comes with independent tasks, a processor can be dedicated to each task in order to exploit the parallelism in the job. In a way, multiprocessor system can be looked as a solution to offer hardware parallelism to match the available software parallelism in the job. This architecture corresponds to instruction level parallelism allowing independent instructions (or sub tasks) being assigned to different processors in the multiprocessor system allowing their parallel execution. If all the processors are same, the system is referred to as a Symmetric Multi-Processor (SMP). The multiprocessor architecture shares the computing environment viz. memory, OS, system clock etc.

In a conventional computer system, you had only one CPU available, which is responsible for the job execution focusing on only one task at a time. However, now a days, you hear about terms like a computer with quad core processor or an octa core processor. This architecture is referred to as the multicore architecture corresponding to both instruction level and thread level parallelism in a uniprocessor system. In this case, a single processor CPU is fabricated to have more than one CPU cores inside it. Each core acts an independent processing unit (CPU) and can execute independent threads, if permitted by the program. Thus, a quad core processor comprises for four cores whereas an octa core processor has eight cores capable of working independently on the same processor.

The difference between multiprocessor architecture and multicore architecture lies in the fact that multiprocessor architecture has more than one processor available whereas multicore processors have a single processor with multiple cores used as independent CPUs. Thus, multicore architecture is multiprocessing in a single packaged unit. Both multiprocessor and multicore architectures correspond to the MIMD category of Flynn's classification with multiprocessor architecture being more pure form of parallel processing. Practically, for improved performance, you can have multiprocessor machines with each processor having multiple cores.

1.6.6 Mobile Architecture

Nowadays, mobile handsets are no longer a means of enabling conversations but have turned into a small but powerful computer having many features like fast memory, support for software for numerous applications like chatting, document editing, entertainment, news etc. These phones have become really handy equipped with efficient performance and it has become really difficult to have a demarcation between a computer and a mobile phone.

Just like any computer system, mobiles too have an input unit, a Central Processing Unit (CPU) and an output unit as shown in Figure 1.15. The input unit could be a keyboard based or a touch screen based input mechanism and the output unit can be the screen or audio or video system. However, the CPU in the case of mobiles can be considered to be having two processing units viz. Communications Processing Unit and an Applications Processing Unit to cater to mobile calls and handling various applications available in the form of Apps. In addition, some other functional units like Display management, Memory management, Power management, Data management can be considered to be associated with the mobile system architecture but the discussion is beyond the scope of this course. All these units, under the mobile CPU can be understood to be working under the control of a mobile Control Unit for control and synchronization purposes.

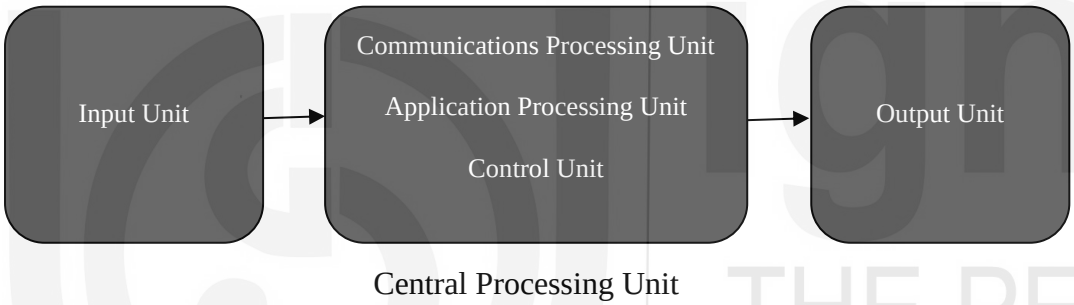


Figure 1.14: Block diagram of Mobile System

Check your progress 3:

Q1: Differentiate between the micro-architecture and ISA.

.....

.....

.....

.....

Q2: Differentiate von Neumann architecture and Harvard architectures.

.....

.....

.....

.....

Q3: Differentiate multiprocessor and mullticore processor

.....

.....

1.7 SUMMARY

This unit introduces you to some of the basic architectural features of a computer system. A computing device consists of some basic units, like processing unit, which includes control unit and arithmetic logic unit, memory, input and output devices and data paths. This unit also explains the concepts of various computer architecture, which represents, how these various component of computer can be used to execute an instruction set. The unit also introduces to the concept of ISA, multiprocessor and multicore and mobile architecture. A detailed discussion on the various aspects of Computer organization has not been included in this unit, they will be discussed in the subsequent blocks and units. This unit also introduces you to the concept of instruction and its execution by a computer.

1.8 SOLUTIONS/ANSWERS

Check Your Progress 1

1) (a) True (b) True (c) False (d) True

2) An instruction in a computer system is a binary code, which is interpreted by the control unit of a computer and it communicates the following information to CPU:

- the operation which is to performed
- the operands and their location
- the possible information of storage of results

etc.

3) The CU controls - (1) the sequence of instruction execution, (2) the communication through the data paths, (3) the communication among different units (4) the operation to be performed by ALU (5) what to do in case of errors etc. Thus, in general, CU is responsible for complete control of a computer

The ALU primarily performs the arithmetic and logical and shift operations on the specified data. Memory stores the data as well as instructions and I/O devices are used to input data or instructions as well as display of results.

Check Your Progress 2

1) (i) True (ii) False (iii) False (iv) False (v) True

2) An interrupt is the process of stopping the execution of currently executing program due to occurrence of some event, which requires attention of the CPU. The causes of the interrupt may be division by zero; arithmetic overflow, program address space violation, starting or completion of I/O, errors etc.

3) Interrupt can be processed by suspending the execution of currently executing program and executing the ISR of the interrupting event. It may be noted that interrupts can be acknowledged only if the interrupts are enabled.

Check your progress 3:

1) An ISA defines the set of instructions, but a micro-architecture provides details on how various functions will be performed by various units. An ISA is a top level design, whereas micro-architecture is a detailed design. A faster compatible computer can be designed with same ISA but more efficient

micro-architecture. ISA simplifies the job of programmers, who may use same instruction set to write programs.

2) The von Neumann architecture uses same memory for data and instruction, the Harvard architecture may have separate memories for instructions and data. In von Neumann architecture data and instructions cannot be accessed simultaneously but in Harvard architecture it is possible. The von Neumann architecture is simpler to implement in comparison to Harvard architecture.

3) A multiprocessor system may have multiple processors, whereas multicore processor have multiple CPUs in a single processor. Today, a multiprocessor system can be constructed using multicore processor chips. Both these technology are of the type MIMD, which is multiple instruction and multiple data form of multiprocessing, thus, allow multiple sections of programs being executed at the same time operating on separate data.

ignou
THE PEOPLE'S
UNIVERSITY

UNIT 2 DATA REPRESENTATION

Structure **Page Nos.**

2.0 Introduction
2.1 Objectives
2.2 Data Representation in Computer
2.3 Representation of Characters
2.4 Number Systems
2.5 Negative Number Representation Using Complements
2.5.1 Fixed Point Representation
2.5.2 Binary Arithmetic using Complement notation
2.5.3 Decimal Fixed Point Representation
2.6 Floating Point Representation
2.7 Error Detection and Correction Codes
2.8 Summary
2.9 Solutions/ Answers

2.0 INTRODUCTION

In the first Unit of this Block, you have learnt the concepts relating to different architectures of a Computer System. It also explains the process of execution of an instruction highlighting the use of various components of a Computer system. This Unit explains about how the data is represented in a computer system.

The Unit first defines the concepts of number systems in brief, which is followed by discussion on conversion of numbers of different number systems. An important concept of signed complement notation, which is used for arithmetic operations on binary numbers, has been explained in this Unit. This is followed by discussion on the fixed point and floating point numbers, which are used to represent the numerical data in computer systems. This Unit also explains the error detection and correction codes and introduces you to basics of computer arithmetic operations.

2.1 OBJECTIVES

At the end of the unit you will be able to:

- Represent numeric data using binary, octal and hexadecimal numbers;
- Perform number conversions among various number bases;
- Define the ASCII and UNICODE representation for a character set;
- Explain the fixed and floating point number formats;
- Perform arithmetic operations using fixed and floating point numbers ; and
- Explain error detection and correction codes

2.2 DATA REPRESENTATION IN COMPUTER

A computer system is an electronic device that processes data. An electronic device, in general, consists of two stable states represented as 0 and 1. Therefore, the basic unit of data on a computer is called a **B**inary Dig**it** or a **Bit.** With the advances in quantum computing technology a new basic unit called Qubits has emerged, which also represent 0 and 1, but with the difference that it can also represent both the states at the same time. The concepts of Quantum computing is beyond the scope of this unit.

A computer performs three basic operation on data, viz. data input, processing and data output. The data input and information output, in general, is presented in text, graphics, audio or other human recognizable form. Therefore, all human readable characters, graphics, audio and video should be coded using bits such that computer is able to interpret them. The most common code to represents characters into computer are ASCII and UNICODE. Pictures and graphs can be represented using pixel (picture elements), digital sound and video are represented by coding the frames in digital formats. Since graphics, digital audio and digital video, which are stored on storage devices as files, are very large in size, therefore, a large number of storage formats that use data compression techniques are used for represent digital information. Some of these concepts are explained in Unit 8.

The numeric data is used for computation in computer. However, as computer is an electronic device, it can only process binary data. Thus, in general, numeric data is to be converted to binary for computation. Computer uses fixed point and floating point representation for representing numeric data. Data in computer is stored in random access memory (RAM) and is required to be transferred in or out of the RAM for the purpose of processing, therefore, an error detection mechanism may be employed to identify and correct simple errors while transfer of binary data. The subsequent sections of the Unit explains the character representation, representation of binary numbers and error detection mechanism.

2.3 REPRESENTATION OF CHARACTERS

A character can be presented in a computer using a binary code. This code should be same for different types of computers, else the information from one computer will not be transferable to other computers. Thus, there is need of a standard for character representation. A coding standard has to address two basic issues, viz. the length of code, and organisation of different types of characters, which include printable character set of different languages and special characters. Two important character representation schemes are ASCII and UNICODE, which are discussed next.

American Standard Code for Information Interchange (ASCII)

ASCII was among the first character encoding standard. The length of ASCII is 7-bit. Thus, it can represent 2^7=128 characters. It represents printable characters - English alphabets (both lower case and upper case), decimal digits, special characters as present on the present day keyboard, certain graphical characters etc.; and non-printable control characters. American National Standards Institute (ANSI) has designed a standard ISO 646 in 1964, which is based on ASCII.

However, as the basic unit of computer storage was 8, 16, 32 or 64 bits, the ASCII was extended to create an 8 bit code. This code could represent 2^8=256 characters, most of the additional characters in extended code were graphics characters. ANSI has designed a code ISO 8859 for extended ASCII. It may be noted that ASCII has many variants, which are based on characters used in different countries.

In ASCII, the coding sequence of characters is very interesting. Simple binary arithmetic operations can result in conversion of lower case to upper case characters. For example, character 'A' in ASCII is represented as the binary value 100 0001, which is equivalent to a value 65 in decimal notation, whereas the character 'a' is stored as binary value 110 0001, which is the decimal 97. Thus, conversion from lowercase to upper case and vice-versa may be performed by subtracting or adding 32, as the case may be.

ISO 8859 which is based on extended ASCII is a good representation; however, all the languages cannot be represented using ASCII as its length is very small. Therefore, a

new standard that could represent almost all the characters of all the languages was developed. This is called the UNICODE.

Unicode

Unicode is a standard for character representation, which provides a unique code also called *code point*, for every character of almost all the languages of the world. The set of all the codes is called *code space*. The code space is divided into 17 continuous sequences of codes called *code planes*, with each code plane can represent 2^{16} codes. Thus, Unicode values ranges from $U+0000_{16}$ to $U+10FFFF_{16}$. Here U+ represents the Unicode followed by the hexadecimal value of a code point. The code planes of the Unicode being $U+00000_{16}$ to $U+0FFFF_{16}$; $U+10000_{16}$ to $U+1FFFF_{16}$; $U+20000_{16}$ to $U+2FFFF_{16}$; … , $U+F0000_{16}$ to $U+FFFFF_{16}$; and $U+100000_{16}$ to $U+10FFFF_{16}$. You can learn about more details on Unicode from the further readings. Also read the hexadecimal number system given in the next section to learn about the hexadecimal values given above.

One of the major advantages of using Unicode is that it helps in seamless digital data transfer among the applications that use this character formatting, thus, not causing any compatibility problem.

Unicode code points may consist of about 24 binary digits, however, all of these code points may not be required for a given set of data. In addition, a digital system requires the data in the units of bytes. Thus, a number of encodings has been designed to represent Unicode code points in a digital format. Two of these popular encodings - Unicode Transformation Formats are UTF-8 and UTF-16. UTF-8 uses 1 to 4 bytes to represent the code points of Unicode. Most of the 1 byte UTF-8 code points are compatible to ASCII. UTF-16 represents code points as one or two 16-bit code units. The standard ISO 10646 represents various Unicode coding formats.

In general, if you are working with web pages having mostly English language, UTF-8 may be a good choice of character representation. However, if you are creating a multi-lingual web page, it may be a good idea to use UTF-16.

Indian Standard Code for information interchange (ISCII)

The ISCII is and ASCII compatible code consisting of eight-bits. The code for values 0 to 127 in ISCII is similar to ASCII; however, for the values 128 to 225 it represents the characters of Indian scripts. IS 13194:1991 BIS standard defines the details of ISCII. However, with the popularity of Unicode, its use has now been limited.

2.4 NUMBER SYSTEMS

A number system is used to represent the quantitative information. This section discusses the binary, octal and hexadecimal number systems

Formally, a number system is represented using a base or radix, which is equal to the distinct digits used by that system, and the position of a digit in a number. For example, the decimal number system has a base 10. It consists of ten decimal digits, viz. 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9; and a place value as shown in the following example:

$$9\times10^4 + 8\times10^3 + 7\times10^2 + 6\times10^1 + 5\times10^0 + 4\times10^{-1} + 3\times10^{-2} + 2\times10^{-3} + 1\times 10^{-4}$$
$$= 98765.4321$$

Binary Numbers: A binary number system has a base 2 and consists of only two digits 0 and 1, which are also called the bits. For example, 1001_2 represent a binary number with four binary digits. The subscript 2 represents that the number 1001 has a base 2 or in other words is a binary number.

Note: The subscript shown in the numbers represents the base of the number. In case a subscript is not given then please assume it as per the context of discussion.

Conversion of binary number to Decimal equivalent:

A binary number is converted to its decimal equivalent by multiplying each binary digit by its place value. For example, a seven digit binary number 1001001_2 can be converted to decimal equivalent value as follows:

Binary Digits of Number	1	0	0	1	0	0	1
The place value	2^6 =64	2^5 =32	2^4 =16	2^3 =8	2^2 =4	2^1 =2	2^0 =1
Binary digit × Place value	1×64	0×32	0×16	1×8	0×4	0×2	1×1
Computed values	64	0	0	8	0	0	1
Sum of the computed values	64+0+0+8+0+0+1 = 73 in Decimal						

You may now try converting few more numbers. Try 0010001, which will be 16+1=17; 1111111 will be 64+32+16+8+4+2+1=127. So a 7 bit binary number can contain decimal values from 0 to 127.

Octal Numbers: An Octal number system has a base of 8, therefore, it has eight digits, which are 0,1,2,3,4,5,6,7. For example, 76543210_8 is an octal number.

Conversion of Octal number to Decimal equivalent:

An Octal number is converted to its decimal equivalent by multiplying each octal digit by its place value. For example, an octal number 5432_8 can be converted to decimal equivalent value as follows:

Octal Digits of Number	5	4	3	2
The place value	8^3 =512	8^2 =64	8^1 =8	8^0 =1
Octal digit × Place value	5×512	4×64	3×8	2×1
Computed values	2560	256	24	2
Sum of the computed values	$2560+256+24+2=2842_{10}$			

Hexadecimal Numbers: A hexadecimal number system has a base of 16, therefore, it uses sixteen digits, which are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A(10), B(11), C(12), D(13), E(14), F(15). For example, $FDA9_{16}$ is a hexadecimal number.

Conversion of Hexadecimal number to Decimal equivalent:

A hexadecimal number is converted to its decimal equivalent by multiplying each hexadecimal digit by its place value. For example, a hexadecimal number $13AF_{16}$ can be converted to decimal equivalent value as follows:

Hexadecimal Digits of Number	1	3	A=10	F=15
The place value	16^3 =4096	16^2 =256	16^1 =16	16^0 =1
Hexadecimal digit × Place value	1×4096	3×256	10×16	15×1
Computed values	4096	768	160	15
Sum of the computed values	$4096+768+160+15=5039_{10}$			

Conversion of Decimal to Binary: A decimal number can consists of Integer and fraction part. Both are converted to binary separately.

Process:

For Integer part: Repetitively divide the quotient of integer part by 2 keeping remainder separate till quotient is 0. Collect all the remainders from last remainder to first to make equivalent binary

For Factional part: Repetitively multiply the fraction by 2 and maintain the list of integer value that is obtained till fraction becomes 0. Collect all the integer values.

The following example explains the process of Decimal to binary conversion.

Example 1: Convert the decimal number 22.25 to binary number.

Solution:

For Integer part: Repetitively divide the quotient of integer part by 2 keeping remainder separate till quotient is 0. Integer value of example: 22				**For Factional part:** Repetitively multiply the fraction by 2 and maintain the list of integer value that is obtained till fraction becomes 0. Fraction value of example:.25			
Integer Part	**After Division by 2**		**Direction of Reading the Result**	**Fraction Part**	**After multiplication by 2**		**Direction of Reading the Result**
	Quotient	**Remainder**			**Result**	**Integer part**	
22	11	0	↑	.25	0.50	0	↓
11	5	1		.50	1.00	1	
5	2	1		.00	**STOP**		**Ans: 01**
2	1	0					
1	0	1					
0	**STOP**		**Ans: 10110**				

22.25_{10} in binary is 10110.01_2

Verification

Binary Digits of Number	**1**	**0**	**1**	**1**	**0**	**.**	**0**	**1**
The place value	2^4 =16	2^3 =8	2^2 =4	2^1 =2	2^0 =1		2^{-1} =1/2	2^{-2} =1/4
Digit × Place value	1×16	0×8	1×4	1×2	0×1	.	0×0.5	1×0.25
Computed values	16	0	4	2	0		0	0.25
Sum of the computed values	22.25_{10}							

--

The method as shown above is a standard technique. You must start using this method for various problems. However, you may use the following simple technique of converting integer part of decimal numbers to binary.

Conversion from Decimal to Binary a simpler Process:

Assume a decimal number *N* is to be converted to binary. Now perform the following steps:

1. Is the decimal number *N* equal to a binary place value, then assign that place value to *P* and move to step 3.
2. Else, find the binary place values which is just lower to the decimal number *N*. Assign this place value to *P*. For example, for number 73 just lower place value is 64, as $2^6 = 64$ and $2^7=128$.
3. Put 1 in the position of *P* and subtract the place value *P* from *N*.

4. If $(N\text{-}P) \neq 0$ then Repeat the steps 1 to 3 by taking new $N=N\text{-}P$
5. Put 0 in all the remaining places, where 1 has not been put.

The following example demonstrate this technique showing conversion of 73 to binary.

Example 2: Convert decimal numbers 73, 39 and 20 into binary using the method as above.

The following table shows the process of the conversion.

The place value	2^6 =64	2^5 =32	2^4 =16	2^3 =8	2^2 =4	2^1 =2	2^0 =1
***N* = 73**	128 > 73 > 64, therefore, *P*=64						
Step 2 and 3	1	*N* = *N-P* = 73-64 =9 (Not 0 so repeat)					
Step 2	16 > 9 > 8 so new *P*=8 and New *N*=9-8=1						
Step 3	1			1			
Step 1	Since 1 is a place value, new *P*=1 and New *N* =1-1=0						
Step 3	1			1			1
Step 4	**1**	**0**	**0**	**1**	**0**	**0**	**1**
Place values	64	32	16	8	4	2	1
N=39		1					
New *N=7*		1			1		
New *N=3*		1			1	1	
New *N=1*		1			1	1	1
Step 4	**0**	**1**	**0**	**0**	**1**	**1**	**1**
Place values	64	32	16	8	4	2	1
N=20			1				
New *N=4*			1		1		
Step 4	**0**	**0**	**1**	**0**	**1**	**0**	**0**

The logic as presented here can be extended to the fractional part, however, it is recommended that you may follow the repeated multiplication method as explained earlier for the fractions.

Conversion of Binary number to Octal Number

The base of a binary number is 2 and the base of octal number is 8. Interestingly, $2^3=8$. Thus, if you simply group three binary digits, the equivalent value may form the octal digit. However, you may be wondering how to group binary numbers. This is explained with the help of following example.

Example 3: Convert the binary 11001101.00111_2 into equivalent Octal number.

Process: The process is to group three binary digits. The grouping before the binary point is done from right to left and after the binary point from left tonright. Each of the group then is converted to equivalent octal digit. The following table shows this conversion process.

Binary Number	-	1	1	0	0	1	1	0	1	.	0	0	1	1	1	-
Grouping Directions	◄	◄	◄	◄	◄	◄	◄	◄	◄	.	►	►	►	►	►	►
Grouped (- replaced by 0)	0	1	1	0	0	1	1	0	1	.	0	0	1	1	1	0
Binary place values	4	2	1	4	2	1	4	2	1	.	4	2	1	4	2	1
Equivalent Octal Digit	0+2+1=3			0+0+1=1			4+0+1=5			.	0+0+1=1			4+2+0=6		
Octal Number	**3**			**1**			**5**			**.**	**1**			**6**		

Therefore, 11001101.00111_2 is equivalent to **315.16_8**

Conversion of Binary number to Hexadecimal Number

The base of a binary number is 2 and the base of hexadecimal number is 16. You may notice that 2^4=16. Therefore, conversion of binary to hexadecimal notation may require grouping of 4 binary digits. This is explained with the help of following example.

Example 4: Convert the binary 11001101.00111_2 into equivalent hexadecimal number.

Process: The process is almost similar to binary number to octal number conversion expect now four binary digits are combined as given in the following table.

Binary Number	1	1	0	0	1	1	0	1	.	0	0	1	1	1	-	-	-
Grouping Direction	←									.	→						
Grouped	1	1	0	0	1	1	0	1	.	0	0	1	1	1	0	0	0
Binary place values	8	4	2	1	8	4	2	1	.	8	4	2	1	8	4	2	1
Hexadecimal digit	8+4+0+0=12				8+4+0+1=13				.	0+0+2+1=3				8+0+0+0=8			
Hexadecimal	12 is C				13 is D				.	3				8			

Therefore, **11001101.00111_2** is equivalent to **315.16_8** and **$CD.38_{16}$**

As computer is a binary device, therefore, all the numbers of different number systems may be represented in binary format. This is shown in the following table.

Decimal Number	Binary Coded Decimal	Equivalent Octal Number	Binary coded Octal	Hexadecimal Number	Binary-coded Hexadecimal
0	0000	0	000	0	0000
1	0001	1	001	1	0001
2	0010	2	010	2	0010
3	0011	3	011	3	0011
4	0100	4	100	4	0100
5	0101	5	101	5	0101
6	0110	6	110	6	0110
7	0111	7	111	7	0111
8	1000	10	001 000	8	1000
9	1001	11	001 001	9	1001
10	0001 0000	12	001 010	A	1010
11	0001 0001	13	001 011	B	1011
12	0001 0010	14	001 100	C	1100
13	0001 0011	15	001 101	D	1101
14	0001 0100	16	001 110	E	1110
15	0001 0101	17	001 111	F	1111
16	0001 0110	21	010 000	10	0001 0000
17	0001 0111	22	010 001	11	0001 0001
…					
49	0100 1001	61	110 001	31	0011 0001
…					
63	0110 0110	77	111 111	3F	0011 1111

Table 1: Decimal, Octal, Hexadecimal Numbers

Please note the following points in the Table 1 given above.

- The Binary coded decimal (BCD) is the representation of each decimal digit to a sequence of 4 bits. For example, a decimal number 12 in BCD is 0001 0010. This representation is used in several calculators for performing computation.
- It may be noted that BCD is not binary equivalent value. For example, the BCD value of decimal 49 is 0100 1001 but its binary equivalent value is 0011 0001.
- Please also note that binary coded hexadecimal values are equivalent to binary value of a number. For example, decimal value 63 in hexadecimal binary notation is 0011 1111, which is same as its binary value.

The conversion of decimal to octal and hexadecimal may be performed in the same way as done using repeated division or multiplication of binary. The process is exactly same except, in decimal number to octal or hexadecimal number conversion division is done by 8 or 16 respectively.

Check Your Progress 1

1) Perform the following conversions:

   i) 11100.01101_2 to Octal and Hexadecimal

   ii) 1101101010_2 to Octal and Hexadecimal

   .....

   .....

   .....

2) Convert the following numbers to binary.

   i) 119_{10}

   ii) 19.125_{10}

   iii) 325_{10}

   .....

   .....

   .....

3) Convert the numbers to hexadecimal and octal.

   i) 119_{10}

   ii) 19.125_{10}

   iii) 325_{10}

   .....

   .....

   .....

2.5 NEGATIVE NUMBER REPRESENTATION USING COMPLEMENTS

You have gone through the details of binary representation of character data and the number systems. In general, you use positive and negative integers and real numbers for computation. How these numbers can be represented in binary? This section describes how positive and negative numbers can be represented in binary for performing arithmetic operations.

In general, Integer numbers can be represented using the sign and magnitude of the number, whereas real numbers may be represented using a sign, decimal point and magnitude of integral and fractional part. Real numbers can also be represented using a scientific exponential notation. This section explains how integers can be represented as binary numbers in a computer system.

Integer representation in binary:

An integer is represented in binary using fixed number of binary digits. One of the simplest representations for representing integer would be - to represent the sign using a bit; and magnitude may be represented by the remaining bits. Fortunately, the value of sign can be either positive or negative, therefore, it can easily be represented in binary. The + sign can be represented using 0 and – sign can be represented using 1.

For example, a decimal number 73 has a sign + (bit value 0) and magnitude 73 (binary equivalent 1001001). The following table shows some of the numbers using this *Sign-magnitude Representation*:

Number	Sign Bit	Magnitude						
+73	0	1	0	0	1	0	0	1
-73	1	1	0	0	1	0	0	1
+39	0	0	1	0	0	1	1	1
-39	1	0	1	0	0	1	1	1
+127	0	1	1	1	1	1	1	1
-127	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0
-0	1	0	0	0	0	0	0	0

Table 2: 8-bit Sign-Magnitude representation

Please note the following points about this 8 bit sign-magnitude representation:

- It represents number in the range 0 to +127 and -0 to -127. Therefore, the range of the numbers that can be represented using these 8 bits is -127 to +127.
- It represents 255 different numbers viz. -127 to -1, ±0 and +1 to +127.
- The number of bits, which are used to represent the magnitude of the number, can be used to determine the maximum and minimum numbers that are representable.
- There are two representations of zero +0 and -0

Is this representation suitable for representing numbers for computation? It has one basic problem that the sequences of steps to perform arithmetic operations are not straight forward. For example, for adding +73 and -39, first you need to compare the sign of the numbers, and as the signs are different in this case, therefore, you should perform subtraction of smaller number from the bigger number and finally assigning the sign of bigger number to the result.

Is there any better representation? Yes, an interesting representation that uses complement of a number to represent negative numbers has been designed. What is a complement of a number?

Complement notation: A complement, by definition, is a number that makes a given number complete. For the decimal numbers, this completeness can be defined with respect to the highest value of the digit, i.e. 9 or the next higher value, i.e. 10. These are called 9's and 10's complement respectively for the decimal numbers.

For example, for a decimal digit 3, the 9's complement would be 9-3 =6 and 10's complement would be 10-3=7.

In general, for a number with base *B* two types of complements are defined –(*B*-1)'s complement and *B*'s complement. For example, for decimal system base value *B* is10. Therefore, for decimal numbers two complements, viz. 9's and 10's complements, are defined. Thus, for binary system where base is 2, the two complements, viz. 1's complement and 2's complement, are defined. The following example illustrates the steps of finding 9's and 10's complement for decimal numbers.

Example 5: Compute the 9's complement and 10's complement for a four digit decimal number 1095, 8567 and 0560.
Solution: Following table shows the process:

Complement	Operation	The Number			
9's Complement	Number	1	0	9	5
	Subtract each digit from 9	8	9	0	4
10's Complement	Add 1 in the 9's complement	-	-	-	1
	It results in 10's complement	8	9	0	5
9's Complement	Number	8	5	6	7
	Subtract each digit from 9	1	4	3	2
10's Complement	Add 1 in the 9's complement	-	-	-	1
	It results in 10's complement	1	4	3	3
9's Complement	Number	0	5	6	0
	Subtract each digit from 9	9	4	3	9
10's Complement	Add 1 in the 9's complement	-	-	-	1
	It results in 10's complement	9	4	4	0

Table 3: Computation of 9's and 10's complement

Please note that the sum of the number and its 9's complement for the numbers of 4 digits is 9999, and the sum of the number and its 10's complement is 10000. The 9's and 10's complement of the numbers can be used in a computer system when BCD numbers are used instead of binary numbers. Similarly, the 1's and 2's complement can be computed for binary numbers. The following example demonstrates the complement notation in binary.

Example 6: Compute the 1's and 2's complement for the binary numbers 1001_2, 1111_2 and 0000_2 using representation, which has four bits.

Solution:

Solution: Following table shows the process:

Complement	Operation	The Number			
1's Complement	Number	1	0	0	1
	Subtract each digit from 1	0	1	1	0
2's Complement	Add 1 in the 1's complement	-	-	-	1
	It results in 2's complement	0	1	1	1
1's Complement	Number	1	1	1	1
	Subtract each digit from 1	0	0	0	0

2's Complement	Add 1 in the 1's complement	-	-	-	1
	It results in 2's complement	0	0	0	1
1's Complement	Number	0	0	0	0
	Subtract each digit from 1	1	1	1	1
2's Complement	Add 1 in the 1's complement	-	-	-	1
	It results in 2's complement. There will be a carry bit from the last digit	0	0	0	0

Table 4: Computation of 1's and 2's complement

Please note the following in the table as above:

- On subtracting 1 from binary digit will result in change of bit from 0 to 1 OR 1 to 0.
- When you add binary digit 1 with 1, then it results in a sum bit of 0 and carry bit as 1.
- 1's complement of 0000 is 1111, when 1 is added to it, you will get 10000 as the 2's complement. Since, only 4 binary digits are used in the notation as above, the fifth digit, which is 1, is ignored while taking the complement.

An interesting observation from the Table 4 is that 1's complement can be obtained simply by changing 1 to 0 and 0 to 1. For obtaining 2's complement leave all the trailing zeros and the first 1 intact and after that complement the remaining bits. For example, for an eight bit binary number 10101100, the complement can be done as follows:

Number	1	0	1	0	1	1	0	0
1's Complement change every bit from 0 to 1 OR 1 to 0	0	1	0	1	0	0	1	1
Number	1	0	1	0	1	1	0	0
For 2's complement leave the trailing 0's till first 1						1	0	0
then complement remaining bits(change 0 to 1 or 1 to 0)	0	1	0	1	0			
2's Complement of the Number	0	1	0	1	0	1	0	0

Table 5: Computation of 1's and 2's complement

But, how are these complement notations used in a computer system to represent integers? The next sub-section explains the integral representation of computers.

2.5.1 Fixed Point Representation

A computer system uses registers or memory locations to store arithmetic data like numbers. The number stored in these locations are of fixed size, such as 8 or 16 or 32 or 64 or 128 bits etc. Interestingly, binary point is not represented in the numbers, rather its location is assumed. The fixed point number representation assumes that the binary point is at the end of all the binary digits, thus, can be used to represent integers. Since, Integers include positive and negative number both, therefore, fixed point number also use one bit as the sign bit as shown in Table 2. Fixed point numbers may use either signed magnitude notation or complement notation. However, as explained in the previous section signed magnitude notation is not a natural notation for binary arithmetic, the complement notation is used in computers. The complement notation works well for the digital binary numbers as they are of fixed length. For the sake of simplicity, in this unit we will use an complement notation having a length of 8-bits.

For the fixed point number representation signed 1's complement and signed 2's complement notation can be used. The signed complement notation is same as the complement notation as introduced in the previous section, except that it uses a sign bit, in addition to represent magnitude. In signed 1's and 2's complement notation the positive number has the same magnitude as that of binary number with the sign bit as zero, however, the negative numbers are represented in complement form. The

following example explains the process of conversion of decimal numbers to signed 1's or signed 2's complement notation.

Example 7: Represent the +73, -73, +39, -39, +127, -127 and 0 using signed 1's complement notation.

Solution: The table 6 shows the values in signed 1's complement notation of length 8 bits (S is the sign bit). Please note that even in signed 1's complement notation there are two representations for 0. The number range for 1's complement for this 8 bit representation is -127 to -0 and +0 to +127. So it can represent 2^8-1 (as two representation of 0) =255 numbers.

Number	Process	S	7 bits						
+73	Sign is 0 (positive) and 7 bit magnitude is same as binary equivalent value of 73	0	1	0	0	1	0	0	1
-73	**Take 1's complement of all the 8 bits (including sign bit) to obtain -73**	1	0	1	1	0	1	1	0
+39	Follow same process as stated for +73	0	0	1	0	0	1	1	1
-39	Follow same process as stated for -73	1	1	0	1	1	0	0	0
+127	Follow same process as stated for +73	0	1	1	1	1	1	1	1
-127	Follow same process as stated for -73	1	0	0	0	0	0	0	0
0	Follow same process as stated for +73	0	0	0	0	0	0	0	0
-0	Follow same process as stated for -73	1	1	1	1	1	1	1	1

Table 6: 8-bit Signed 1's complement notation

Example 8: Represent the +73, -73, +39, -39, +127, -127, 0 and -128 using signed 2's complement notation.

Solution: The table 7 shows the values in signed 2's complement notation of length 8 bits (S is the sign bit). Please note that in signed 2's complement notation there is a unique representations for 0, therefore, -128 can also be represented. Thus, the range of the number that can be represented using signed 2's complement notation is -128 to +127. Thus, a total of 256 numbers can be represented using signed 2's complement notation.

Number	Process	S	7 bits						
+73	Sign is 0 (positive) and 7 bit magnitude is same as binary equivalent value of 73	0	1	0	0	1	0	0	1
-73	**Take 2's complement of the number (including sign bit) to obtain -73**	1	0	1	1	0	1	1	1
+39	Follow same process as stated for +73	0	0	1	0	0	1	1	1
-39	Follow same process as stated for -73	1	1	0	1	1	0	0	1
+127	Follow same process as stated for +73	0	1	1	1	1	1	1	1
-127	Follow same process as stated for -73	1	0	0	0	0	0	0	1
0	Follow same process as stated for +73	0	0	0	0	0	0	0	0
-0	Follow same process as stated for -73	0	0	0	0	0	0	0	0
-128	-127-1 is = -128	1	0	0	0	0	0	0	0

Table 7: 8-bit Signed 2's complement notation

In general, a signed 2's number representation of n bits can represent numbers in the range -2^{n-1} to + (2^{n-1}-1). Therefore, an 8 bit representation can represent the numbers in the range -2^{8-1} to + (2^{8-1}-1); i.e. -2^7 to +(2^7-1), which is -128 to +127. For a 16 bit representation this range will be -2^{15} to +(2^{15}-1), which is -32768 to +32767. Please relate these ranges to range given in programming languages like C.

Signed 2's complement notation is one of the best notation to perform arithmetic on numbers. Next, we explain the process of performing arithmetic using fixed point numbers.

2.5.2 Binary Arithmetic using Complement notation

In this section, we discuss about binary arithmetic using fixed point complement notation.

Arithmetic addition: Arithmetic addition operation can be performed using any of the signed-magnitude, singed 1's complement and signed 2's complement notation.

Addition using signed-magnitude notation:

The process of addition of two numbers using signed magnitude notation will require the following steps:

Step 1: Check if the numbers have similar or different signs.

Step 2: If signs are same then just add the two numbers, otherwise identify the number having bigger magnitude (in case both numbers have same magnitude then first number may be assumed as bigger number) and subtract the smaller number from bigger number.

Step 3: If signs of the number are same then check if the result exceeds the size of number of bits of the representation, if that happens then report overflow of number. For the numbers with different signs overflow cannot occur.

Step 4: The sign of the final number is the sign of any operand, if signs are same; or the sign of the bigger number, if signs are different.

The following example explains the process of addition using signed-magnitude notation. The example, uses an 8 bit representation.

Example 9: Add the decimal numbers 75 and -80 using signed magnitude notation, assuming the 8-bit length of the notations.

Solution: The numbers are (The left most bit is the Sign bit):

Number	Signed Magnitude								Signed 1's Complement								Signed 2's Complement							
+75	0	1	0	0	1	0	1	1	0	1	0	0	1	0	1	1	0	1	0	0	1	0	1	1
+80	0	1	0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	1	0	1	0	0	0	0
-80	1	1	0	1	0	0	0	0	1	0	1	0	1	1	1	1	1	0	1	1	0	0	0	0

Table 8: 8-bit representation of +75, +80 and -80

For the present example, signs of two numbers are different and the magnitude of -80 is higher than 75, therefore, 75 is subtracted from 80 as shown in the following table and the sign of the -80, which is minus is selected for the output. Please note that during the subtraction, carry is required to be taken as done in the decimal numbers. In binary, if a carry is taken then the number changes to two digit number as shown in the table. Please note that 10 of binary is a value 2 in decimal.

Number	Signed Magnitude							
Carry taken out from the bit	No	No	No	yes	yes	yes	yes	No
Updated bit value				0	~~10~~ 1	~~10~~ 1	~~10~~ 1	10
-80	1	1	0	~~1~~	~~0~~	~~0~~	~~0~~	~~0~~
+75	0	1	0	0	1	0	1	1
Subtraction	Sign bit	1-1	0-0	0-0	1-1	1-0	1-1	10-1
Result =-5	1	0	0	0	0	1	0	1

Table 9: 8-bit addition using Signed magnitude notation

Example 10: Add the decimal numbers 75 and 80 using signed magnitude notation, assuming the 8-bit length of the notations.

Solution: The process of addition of +75 and +80 in signed magnitude representation is shown below:

Number	Signed Magnitude							
Carry from previous bit addition	No	Yes	No	No	No	No	No	No
Carry bit value	1							
+75	0	1	0	0	1	0	1	1
+80	0	1	0	1	0	0	0	0
	1	1+1=10	0+0	0+1	1+0	0+0	1+0	1+0
Result is -27 (error)	1	0	0	1	1	0	1	1

Table 10: 8-bit addition using Signed magnitude notation having overflow

The addition of 7 bit magnitude has resulted in 8 bit output, which cannot be stored in this notation as this notation has a length of 8 bits with 1 sign bit. The last bit will be lost and you will obtain an incorrect result -27. This problem has occurred as the size of the number is fixed. This is called the overflow. You may please note that the actual addition of the 75 and 80 is 155, which is beyond the range of 8 bit signed magnitude representation, which is -127 to +127. This is why you should be careful while selecting the integral data types in a programming languages. For example, in case you have selected small unsigned integer of byte size for a variable, then you can store a only the number values in the range 0 to 255 in that variable.

Addition using signed-1's complement notation:

In signed 1's complement notation the addition process is simpler than signed-magnitude representation. An interesting fact is that in this notation you do not have to check the sign, just add the numbers, why? This is due to the fact that complement of a number as defined makes it complete, the binary digits are complement of each other and even the sign bits are complement. Therefore, the process of addition of two signed 1's complement notation just requires addition of the two numbers, irrespective of the sign. The process of addition in signed 1's complement representation will require the following steps:

Step 1: Just add the numbers, irrespective of sign.

Step 2: Now check the following conditions:

Carry in to the Sign Bit	Carry out of the Sign bit	Comments
No	No	Result is fine
Yes	Yes	Add 1 to result and it is fine
No	Yes	Overflow, incorrect result
Yes	No	Overflow, incorrect result

Table 11: The conditions of 1's complement notation, while addition

The following example demonstrates the process of addition .

Example 11: Add the decimal numbers 75 and -80 using signed 1's complement notation, assuming the 8-bit length of the notations.

Solution: The numbers are (The left most bit is the Sign bit). The Table 8 shows the values of +75 and -80 in signed 1's complement notation.

Number	Signed 1's Complement Notation							
Carry from previous bit	No	No	No	yes	yes	yes	yes	-

addition								
Updated bit value	-	-	-	1	1	1	1	-
+75	0	1	0	0	1	0	1	1
-80	1	0	1	0	1	1	1	1
Addition	Sign bit (0+1=1)	1+0 =1	0+1 =1	1+0+0 =1	1+1+1 =11	1+0+1 =10	1+1+1 =11	1+1 =10
Result (-5)	1	1	1	1	1	0	1	0
Since, the result is negative, so taking a 1's complement to verify the magnitude								
-Result=+5	0	0	0	0	0	1	0	1

Table12: 8-bit addition using Signed 1's complement notation

Example 12: Add the decimal numbers 75 and 80 using signed 1's complemt notation, assuming the 8-bit length of the notations.
Solution: The process of addition of +75 and +80 in signed magnitude representation is shown below:

Number	Carry out (9th bit)	Signed 1's Complement Notation							
Carry from previous bit addition	No	Yes	No	No	No	No	No	No	-
Updated bit value	No Carry out of the sign bit addition	Carry in to Sign Bit 1	-	-	-	-	-	-	-
+75		0	1	0	0	1	0	1	1
+80		0	1	0	1	0	0	0	0
Addition		Sign bit (0+0+1=1)	1+1 =10	0+0 =0	0+1 =1	1+0 =1	0+0 =0	1+0 =1	1+0 =1
Result (A negative number)		1	0	0	1	1	0	1	1
There is carry in the sign bit (1) and no carry out of the sign bit. Therefore, as per Table 11, there is an overflow and the result is incorrect. You can observe that the result is negative for addition of two positive numbers, which is NOT possible.									

Table 13: 8-bit addition using Signed 1's complement notation having overflow

Once again, please observe that the range of numbers in 8-bit 2's complement notation is -127 to +127, and the addition of two numbers 155 cannot be represented in 8-bits. Hence, there is an overflow.

Addition using signed-2's complement notation:

In signed 2's complement notation the addition process is simplest of these three representations. In this notation also, you do not have to check the sign, just add the numbers including the sign bit. The process of addition in signed 2's complement representation will use the following steps:

Step 1: Just add the numbers, irrespective of sign.

Step 2: Now check the following conditions:

Carry in to the Sign Bit	Carry out of the Sign bit	Comments
No	No	Result is fine
Yes	Yes	Result is fine
No	Yes	Overflow, incorrect result
Yes	No	Overflow, incorrect result

Table 14: The conditions of 2's complement notation, while addition

The following example demonstrates the process of addition using signed 2's complement notation.

Example 13: Add the decimal numbers (i) -69 -59 (ii) -69+59 (iii) +69-59 and (iv) +69=59

Solution: The numbers are (The left most bit is the Sign bit). The Table 14 shows the numbers in signed 2's complement notation. The left most bit is the sign bit

Number	Signed 2's Complement							
+69	0	1	0	0	0	1	0	1
- 69	1	0	1	1	1	0	1	1
+59	0	0	1	1	1	0	1	1
-59	1	1	0	0	0	1	0	1

Table 15: Numbers of example 13 in 2's complement notation

(i) -69-59

Number	Carry out (9th bit)	Signed 2's Complement Notation							
Carry from previous bit addition	yes	Carry in to Sign bit yes	yes	yes	yes	yes	yes	yes	-
Carry for addition	1	1	1	1	1	1	1	1	-
-69		1	0	1	1	1	0	1	1
-59		1	1	0	0	0	1	0	1
Addition of bits given above		1+1+1 =11	1+0+1 =10	1+1+0 =10	1+1+0 =10	1+1+0 =10	1+0+1 =10	1+1+0 =10	1+1 =10
Result	1	1	0	0	0	0	0	0	0
There is carry in to the sign bit (1) and there is a carry out of the sign bit (1). Therefore, as per Table 14, there is NO overflow and the result is correct and equal to -128. Discard the carry out bit (the 9th bit).									

Table 16: Addition of two negative numbers without overflow

(ii) -69+59

Number	Carry out (9th bit)	Signed 2's Complement Notation							
Carry from previous bit addition	No	Carry in to Sign bit No	yes	yes	yes	No	yes	yes	-
Carry for addition	-	-	1	1	1	-	1	1	-
-69		1	0	1	1	1	0	1	1
+59		0	0	1	1	1	0	1	1
Addition of bits given above		1+0 =1	1+0+0 =1	1+1+1 =11	1+1+1 =11	1+1 =10	1+0+0 =1	1+1+1 =11	1+1 =10
Result	-	1	1	1	1	0	1	1	0
There is No carry in to the sign bit and there is No carry out of the sign bit. Therefore, as per Table 14, there is NO overflow and the result is correct and equal to -10. Verify the result yourself.									

Table 17: Addition of bigger negative number and smaller positive numbers. No overflow is possible.

(iii) +69-59

Number	Carry out (9th bit)	Signed 2's Complement Notation							
Carry from previous bit addition	yes	Carry in to Sign bit yes	No	No	No	yes	No	yes	-
Carry for addition	1	1				1		1	-
+69		0	1	0	0	0	1	0	1
-59		1	1	0	0	0	1	0	1
Addition of bits given above		1+0+1 =10	1+1 =10	0+0 =0	0+0 =0	1+0+0 =1	1+1 =10	1+0+0 =1	1+1 =10
Result	1	0	0	0	0	1	0	1	0
There is a carry in to the sign bit (1) and there is a carry out of the sign bit (1). Therefore, as per Table 14, there is NO overflow and the result is correct and equal to +10. Discard the carry out bit (the 9th bit). Verify the result yourself.									

Table 18: Addition of smaller negative number and bigger positive numbers. No overflow is possible.

(iv) +69+59

Number	Carry out (9th bit)	Signed 2's Complement Notation							
Carry from previous bit addition	No	Carry in to Sign bit yes	yes	yes	yes	yes	yes	yes	-
Carry for addition	-	1	1	1	1	1	1	1	-
+69		0	1	0	0	0	1	0	1
+59		0	0	1	1	1	0	1	1
Addition of bits given above		1+0+0 =1	1+1+0 =10	1+0+1 =10	1+0+1 =10	1+0+1 =10	1+1+0 =10	1+0+1 =10	1+1 =10
Result	-	1	0	0	0	0	0	0	0
There is a carry in to the sign bit (1) but there is NO carry out of the sign bit. Therefore, as per Table 14, there is an *overflow* and the result is incorrect. Verify the result yourself. Overflow has occurred as the addition of the two numbers is +128, which is out of the range of numbers that can be represented using 8-bit signed 2's complement notation.									

Table 19: Addition of two positive numbers.

It may be noted that for the signed 2's complement notation, which is using 8 bits representation, is -128 to +127, which can be checked from table 16 and table 19.

Overflow formally is defined as the situation where the result of operation on two or more numbers, each of size n digits, exceeds the size n.

Overflow may cause even your correct programs to output incorrect results, therefore, is a very risky error. One of the ways of avoiding overflow in programs is to select appropriate data types and verifying the results range.

Arithmetic Subtraction: In general, a computer system uses the signed 2's complement notation, which simplifies the process of addition and subtraction as well as has a single representation for 0. You can perform subtraction by just taking the 2's complement of the number that is to be subtracted, and thereafter just adding the two numbers just like it has been shown in this section.

Multiplication and division: Multiplication and division operations using signed 2's complement notations are not straight forward. One of the simplest approach to multiply two signed 2's complement numbers is by multiplying the positive numbers and then adjusting the result based on the sign. However, this approach is time consuming as well as not used for implementation of multiplication operation. There are a number of algorithms for performing multiplication and division. One such algorithm is the Booth's algorithm. A detailed discussion on these topics is beyond the scope of this course.

In several arithmetic computations binary representation of decimal number is used for performing arithmetic operations. The next subsection briefly explains this representation.

2.5.3 Decimal Fixed Point Representation

Decimal digits can be represented in binary directly using four bits, as there are only 10 decimal digits, whereas 2^4=16 different values can be expressed using 4 bits. Thus, a BCD may be represented as 0000 (for decimal digit 0) to 1001 (for decimal digit 9). In addition, the sign can be represented using a single bit; however, it may change the format of representation. Thus, in decimal fixed point representation even sign is represented as four bits. Interestingly, the positive sign is represented using 1100 and negative sign is represented using 1101. Please note these two combinations are different from the representation of decimal digits, which is 0000 to 1001.

Example14: Represent +125 as BCD and a binary number.

+125 in BCD is given below:

Sign Digit	1	2	5
1100	0001	0010	0101

+125 in Binary:

S	7-bit magnitude						
-	64	32	16	8	4	2	1
0	1	1	1	1	1	0	1

Why is this representation needed? In several computing devices the computations are performed on binary coded decimals directly, without conversion to binary. One such device was old calculator. You may refer to further readings for more details on BCD arithmetic.

Check Your Progress 2

1) Write the BCD for the following decimal numbers:
   i) -23456
   ii) 17.89
   iii) 299

   .....

   .....

.....

.....

2) Compute the 1's and 2's complement of the following binary numbers. Also find the decimal equivalent of the number.

i) 1110 0010

ii) 0111 1110

iii) 0000 0000

.....

.....

.....

…….

3) Add the following decimal numbers by converting them to 8-bit signed 2's complement notation.

i) +56 and – 56

ii) +65 and –75

iii) +121 and +8

Identify, if there is overflow.

.....

…….. .....

…….....

…….

2.6 FLOATING POINT REPRESENTATION

In most real numbers, you may use a decimal point to distinguish integer and fraction part. However, in a computer system the position of binary point is assumed in the numbers. Fixed point representation, in general, fixes the location of the point towards the right most. Thus, integer values are represented using fixed point representation. What about the real numbers. A real number can be represented using an exponential notation. This forms the basis of binary number representation called floating point representation. For example, a decimal real number 29.25 can be represented as 0.2925×10^{2} or 2925×10^{-2}.

The first part of the number is called the "mantissa" of "significand" and second part of the number is called the exponent. You may please note that the mantissa can either be integer or fraction as shown in the example; the exponent value is adjusted accordingly. In computer the mantissa and exponent both are represented as binary numbers and the location of binary point is assumed. The following example explains the binary floating point representation. This representation is IEEE 754 standard for 32-bit floating point number.

It has the following format:

Bit Positions from the left	**1**	**2 to 9**	**10 to 32**

(a) Basic details

Length of Field	1 bit	8 bits	23 bits
Purpose	To store the *Sign* bit	To store the *Exponent*	Stores the fractional *Significand* of the Number
Comment	The *Sign* bit is for the *Significand*	The exponent is stored in *biased* form with a *bias* of 127	The *Significand* is stored as a *normalized* binary number

***Exponent* (8 bits) so possible values 0 to 255.** A bias of 127 is assumed. Let the exponent be *exp*	***Significand* values (23 Bits)** Assume that *Significand* be *M*, which is 23 bit long	**The Number Represented**
For *Exponent* value (*exp*) 0	All the bits of *M* are zeros. *M* is NOT zero (*M* may not be normalized)	The number is ±0 depending on the sign bit. The Number is $\pm 0.M \times 2^{-126}$
For *Exponent* values (*exp*) from 1 to 254	Normalized representation is used, therefore, *the first bit is assumed to be 1.*	The Number is $\pm 1.M \times 2^{exp-127}$
For *Exponent* value (*exp*) 255	All bits of *M* are zeros *M* is NOT zero.	The number is ±∞ depending on the sign bit It does NOT represent a valid Number

(b) Single Precision 32-bit IEEE-754 Standard

Table 20: IEEE 754 Floating Point 32-bit Number Representation

The three terms in Table 20 are *fractional Significand*, *bias* and *normalized*. They are explained below:

fractional Significand: Floating point number assumes that the position of binary point is prior to the *Significand*, therefore, *Significand* is a fraction (Refer to example 15).

Bias: It is an interesting way to store signed numbers without using any sign bit. It stores the number by adding a value in the exponent. For example, a 4 bit binary number can store values 0000 to 1111, i.e. values 0 to 15. A bias of 8 will allow values -8 to +7 to be stored in this range by adding the bias. In other words exponent value -8 will be coded as (-8+8) 0, -7 will be coded as (-7+8) 1, and so on till +7, which will be coded as (+7+8) 15. But, why is biasing used for exponent? The basic reason here is that biased numbers simplify the floating point arithmetic. This is explained later with the help of an example.

Normalized: A fraction is called normalized if it starts with a bit value 1 and not with bit value 0. For example, the values .1001, .1111, .1000, .1010 are normalized, but the values .0100, .0001, .0010, .0011 are not normalized.

The following example explains the process of converting a decimal real number to a floating point number representation using IEEE-754 standard (32-bit representation). You may solve similar problems using double precision representation also, where only the size of exponent (and bias) and significand is different.

Example 15: Represent the number -29.25 using IEEE 754 (32 bit) representation as shown in Table 20.

Solution:

Step 1: Convert the number to binary

The number should first be converted to binary as follows:

Sign bit = 1 as number is negative

29 can be represented in 7 bits as 001 1101

.25 can be represented in 4 bits as .0100

Thus, 29.25 without sign is 001 1101 . 0100

Step 2: Normalize the number

Normalizing the number requires binary point to be moved before the most significant 1, it requires point to be shifted to left by 5 spaces. Thus, the normalized number now is: $0.1\ 1101\ 0100 \times 2^5$.

Step 3: Adjust the normalized number

It may be noted from Table 20(b), that in IEEE-754 representation, when the exponent is between 1 and 254, the first bit is assumed to be 1, therefore, the *Significand* whose size is 23 bits, actually represents 24 bit *Significand.* In addition, please note that as the number is assume to be $\pm 1.M \times 2^{exp-127}$, therefore, the value to be represented ($0.1\ 1101\ 0100 \times 2^5$) must be adjusted to this format by shifting the binary point one place to the right and adjusting the exponent. Thus, the adjusted number is 1.11010100×2^4.

Step 4: Compute the exponent using the bias

Finally add bias to the exponent value to obtain *exp* value of IEEE-754. In this case, *exp* = 4+127 (127 is the bias value)=131.

Step 5: Represent the final number

Represent the sign bit (S), *exp* in 8 bits and *Significand* in 23 bits, as follows:

S	*exp* of length 8 bits (value 131)	*Significand* of length 23 bits (value 1.11010100) Represented as ~~1~~. 110 1010 0
1	1 0 0 0 0 0 1 1	1 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Example 16: A number using IEEE 754 (32 bit) is given below, what is the equivalent decimal value.

S	*exp* of length 8 bits	*Significand* of length 23 bits (*M*)
1	1 0 0 0 1 0 0 1	1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Solution:

The number is represented as: $\pm 1.M \times 2^{exp-127}$

The sign bit states it is a negative number

M is 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Exp is 1 0 0 0 1 0 0 1 = 137 in decimal.

The number is $-1.11110000000000000000000 \times 2^{137-127}$

$= -1.11110000000000000000000 \times 2^{10}$

$= -11111000000.0000000000000$

$= -11111000000$

$= -1984$ in decimal

In floating point numbers a term *precision* is very important. What is precision? The precision defines the correctness of representation. For example, suppose you *just use 2 decimal digits* in a fractional decimal numbers, then you can represent the numbers 0.10, 0.11, 0.99 etc precisely. The number 0.985 may be either truncated to 0.98 or rounded off to 0.99. This introduces an error in number, which is due to the fact that the size of *Significand* is limited. For scientific computations such errors may lead to failure. Therefore, IEEE-754 defines many different precision of numbers, few such popular precisions are single precession IEEE-754 number, which is a 32-bit representation, explained above; **IEEE-754 double precision number**, which is a 64

bit representation with 1 sign bit, 11 bit exponent and 52 bit *Significand*; and IEEE-754 quadruple precision number, which is a 128 bit representation with 1 sign bit, 15 bit exponent and 112 bit *Significand*. It may be noted that in programming languages you use data types *float* and *double*, which corresponds to the IEEE-754 single and double representation respectively.

Finally, what is the range of the numbers that can be represented using the IEEE-754 representation? As stated in Table 20, the minimum exponent value for a normalized number is 1 and maximum is 254. Therefore, the minimum (negative) number will be:

S	*exp* of length 8 bits	*Significand* of length 23 bits (*M*)
1	0 0 0 0 0 0 0 1	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

This will be equal to $\pm 1.M \times 2^{exp-127}$

$= -1.000\ 0000\ 0000\ 0000\ 0000\ 0000 \times 2^{1-127}$

$= -1 \times 2^{-126}$

The maximum (positive) number will be:

S	*exp* of length 8 bits	*Significand* of length 23 bits (*M*)
1	1 1 1 1 1 1 1 1 0	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

This will be equal to $\pm 1.M \times 2^{exp-127}$

$= +1.111\ 1111\ 1111\ 1111\ 1111\ 1111 \times 2^{254-127}$

$= +(1.111\ 1111\ 1111\ 1111\ 1111\ 1111$

$+0.000\ 0000\ 0000\ 0000\ 0000\ 0001$

$-0.000\ 0000\ 0000\ 0000\ 0000\ 0001) \times 2^{127}$

$= +(10.\ 000\ 0000\ 0000\ 0000\ 0000\ 0000$

$-0.000\ 0000\ 0000\ 0000\ 0000\ 0001) \times 2^{127}$

$= +(2-1\times 2^{-23}) \times 2^{127}$

You may please note that IEEE-754 has a representation for *0 and infinite*.

Arithmetic Using Floating Point Numbers:

As you have noticed that addition and subtraction using 2's complement notation was direct, but addition and subtraction of floating point number requires several steps. These steps are explained with the help of the following example.

Example 17: Add the following floating point numbers

Equivalent Numbers		**IEEE 754 32 bit representation**		
Decimal	*Binary*	*S*	*exp*	*Significand* (*M*)
-7	$-1.11\times 2^{129-127}$ $= -1.11\times 2^{2}$ $= -111.0$	1	1000 0001	110 0000 0000 0000 0000
+24	$+1.1\times 2^{131-127}$ $= +1.1\times 2^{4}$ $= +11000.0$	0	1000 0011	100 0000 0000 0000 0000

Solution:

Step 1: Find the difference in exponents of the numbers

1000 0011 - 1000 0001 = 0000 0010 = 2 in decimal

Step 2: Align the *Significand* of the smaller number by denormalizing it

Shift the smaller number to right by the difference of exponents as shown:

	The value of 1.*M*
Significand of First Number (Smaller)	1.110 0000 0000 0000 0000
Shift it to right twice (denormalized)	0.011 1000 0000 0000 0000

Step 3: Check the sign of the two numbers, if same add else subtract smaller number from bigger number.

The signs are different; therefore, subtract smaller number from larger number

	The value of 1.*M*
Significand of Second Number (Larger)	1.100 0000 0000 0000 0000
Denormalized first number (smaller)	0.011 1000 0000 0000 0000
Result of Subtraction	1.000 1000 0000 0000 0000

Step 4: Select the sign and exponent of the bigger number as sign and exponent of the result and Normalize the *Significand* by adjusting the exponent
The result is shown below. Please note that in this case, there is no need to normalize the result as it is already normalized.

Result of addition operation (verification)		**IEEE 754 32 bit representation**		
Decimal	*Binary*	*S*	*exp*	*1.M*
+17	$+1.0001\times2^{131-127}$ $=+1.0001\times2^{4}$ = +10001.0	0	1000 0011	1.000 1000 0000 0000 0000

Likewise, subtraction operation can be performed.

Multiplication and division operations in floating point number require multiplication or division of significands as well as addition or subtraction of exponents. In addition, these operations may require normalizing the result. The following example shows the multiplication of two floating point numbers.

Example 18: Multiply the following floating point numbers

Equivalent Numbers		**IEEE 754 32 bit representation**		
Decimal	*Binary*	*S*	*exp*	*Significand* (*M*)
-7	- 111.0	1	1000 0001	110 0000 0000 0000 0000
+24	+11000.0	0	1000 0011	100 0000 0000 0000 0000

Solution:

Step 1: Multiply the *Significand* values of the two numbers and truncate to 23 bits (plus one assumed bit)

	The value of 1.*M*
Significand of First Number	1.110 0000 0000 0000 0000
Significand of Second Number	1.100 1000 0000 0000 0000
Significand after multiplication	10.101 0000 0000 0000 0000

Step 2: If multiplication, add the exponents and subtract the bias as both the numbers have biased exponents; if division, subtract the exponent of divisor from the exponent of dividend and add the bias as it get canceled in subtraction.

Exponent of First Number	1000 0001
Exponent of Second Number	1000 0011
Multiplication operation, so add and subtract bias	1 0000 0100
-127	-0111 1111
The new exponent	1000 0101

Step 3: Check the sign of the two numbers, if same result has + sign else - sign
The signs are different; therefore, result has negative sign. Also, normalize the *Significand* of the result

Result of Multiplication	**IEEE 754 32 bit representation**

operation				
Decimal	*Binary*	*S*	*exp*	*1.M*
Normalize the result		1	1000 0101	10.10 1000 0000 0000 0000
Normalized result		1	1000 0110	1.010 1000 0000 0000 0000
-168	$-1.0101\times2^{134-127}$ $= -1.0101\times2^{7}$ = -10101000.0	1	1000 0110	1.010 1000 0000 0000 0000

Likewise, division operation can be performed. You may refer to further readings for finding more details on floating point numbers and arithmetic.

2.7 ERROR DETECTION AND CORRECTION CODES

In the previous sections you have gone through various binary codes and representations. A computer works on these binary numbers and during the operations of the computer data is transferred from a source to one or more destinations. During this process of transmission of data, there is a possibility of transmission errors. The purpose of error detection and correction codes is to identify those data transmission errors and correct the data, as far as possible. As the data in computer consists of binary digits, therefore, an error in a bit can result in change of its value from 0 to 1 or vice versa. This section explains one error detection code called Parity and one error detection and correction code called Hamming Error correction code.

Parity bit: The purpose of a parity bit is to detect an error in a group of bits. But how does it perform the task of checking error? It is explained with the help of following process:

Steps	Source Side	Destination Side
1	A parity bit is generated at source, which ensures that: Number of 1's in sources data and source parity bit is odd (called Odd parity) OR Number of 1's in sources data and source parity bit is even (called Even parity)	
2	The source data and source parity bits are sent to the designation.	
3		The source data and source parity bits are received at the designation and a destination parity bit is generated using only the data received (not source parity) by using the same process i.e. Even of Odd parity used at source.
4		Source parity bit and destination parity bit are compared. If they are same, then no error in data is detected, else either the data or parity bit is in error, which is reported.

Table 21: Error detection using parity bit

Example 19 explains the process as given above.

Example 19: 7-bit data 010 1001 is sent from a source, such as CPU register, to a destination, such as RAM. The data is received at the destination as 010 1000 having error in one bit. How does this error be detected by parity bit?

Solution:

Steps	*The Process*
Step 1: At Source	Data to be sent: 010 1001 Odd Parity bit is computed as: The data has 3 bits having value 1. So odd parity bit =*0*
Step 2: At Source	The source parity + source data is sent as: *0* 010 1001
Step 3: At Destination	As per the statement of the example the data is received as: *0* 010 1000 Source Parity = *0* is received correctly as error is in one bit only Destination parity is computed on data received, which is 010 1000. It has 2 bits as 1, therefore, Odd parity bit at destination=*1*
Step 4: At Destination	Source parity bit (*0*) ≠ Destination parity bit(*1*) ERROR in data

It may be noted that parity bit can detect errors in case 1 bit is in error. In case 2 bits are in error, then it will fail to detect the error.

Hamming Error-Correcting Code: The Hamming code was conceptualized by Richard Hamming at Bell Laboratories. This code is used to identify and correct the error in 1 bit. Thus, unlike parity bit, which just identifies the existence of error, this code also identifies the bit that is in error. The idea of Hamming's code is to divide the data bits into a number of groups; and using the parity bit to identify, which groups are in error; and based on the groups in error, identify the bit which has caused the error. Thus, the grouping process has to be very special, which is explained below:

How to Group data bits? Before grouping, you may assume the placement of data and parity bits using the following considerations.

A bit position that is exact power of 2 will be used for storing parity bit. For example, 2^0=1, that is 1st bit position will be used to store parity bit, likewise 2^1=2, 2^2=4, and 2^3=8, i.e. 2nd , 4th and 8th bit positions will also be used to store parity bit. Thus, you have now 7 bit data and 4 parity bits, so a total of 11 bit positions. (*p* indicates parity bit and *d* indicates data bit)

Bit Position	12	11	10	9	8	7	6	5	4	3	2	1
Stores	*d8*	*d7*	*d6*	*d5*	***p4***	*d4*	*d3*	*d2*	***p3***	*d1*	***p2***	***p1***
For grouping the data bit number is used to identify the parity bit to which data should be member of												
Bit position 12 (8+4) contains (d8)					8				4		-	-
Bit position 11 (8+2+1) contains (d7)					8				-		2	1
Bit position 10(8+2) contains (d6)					8				-		2	-
Bit position 9(8+1) contains (d5)					8				-		-	1
Bit position 8 contains (p4)					*p*4							
Bit position 7(4+2+1) contains (d4)					-				4		2	1
Bit position 6(4+2)					-				4		2	-

contains (d3)												
Bit position 5(4+1) contains (d2)					-				4		-	1
Bit position 4 contains (p3)									*p3*			
Bit position 3(2+1) contains (d1)					-				-		2	1
Bit position 2 contains (p2)											*p2*	
Bit position 1 contains (p1)												*p1*

Table 22: Placement of data and parity bits for Hamming's error detection and correction code

Groups for parity bits: The groups are made for each on the basis of bit positions, on the basis of above Table. A bit position, which includes a parity bit position is included in the group of that parity bit. For example, the bit at bit position 12 will be included in group of parity bit *p*4 and *p*3; similarly, bit position 7 will be included in group of parity bit *p*3, *p*2 and *p*1. But why these grouping? You may please note that each data bit is part of unique combination of groups, so if it is in error, it will cause errors in all those groups to which it is a part of. Thus, by identifying all the groups, which has parity mismatch, will identify the bit which is in error. The following table shows these groups for 8 bit data.

Group for Parity bits	Bit positions and data bit
*p*4	*Bit position 12 data bit d8, Bit position 11 data bit d7, Bit position 10 data bit d6 and Bit position 9 data bit d5*
*p*3	*Bit position 12 data bit d8, Bit position 7 data bit d4, Bit position 6 data bit d3 and Bit position 5 data bit d2*
*p*2	*Bit position 11 data bit d7, Bit position 10 data bit d6, Bit position 7data bit d4, Bit position 6 data bit d3 and Bit position 3data bit d1*
p1	*Bit position 11 data bit d7, Bit position 9 data bit d5, Bit position 7data bit d4, Bit position 5 data bit d2 and Bit position 3data bit d1*

Therefore, the parity bits will be generated using the following data bits:

Parity bit	Compute Odd parity of Data bits
*p*4	*d8, d7, d6 and d5*
*p*3	*d8, d4, d3 and d2*
*p*2	*d7, d6, d4, d3 and d1*
p1	*d7, d5, d4, d2 and d1*

So, how the data bit in error be recognised? It is illustrated with the help of following example

Example 20: 8-bit data 1010 1001 is sent from a source to a destination. The data is received at the destination as 1000 1001 having error in only one bit. How does this error be detected and corrected by Hamming's error detection and correction code?
Solution:
Step 1: Place the bits as shown in Table 22 and generate parity bits at the source, for example, the odd parity bit *p4* is computed using *d8, d7, d6* and *d5* (shown as shaded cells in the following table). Their values are 1, 0, 1, 0 as shown in the table, as there are only two bits containing 1, therefore, the odd parity value for *p4* is 1. Likewise compute the other parity bits as shown in Table 23.
Step 2: Data and the associated parity bits in the sequence as shown below are sent to the destination, where once again parity bits are computed for the received data.

Step 3: Compare the source parity bits and destination parity bits as shown in Table 23. Please note when two parity bit match, a 0 is put in the compare word else a 1 is put. The magnitude of comparison word, indicates the bit position that is in error.

Step 4: If there is an error, then the data at bit position that is in error is complemented.

Step 5: The data is used at the destination after omitting the parity bits.

Bit Position	12	11	10	9	8	7	6	5	4	3	2	1
Stores	*d8*	*d7*	*d6*	*d5*	***p4***	*d4*	*d3*	*d2*	***p3***	*d1*	***p2***	***p1***
Data Bits	1	0	1	0		1	0	0		1		
Compute Odd parity bit p4 using d8, d7, d6 and d5					1							
Compute Odd parity bit p3 using d8, d4, d3 and d2									1			
Compute Odd parity bit p2 using d7, d6, d4, d3 and d1											0	
Compute Odd parity bit p1 using d7, d5, d4, d2 and d1												1
Data and Parity bits at Source	1	0	1	0	1	1	0	0	1	1	0	1
Data is sent to the destination, where data is received with 1 bit in error (given), therefore all the source parity bits are received without any error												
Data received at destination including parity bits	1	0	0	0	1	1	0	0	1	1	0	1
***Step 2**: Compute the parity bits using the data received at the destination*												
Data Bits Received	1	0	0	0		1	0	0		1		
Compute Odd parity bit p4 using d8, d7, d6 and d5					0							
Compute Odd parity bit p3 using d8, d4, d3 and d2									1			
Compute Odd parity bit p2 using d7, d6, d4, d3 and d1											1	
Compute Odd parity bit p1 using d7, d5, d4, d2 and d1												1
***Step 3:** Compare the source parity bits and destination parity bits.*												
Source Parity bits					1				1		0	1
Destination Parity bits					0				1		1	1
Parity Comparison word (0 if source and destination parity match else 1)					1				0		1	0
The comparison word is 1010 = 10 in decimal, i.e. bit position 10 is in error. The error in this bit can be corrected by complementing the bit position 10.												
Corrected Data	1	0	1	0		1	0	0		1		

Table 23: Example of Hamming's error detection and correction code

It may be noted in Table 23 that the value of comparison word 0000 would mean that there is no error in transmission of data. In addition, the values 1000, 0100, 0010 and 0001 would mean that one bit error has occurred in the transmission of source parity

bits *p4, p3, p2* and *p1* respectively. Thus, no change would be needed in the received data bits at the destination in such cases.

It may please be noted that Hamming's code presented in this section can detect and correct errors in a single bit ONLY. It will not work, in case two or more bits are in error. One final question is about the size of the code needed to correct single bit error. The size will be dependent on the size of data. A simple rule is that the size of code and the data should be less than the possible bit positions that can be flagged by the comparison word. If the data to be transmitted is of size *D* bits and *P* is the number of parity bits needed for the given Hamming's code, then size of the code is the smallest value of *P*, which satisfies that following equation:

$$D + P < 2^P$$

For example, for a *D*=4 bits, the value of *P* would be 3 as:

$$4 + 3 < 2^3 \quad \text{as } 7<8$$

and for a *D*=8 bits, the value of *P* would be 4 as:

$$8 + 4 < 2^4 \quad \text{as } 12<16$$

Check Your Progress 3

1) Represent the following numbers using the IEEE-754 32bit standard:

   i) 39.125
   ii) -0.000011000_2

2) Compute the Odd and Even parity bits for the following data:

   i) 0111110
   ii) 0110000
   iii) 1110111
   iv) 1001100

   .....

   .....

3) A 4 bit data 1011 is received at the destination as 1111, assuming single bit is in error, illustrate how Hamming's single error correction code will detect and correct the error

   .....

   .....

   .....

2.7 SUMMARY

This Unit has introduced you to the basic aspects of data representation. It introduces the character representing including ASCII and Unicode. In addition, the Unit explains number conversion and fixed point representation of binary number. The Unit also highlights the arithmetic operations. This was followed by a detailed discussion on the floating point numbers. Though only IEEE 754 32-bit single precision numbers are explained, however, the logic discussed is applicable to double precision numbers too. The Unit finally introduces you to error detection code -parity bit and error detection and correction code. You must practice the data conversions and these codes as they would be useful, when you deal with binary numbers.

You should refer to the further readings for more detailed information on these topics. You are advised to take the help of further readings, Massive Open Online Courses (MOOCs), and other online resources as Computer Science is a dynamic area.

2.8 SOLUTIONS/ANSWERS

Check Your Progress 1

1) i) 11100.01101_2 to Octal and Hexadecimal

Binary Number	0	1	1	1	0	0	.	0	1	1	0	1	0
Grouping Directions	←							→					
Grouped (- replaced by 0)	0	1	1	1	0	0	.	0	1	1	0	1	0
Binary place values	4	2	1	4	2	1	.	4	2	1	4	2	1
Equivalent Octal Digit	0+2+1=3			4+0+0=4			.	0+2+1=3			0+2+0=2		
Octal Number	**3**			**4**			**.**	**3**			**2**		

Binary Number	0	0	0	1	1	1	0	0	.	0	1	1	0	1	0	0	0
Grouping Directions	←								.	→							
Grouped	0 0 0 1			1 1 0 0					.	0 1 1 0			1 0 0 0				
Binary place values	8	4	2	1	8	4	2	1	.	8	4	2	1	8	4	2	1
Equivalent Hexadecimal Digit	0+0+0+1=1				8+4+0+0=C				.	0+4+2+0=6				8+0+0+0=8			
Hexadecimal Number	**1**				**C**				**.**	**6**				**8**			

ii) 1101101010_2 to Octal and Hexadecimal

Binary Number	0	0	1	1	0	1	1	0	1	0	1	0
Grouping Directions	←											
Grouped (- replaced by 0)	0	0	1	1	0	1	1	0	1	0	1	0
Binary place values	4	2	1	4	2	1	4	2	1	4	2	1
Equivalent Octal Digit	0+0+1=1			4+0+1=5			4+0+1=5			0+2+0=2		
Octal Number	**1**			**5**			**5**			**2**		

Binary Number	0	0	1	1	0	1	1	0	1	0	1	0
Grouping Directions	←											
Grouped	0	0	1	1	0	1	1	0	1	0	1	0
Binary place values	8	4	2	1	8	4	2	1	8	4	2	1
Equivalent Hexadecimal Digit	0+0+2+1=3				0+4+2+0=6				8+0+2+0=A			
Hexadecimal Number	**3**				**6**				**A**			

2) i) 119_{10} to binary

The place value	2^6 =64	2^5 =32	2^4 =16	2^3 =8	2^2 =4	2^1 =2	2^0 =1
***N* = 119**	119-**64**=55; 55-**32**=23; 23-**16**=7; 7-**4**=3; 3-**2**=1; 1-**1**=0						
Equivalent Binary	**1**	**1**	**1**	**0**	**1**	**1**	**1**

ii) 19.125_{10}

The place	2^4	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}

value	=16	=8	=4	=2	=1		=0.5	=0.25	=0.125
***N* = 19.125**	19-**16**=3; 3-**2**=1; 1-**1**=0; and 2^{-3} =**0.125**								
Equivalent Binary	**1**	**0**	**0**	**1**	**1**	**.**	**0**	**0**	**1**

iii) 325_{10}

The place value	2^8 =256	2^7 =128	2^6 =64	2^5 =32	2^4 =16	2^3 =8	2^2 =4	2^1 =2	2^0 =1
***N* = 325**	325-**256**=69-**64**=5; 5-**4**=1; 1-**1**=0								
Equivalent Binary	**1**	**0**	**1**	**0**	**0**	**0**	**1**	**0**	**1**

3 i) 119_{10}

The place value	2^6 =64	2^5 =32	2^4 =16	2^3 =8	2^2 =4	2^1 =2	2^0 =1
Equivalent Binary	**1**	**1**	**1**	**0**	**1**	**1**	**1**
Equivalent Octal	**1**	**6**			**7**		
Equivalent Hexadecimal	**7**			**7**			

ii) 19.125_{10}

The place value	2^4 =16	2^3 =8	2^2 =4	2^1 =2	2^0 =1	.	2^{-1} =0.5	2^{-2} =0.25	2^{-3} =0.125
Equivalent Binary	**1**	**0**	**0**	**1**	**1**	**.**	**0**	**0**	**1**
Equivalent Octal	**2**		**3**			**.**	**1**		
Equivalent Hexadecimal	**1**	**3**				**.**	**0010=2**		

iii) 325_{10}

The place value	2^8 =256	2^7 =128	2^6 =64	2^5 =32	2^4 =16	2^3 =8	2^2 =4	2^1 =2	2^0 =1
Equivalent Binary	**1**	**0**	**1**	**0**	**0**	**0**	**1**	**0**	**1**
Equivalent Octal	**5**			**0**			**5**		
Equivalent Hexadecimal	**1**	**4**			**5**				

Check Your Progress 2

Check Your Progress 2

1) i) -23456 to BCD

Sign Digit	2	3	4	5	6
1101	0010	0011	0100	0101	0110

ii) 17.89

Sign Digit	1	7	.	8	9
1100	0001	0111	.	1000	1001

iii) 299

Sign Digit	2	9	9
1100	0010	1001	1001

2)

Deci	The Number	Signed 1's Complement	Signed 2's Complement

mal																							
-30	1	1	1	0	0	0	1	0	0	0	0	1	1	1	0	1	0	0	0	1	1	1	1	0
+126	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	1	1	0	0	0	0	0	1	0
0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0

3) i) +56 and – 56

Number	Carry out (9th bit)	Signed 2's Complement Notation							
Carry for addition	1	1	1	1	1				-
+56		0	0	1	1	1	0	0	0
-56		1	1	0	0	1	0	0	0
Addition of bits given above		1+0+1 =10	1+0+1 =10	1+1+0 =10	1+1+0 =10	1+1 =10	0+0 =0	0+0 =0	0+0 =0
Result	~~1~~	0	0	0	0	0	0	0	0
There is a carry in to the sign bit (1) and there is a carry out of the sign bit (1). Therefore, as per Table 14, there is NO overflow and the result is correct and equal to 0. Discard the carry out bit (the 9th bit). Verify the result yourself.									

ii) +65 and –75

Number	Carry out (9th bit)	Signed 2's Complement Notation							
Carry for addition								1	-
+65		0	1	0	0	0	0	0	1
-75		1	0	1	1	0	1	0	1
Addition of bits given above		0+1 =1	1+0 =1	0+1 =1	0+1 =1	0+0 =0	0+1 =1	1+0+0 =1	1+1 =10
Result		1	1	1	1	0	1	1	0
There is No carry in to the sign bit and there is No carry out of the sign bit. Therefore, as per Table 14, there is NO overflow and the result is correct and equal to -10. Verify the result yourself.									

iii) +121 and +8

Number	Carry out (9th bit)	Signed 2's Complement Notation							
Carry for addition		1	1	1	1				-
+121		0	1	1	1	1	0	0	1
+8		0	0	0	0	1	0	0	0
Addition of bits given above		1+0+0 =1	1+1+0 =10	1+1+0 =10	1+1+0 =10	1+1 =10	0+0 =0	0+0 =0	0+1 =1
Result		1	0	0	0	0	0	0	1
There is a carry in to the sign bit (1) and there is NO carry out of the sign bit. Therefore, as per Table 14, there is OVERFLOW and the result is incorrect.									

Check Your Progress 3

1) i) 39.125

Step 1: Convert the number to binary

Sign bit = 0 as number is positive

39 can be represented in 7 bits as 010 0111

.125 can be represented in 4 bits as .0010

Thus, 39.125 without sign is 0100111.0010

Step 2: Normalize the number

Normalizing the number requires binary point to be moved before the most significant 1, it requires point to be shifted to left by 6 spaces. Thus, the normalized number now is: 0.1001110010×2^6.

Step 3: Adjust the normalized number

The number is assume to be $\pm 1.M \times 2^{exp-127}$, therefore, the value to be represented (0.1001110010×2^6) must be adjusted. The adjusted number is 1.001110010×2^5.

Step 4: Compute the exponent using the bias

Add bias to the exponent value.

exp = 5+127 (127 is the bias value)=132.

Step 5: Represent the final number

Represent the sign bit (S), *exp* in 8 bits and *Significand* in 23 bits, as follows:

S	*exp* of length 8 bits (value 132)	*Significand* of length 23 bits (value *1.001110010*) Represented as ~~1~~.*001110010*
0	1000 0100	001 1100 1000 0000 0000 0000

ii) -0.000011000_2

Sign bit = 1 as number is negative

The number is of the format $\pm 1.M \times 2^{exp-127}$,

1.1000×2^{-5}

$exp = -5+127=122$

S	*exp* of length 8 bits (value 122)	*Significand* of length 23 bits (value *1.1000*) Represented as ~~1~~.*1000*
0	0111 1010	100 0000 0000 0000 0000 0000

2)

The Number	Even Parity	Odd Parity
0111110	1	0
0110000	0	1
1110111	0	1
1001100	1	0

3) The size of data (*D*)=4 bits, the value of *P* would be 3 as:

$4 + 3 < 2^3$ as 7<8

The bit positions for this code would be:

Bit Position	7	6	5	4	3	2	1
Stores	*d*4	*d*3	*d*2	***p*3**	*d*1	***p*2**	***p*1**
For grouping the data bit number is used to identify the parity bit to which data should be member of							
Bit position 7(4+2+1) contains (d4)				4		2	1
Bit position 6(4+2) contains (d3)				4		2	-
Bit position 5(4+1) contains (d2)				4		-	1
Bit position 4 contains (p3)				*p3*			
Bit position 3(2+1) contains (d1)				-		2	1
Bit position 2 contains (p2)						*p2*	
Bit position 1 contains (p1)							*p1*

Parity bit	Compute Odd parity of Data bits
*p*3	*d4, d3 and d2*
p2	*d4, d3 and d1*
p1	*d4, d2 and d1*

Bit Position	7	6	5	4	3	2	1
Stores	*d*4	*d*3	*d*2	***p*3**	*d*1	***p*2**	***p*1**
Data Bits	**1**	**0**	**1**		**1**		
Compute Odd parity bit p3 using d4, d3 and d2				1			
Compute Odd parity bit p2 using d4, d3 and d1						1	
Compute Odd parity bit p1 using d4, d2 and d1							0
Data and Parity bits at Source	1	0	1	1	1	1	0
Data received at destination including parity bits	1	1	1	1	1	1	0
Data Bits Received	1	1	1		1		
Compute Odd parity bit p3 using d4, d3 and d2				0			
Compute Odd parity bit p2 using d4, d3 and d1						0	
Compute Odd parity bit p1 using d4, d2 and d1							0
Source Parity bits				1		1	0
Destination Parity bits				0		0	0
Parity Comparison word (0 if source and destination parity match else 1)				**1**		**1**	**0**
Location 6 is in error, which is decimal equivalent of 110 the comparison word. So Corrected Data	1	0	1		1		

UNIT 3 LOGIC CIRCUITS - AN INTRODUCTION

Structure **Page Nos.**

3.0 Introduction
3.1 Objectives
3.2 Logic Gates
3.3 Boolean Algebra
3.4 Logic Circuits
3.5 Combinational Circuits
3.5.1 Canonical and Standard Forms of an Boolean expression.
3.5.2 Minimization of Gates
3.6 Design of Combinational Circuits
3.7 Examples of Logic Combinational Circuits
3.7.1 Adders
3.7.2 Decoders
3.7.3 Multiplexer
3.7.4 Encoder
3.7.5 Programmable Logic Array
3.7.6 Read Only Memory ROM
3.8 Summary
3.9 Solutions/ Answers

3.0 INTRODUCTION

In the previous units, we have discussed the basic configuration of computer system, simple instruction execution, data representation and different computer organisations. In addition, ISA and micro-architecture was discussed in unit 1 of this block. It may be noted that, as stated in the unit1, gates and logic circuits are the building blocks of a computer system. This unit introduces you to some of the basic components of computer system that are essential for learning the logic of binary computing devices. In this unit, you will be introduced to the concepts of logic gates, binary adders, logic circuits and combinational circuits. These circuits form the backbone of any computer system and knowing them will be useful in lower level programming.

3.1 OBJECTIVES

After going through this unit you will be able to:

- explain the basic functions of logic gates;
- describe role of Boolean algebra in digital circuits;
- perform the minimization the number of gates for a Boolean expression;
- identify and explain the basic circuits in a computer system
- design simple combinational circuits.

3.2 LOGIC GATES

A computer system is a binary device that uses electronic signals to perform basic computation on the digital data, which is also stored electronically. The

basis of such computation, called digital logic, are the electronic circuits fabricated on the semi-conductor chips that are used to formulate a set of operations. These basic sets of operations are then used to create complex circuitry, which is able to perform arithmetic, logical and control operations in a computer system. Thus, the simplest form of binary logic is: how a set of inputs can be used to create a typical output sequence, which is achieved using electronic gates.

A logic gate is an electronic circuit made of transistors, which produces a characteristic output signal for a typical input. In general, the input accepts one to several input values and produces a specific output. This output can be 0 or 1. Logic gates are used for implementing basic Boolean operations, which is explained in the subsequent sections. A logic gate is represented using a graphics symbol and performs a simple binary function, which can be represented with the help of a truth table. Figure 3.1 shows the basic logic gates. Please note that in Figure 3.1, the character *I* represent input values and *F* represent output values.

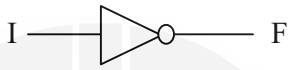
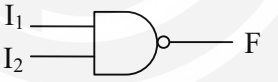
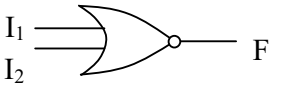
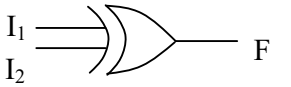
Gate	Graphical Symbol	Truth Table	Description
NOT		I F 0 1 1 0	It simply inverts the input value, i.e. input 0 will be converted to 1 and vice-versa.
OR		I1 I2 F 0 0 0 0 1 1 1 0 1 1 1 1	This gates output a value 1, if at least one of its input is 1.
AND		I1 I2 F 0 0 0 0 1 0 1 0 0 1 1 1	This gate output is 1, if both the input values are 1 else output is 0.
NAND		I1 I2 F 0 0 1 0 1 1 1 0 1 1 1 0	This is NOT AND, so where ever AND gate produce 0, this gate outputs 1.
NOR		I1 I2 F 0 0 1 0 1 0 1 0 0 1 1 0	This is NOT OR.
XOR		I1 I2 F 0 0 0 0 1 1 1 0 1 1 1 0	Exclusive OR produces output 1 when the two input are dissimilar.

Figure 3.1: Logic Gates

In the next few sections, we explain how these simple logic gates can be used to construct logic circuits. The next section explains the mathematics of logic circuits.

3.3 BOOLEAN ALGEBRA

Boolean algebra was designed by George Boole in the 19th century. It presents mathematical foundation for performing various functions on binary variables. Please recall that binary variables can have only two values 0 or 1. The value 0 by convention is taken as False and 1 as True. Please also refer to Figure 3.1, which shows the truth table for various gates. These truth tables can also be represented using the Boolean function. Figure 3.2 shows the Boolean algebraic representation of logic gates of Figure 3.1.

Gate	Boolean Representation	Explanation
NOT	$F = I'$	The symbol '′' in Boolean expression represents negation operator. Thus, output F is complement or negation of the value of I.
OR	$F = I_1 + I_2$	The OR is represented by Boolean operator '+', which represents that the value of F be zero only if both I_1 and I_2 are zero else 1.
AND	$F = I_1 . I_2$	The Boolean operator '.' represents AND operation. The value of F be 1 if both I_1 and I_2 are 1, else F will be 0.
NAND	$F = (I_1 . I_2)'$	NOT of AND
NOR	$F = (I_1 + I_2)'$	NOT of OR
XOR	$F = I_1 \oplus I_2$	$\oplus$ is an exclusive – OR operator.

Figure 3.2: Gates and related Boolean algebraic expression.

The Boolean algebra is very useful for mathematically representing a binary operation. For example, addition of two binary digits can be represented in truth table form as:

I_1	I_2	Carry (C)	Sum (S)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

∴ It can be represented using two Boolean functions, one for each output, viz. Carry and Sum, as:

$C = I_1 . I_2$ and

$S = I_1 \oplus I_2$ (Please refer to Fig. 3.1 & Fig 3.2)

The Boolean algebra is used to simplify logic circuits that are made of logic gates. However, before we demonstrate this process of simplification, first you may go through the basic rules of Boolean algebra. Figure 3.3 shows these rules. Please note that some of the rules are shown with proof using truth table. You can make truth table yourself for the cases for which the proof is not shown.

(i)

Input	Identities			
I	$I + 0 = I$	$I + 1 = 1$	$I \cdot 0 = 0$	$I \cdot 1 = I$
0 1	0 + 0 = 0 1 + 0 = 1	0 + 1 = 1 1 + 1 = 1	0 . 0 = 0 1 . 0 = 0	0 . 1 = 0 1 . 1 = 1

(ii)

Input	Identities			
I	$I + I = I$	$I + I' = 1$	$I \cdot I = I$	$I \cdot I' = 0$
0 1	0 + 0 = 0 1 + 1 = 1	0 + 1 = 1 1 + 0 = 1	0 . 0 = 0 1 . 1 = 1	0 . 1 = 0 1 . 0 = 0

(Please note 0′= 1 and 1′ = 0)

(iii) The rules (given without proof)

$I_1+I_2=I_2+I_1$;

$I_1 . I_2 = I_2 . I_1$;

$I_1+(I_2+I_3)=(I_1+I_2)+I_3$;

$I_1.(I_2.I_3)=(I_1.I_2).I_3$

(iv) The rules (given without proof)

$I_1 . (I_2+I_3) = (I_1 . I_2 + I_1.I_3)$;

$I_1+I_2.I_3=(I_1+I_2) . (I_1+I_3)$

(v) Demorgan's Laws:

$(I_1+I_2)' = I_1'.I_2'$

$(I_1.I_2)' = I_1' + I_2'$

(Very important laws for algebraic simplification.)

(vi) Complement of complement of a number is the Number itself

I	I'	$(I')'$
0	1	0
1	0	1

so $(I')' = I$

Figure 3.3: The Rules of Boolean algebra

All the rules and identities as given in Figure 3.3 can be used for simplification of Boolean function. This is explained with the help of following example.

Example: Simplify the Boolean function:

$F = ((A'+B')' + (A.B)')'$

Solution:

$F = ((A'+B')' + (A.B)')'$

$F = ((A'+B')')' . ((A.B)')'$ (Using Demorgan's Law)

$= (A'+B') . (A.B)$ Using Rule (vi)

$= (A.B) . (A'+B')$ Reversing the terms - Rule (iii)

$= ((A.B).A') + ((A.B) . B')$ Using Rule (vi) taking (A.B) as I_1

$= ((A.A').B) + (A.(B.B')$ Using Rule (iii)

$= 0.B + A.0$ Using Rule (ii)

$= 0+0$ Using Rule (i)

$= 0$

$F = 0$

You can check the above using the following Truth Table

A	B	A′	B′	(A′+B′)	(A.B)	(A′+B′)′	(A.B)′	(A′+B′)′ + (A.B)′	((A′+B′)′+(A.B)′)′
0	0	1	1	1	0	0	1	1	0
0	1	1	0	1	0	0	1	1	0
1	0	0	1	1	0	0	1	1	0
1	1	0	0	0	1	1	0	1	0

The Boolean algebra is very useful in simplification of logic circuits. It is explained in the next section.

3.4 LOGIC CIRCUITS

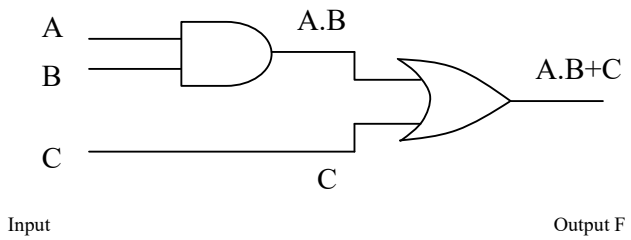
A logic circuit performs the basic operation on binary data. The operation of a logic circuit can be represented using a Boolean function, or in other words a Boolean function can be implemented as a logic circuit. The three basic gates that can be used for drawing logic circuits are - AND, OR and NOT gates. Consider, for example, the Boolean function: -

F (A,B,C) = A.B+C

The relationship between this function and its binary variables A, B, C can be represented in a truth table as shown in Figure 3.4(a). Figure 3.4(b) shows the corresponding logic circuit.

Inputs			Output
A	B	C	F= A.B+C
0	0	0	F = 0 . 0 + 0 = 0
0	0	1	F = 0 . 0 + 1 = 1
0	1	0	F = 0 . 1 + 0 = 0
0	1	1	F = 0 . 1 + 1 = 1
1	0	0	F = 1 . 0 + 0 = 0
1	0	1	F = 1 . 0 + 1 = 1
1	1	0	F = 1 . 1 + 0 = 1
1	1	1	F = 1 . 1 + 1 = 1

(a) Truth Table



(b) Logic Circuit

Figure 3.4: Truth table & logic diagram for F = A . B + C

While fabricating these logic circuits, it is expected that fewer gate types are used; however, these gate types should be able to create all kinds of circuits. Therefore, functionally complete set of gates, which are a set of gates by which *any* Boolean function may be implemented, are used to fabricate the logic circuits. Examples of functionally complete sets are: [AND, OR, NOT]; [NOR]; [NAND] etc. NAND gate, also called universal gate, is a special gate and can be used for fabrication of all kinds of circuits. You may refer to further readings for more details on Universal gates.

Check Your Progress 1

1) What is a logic gate? What is the meaning of term Universal gate?

.....

.....

2) Prove the identity $I_1+I_2.I_3 = (I_1+I_2).(I_1+I_3)$ using Truth Table

…….

…….

3) Simplify the function F = ((A′+B) ′+(A.B′)′)′

…….

…….

…….

…….

…….

4) Draw the logic diagram of the function before simplification.

.....

.....

.....

……..

5) Draw the logic diagram of the simplified function.

.....

.....

…….

3.5 COMBINATIONAL CIRCUITS

Combinational circuit is an interconnection of gates, which produces one or more output based on some Boolean function for which it has been designed. A good combinational circuit does not include feedback loops. A combinational circuit is also represented using equivalent truth table or a Boolean function.

The output of the combinational circuit changes instantaneously with respect of input, though some delay is introduced due to transfer of signal from the circuit. This delay is dependent on the *depth* which is computed as number of gates in the longest path from input to output. For example, the depth of the combinational circuit of Figure 3.5 is 2.

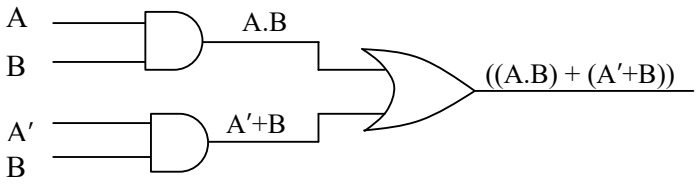


Figure 3.5: A two level AND-OR combinational circuit

Combinational circuits are primarily used to create the computational circuits of computer system logic; therefore, efficient design of combinational circuit may enhance the performance of a computer. Thus, one of the design goals of combinational circuits design is to minimize the number of gates in a combinational circuit. The constraints for combinational circuit design are:

- a combinational circuit should have limited depth.
- The number of input lines to a gate and number of gates to which the output of gate is fed, should be limited.

How can it be achieved? The following sub-sections explains the basic issues for combinational circuit design.

3.5.1 Canonical and Standard Forms of a Boolean expression.

An algebraic expression can exist in two standard forms:

i) Sum of Products (SOP) form
ii) Product of Sums (POS) form

Sum of product form

A SOP expression consists of terms consisting of operator (AND) which are joined by + operator (OR). For example, the expression $A'.B.C + A.B$ is an expression consisting of three variables A, B and C. This expression is in SOP from with two terms $A'.B.C$ and $A.B$. Each of these terms consists of product of variables using AND (.) operator, and the two terms are joined by an OR(+) operator, that is why the name Sum of Products form. In a SOP expression, a term which includes every variable in normal or complement form is called a minterm or standard product term. For example, in the above expression the term A′.B.C is a minterm, but A.B is not a minterm. However, if needed the term A.B can be converted to two minterms as:

$$A.B = A.B.(C+C') = A.B.C + A.B.C'.$$

In addition, please note that the value of minterm be ONE for exactly one possible combination of input values of A, B and C variables. For example, the minterm A′.B.C will have a value 1 if A′=1 and B=1 and C=1. i.e., A=0, B=1 and C=1; for any other combination of values of A, B, C the minterm will have a ZERO value.

Interestingly, the number of minterms depends on number of variables. Given, n variables, the number of minterms will be 2^n. For example, for two variables,

$n=2 \Rightarrow 2^n=2^2=4$. The possible minterms for two variables are shown in the Figure 3.6.

Variables		Minterm	
A	B		
0	0	A′B′	m_0
0	1	A′B	m_1
1	0	AB′	m_2
1	1	AB	m_3

Figure 3.6: Minterms for two variables

A function can be represented as a sum of minterms, for example a function F in two variables using minterms A′B + AB can be represented as:

F(A,B)= A′B + AB
which can be represented as:
F(A,B) = ∑ (1,3)
(Please note that A′B is minterm m_1 or 1 and AB is minterm m_3 or 3,

Product of Sum form

In this form an expression is written as the product (AND) of the terms, which use OR(+) as the basic operation, e.g. a three variables expression (A+B′+C).(A′+B′) is in POS form having the two terms (A+B′+C) and (A′+B′). Both the terms uses + operator and are joined by the AND operator, thus, the name Product of Sum form. In POS form a term, which include all the variables either in normal or complemented form is called a maxterm. For example, the expression (A+B′+C).(A′+B′) has a maxterm (A+B′+C). A maxterm can have a value 0 for exactly one combination of input. For example the maxterm A+B′+C will have value 0 if A=0, B′=0 and C=0, which is A=0, B=1 and C=0.
For any other combination of values of A, B and C, it will have a value 1.
The following Figure shows the maxterms. Please note that the output of maxterm is 0, only for the given combination of input.

A	B	Maxterm	
0	0	A+B	M_0
0	1	A+B′	M_1
1	0	A′+B	M_2
1	1	A′+B′	M_3

Figure 3.7: Maxterms for two variables

Example: Represent the function, whose SOP form is given below into an equivalent function in POS form.

F(A,B) = A′.B + A.B or F(A,B) = ∑ (1,3) or the truth table representation is:

A	B	F(A,B)
0	0	0
0	1	1
1	0	0
1	1	1

Solution:
The complement of this function in SOP form is represented as (the minterms that has 0 as function output).

$$F'(A,B) = A'.B' + A.B' \quad \text{--------(1)}$$

Taking complement of equation (1) , you will get the function F in POS form.

$$(F'(A,B))' = (A'.B'+A.B')'$$

$$\Rightarrow \quad F(A,B) = (A'.B')'+(A.B')'$$

$$= ((A')'+(B')').(A'+(B')')$$

$$= (A+B) . (A'+B)$$

Form table you can determine that function in POS form is:

$F(A,B) = \prod (0,2)$ as the terms are M_0 and M_2

Thus, you can see:

$$F(A,B) = \sum (1,3) = \prod (0,2)$$

(SOP form) (POS form)

With this background of minterm and maxterm, you now are ready to perform the process of grouping of minterms, which will result in minimization of gates needed for a digital circuit. This is discussed in the next section.

3.5.2 Minimization of Gates

The simplification of Boolean expression is useful for the design of a good combinational circuit. There are several methods of doing so, however in this unit only the following two methods are discussed in details.

- Algebraic Simplification
- Karnaugh Maps

Algebraic Simplification

The following example explains the process of algebra simplification

Example : Simplify the function: $F(A,B,C) = \sum (0,1,4,5,6,7)$

Solution: Expanding the Minterms of the functions as:

$$F(A,B,C) = A'.B'.C'+A'.B'.C + A.B'.C' + A.B'.C + A.B.C' + A.B.C$$

$$= A'.B'(C'+C) + A.B'.(C'+C) +A.B.(C'+C)$$

$$= A'.B'.1 +A.B'.1+A.B.1 \quad (\text{as } C'+C = 1)$$

$$= (A'.B'+A.B') + A.B$$

$$= B' (A'+A) + A.B$$

$$= B'+A.B$$

Please note that C input has no effect on the function.

The truth table for the function and the equivalent expression is:

A	B	C	F(A,B,C) = ∑ (0,1,4,5,6,7)	B′+A.B
0	0	0	1	1
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Thus, the logic circuit for the simplified equation F(A,B,C) = AB+B′

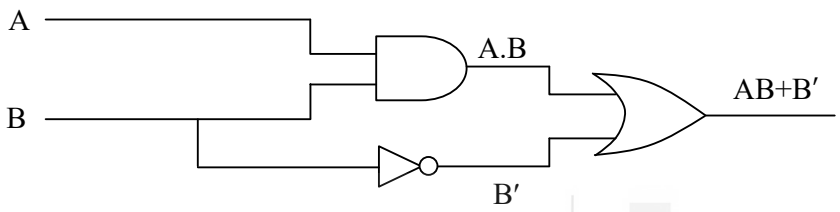


Figure 3.8: Simplified logic function using algebraic Simplifications

The logic diagram of the simplified expression is drawn using one NOT, OR and AND gate each.

The algebraic simplification becomes cumbersome because it is not clear which simplification should be applied next. The Karnaugh map is a simplified process of design of logic circuit using graphical approach. This is discussed next.

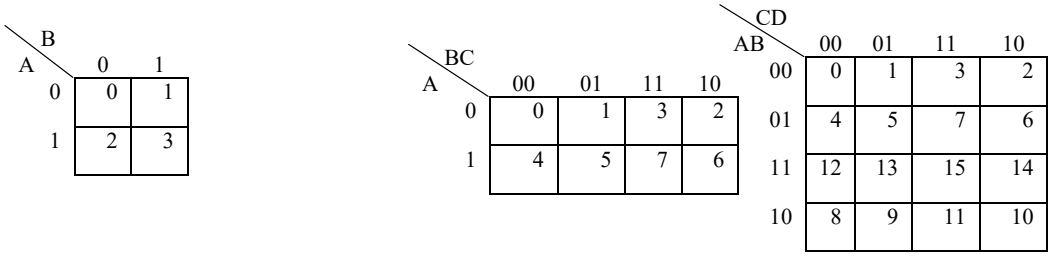
Karnaugh Maps

Karnaugh map is graphical way of representing and simplifying a Boolean function. They are useful for design of circuits involving 2 to 6 variables. The following is the process for simplification of logic circuit using Karnaugh map (K map).

Step 1: Create a Rectangular K-map and Assign binary and decimal equivalent values to each cell

Create a rectangular grid of variables in a function. Figure 3.9 shows the map of two, three and four variables. A map of 2 variables consists of a grid $2^2 = 4$ elements or cells, while a map of 3 variables has $2^3 = 8$ cells and 4 variables has $2^4 = 16$ cells. Please note that the number of cells are same as the maximum possible number of minterms for those number of variables.
Each cell corresponds to a set of variable values, shown on the top or left of the K-map. For example, the values 00, 01, 11, 10 are written on the top of the cells of K-maps of 3 and 4 variables. These represent the values of the variables. For example, for the 3-variable k-map values written on BC side for the first cell 00 indicate B=0 and C=0. Please note that variable values are assigned such that any two adjacent cells (horizontal or vertical) differ only in one variable. For example, cell values 01 and 11 differ in 1 bit only, so are the values 11 and 10. The decimal equivalent values are shown inside the cells. For example, for a 3-varaible K map cell having A=1 and BC=11, which is ABC as 111 is 7. Please note that the sequence of the number is not sequential in 3 variable and 4 variable K maps. This is because of the condition of change in only one variable between two adjacent cells. The decimal equivalent of minterm varaible values are marked inside the cells. For example, decimal equivalent (or minterm equivalent) number placed in the cell having ABCD values as 1111 in the 4 variable k-map is 15.

Please note that bottom row is adjacent to top row; and last column is adjacent to first column as they differ in only one variable respectively.



2-variables K map 3-variables K map 4-variables K-map

Fig. 3.9: K-map of 2, 3 and 4 variables

Step 2: MAP the Boolean function or truth table of Boolean function into K-map. Put a value 1 for every minterm for which the function output is 1.

Step 3: Simplify algebraic expression: Find adjacency of 1's in the K-map. You must find maximum adjacency in a sequence of …, 8, 4, 2. A cell having 1 can appear in more than one adjacencies. Find the maximal adjacencies till all 1's are part of at least one adjacency.

Step 4: Write the Boolean term for each adjacency and join these terms using OR operator.

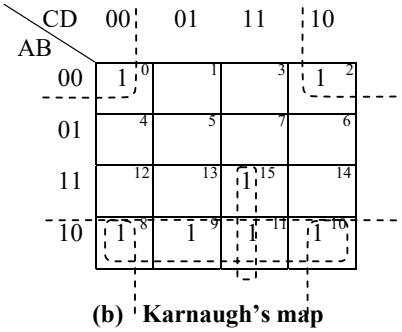
Resultant function is the simplified Boolean expression.

Example: Use K-map for finding the simplified Boolean function for the function $F(A,B,C,D) = \sum (0,2,8,9,10,11,15)$

Solution: The Truth table for the function is given below.

Decimal	A	B	C	D	F
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	0
8	1	0	0	0	1
9	1	0	0	1	1
10	1	0	1	0	1
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	1

(a) Truth table



(b) Karnaugh's map

(i) Adjacency 1: Four Corners (cells Numbered 0, 2, 8, 10)

(ii) Adjacency 2: The bottom Row (cells Numbered 8, 9, 11, 10)

(iii) Adjacency 3: Cell 11 and Cell 15

Figure 3.10: Truth table & K-Map of Function F = ∑ (0, 2, 8, 9, 10, 11, 15)

The three adjacencies of the K-map are shown in the Figure 3.10. You can write the Boolean expression for each adjacency.

1) The adjacency 1 of four corners (cells Numbered 0, 2, 8, 10) can be written algebrically as:

A′.B′.C′.D′ + A′.B′.C.D′ + A.B′.C′.D′ + A.B′.C.D′

= A′.B′.D′ .(C′+C) + A.B′.D′.(C′+C)

= A′.B′.D′ + A.B′.D′ (as C′+C=1)

= (A′+A).B′.D′

= B′.D′ (as A′+A=1)

Please note that an adjacency of 8/4/2 reduces the variables by 3/2/1 respectively.

A direct way of doing so is to identify the variables values of the adjacent cells which does not change, e.g. for this adjacency cell variable ABCD are 0000, 0010, 1000 and 1010. Thus the variable values of B and D does not change in all these 4 cells. In addition, since B and D have zero values among all these four cells, therefore, the expression is B′D′.

2) The four 1's in the bottom row (cells Numbered 8, 9, 11, 10)

The values of variable AB does not change and is 10 for the entire row, therefore, the expression for this adjacency would be A.B′

3) The two 1's in cell 11 and 15

A.B.C.D + A.B′.C.D

= A.C.D.(B+B′)

= A.C.D

Also please infer it directly from values 1111 1011

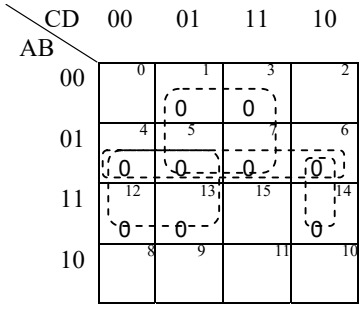
Thus, the simplified Boolean expression using K-Map is

F(A,B,C,D) = B′.D′+A.B′+A.C.D

The simplified expression using K-map is in SOP from. In order to obtain expression in POS form the K-map is created for 0 values and adjacency identified. The following example explains these steps.

Example: Use K-map to find the simplified Boolean function for the function F(A,B,C,D) = ∑ (0,2,8,9,10,11,15) in POS form.

Solution: The truth table is shown in previous example. It can be used to draw K-map for 0 values, which will be for the complement of the function, i.e. F′(A,B,C,D), as:



K-map for F′

Four Adjacencies:

(i) Cells (1,3,5,7) : A′ and D does not change, so the term is A′.D
(ii) 2nd Row (cells 4,5,7,6) : A′ B does not change A′.B
(iii) Cells 4,5,12,13 : B and C′ does not change B.C′
(iv) Cells 6 & 14 : B,C,D′ does not change B.C.D′

Since, you have found the adjacencies of 0's, therefore

F′(A,B,C,D) = A′.D+A′.B+B.C′+B.C.D′

or F(A,B,C,D) = (A′.D+A′.B+B.C′+B.C.D′)′
= (A′.D+A′.B)′ . (B.C′+B.C.D′)′
= (A′.D)′ . (A′.B)′ . (B.C′)′ . (B.C.D′)′
= ((A′)′+D′) . ((A′)′+B′) . (B′+(C′)′) . [(B.C)′+(D′)′]

F(A,B,C,D) = (A+D′) . (A+B′) . (B′+C) . (B′+C′+D), which is in POS form.

In certain digital design situations, some of the input combination has no significance, for example, while designing the circuit for BCD, the output for the input combinations 0000 (digit 0) to 1001 (decimal digit 9) are needed. For the rest of input 1010 to 1111, the output does not matter. Such K-maps are designed using DONOT CARE condition. The output for DONOT CARE input combinations is marked as X in the K-map. The cells marked X can be used for determining the maximal dependencies, but need not be covered as the case is for all 1's output. A detailed discussion on this is beyond the scope of this unit.

What will happen if you went to design circuits for more than 6 variables? With the increase in number of variables K-Maps become more cumbersome and are not suitable. Other methods have been designed to do so, which are beyond the scope of this course.

Check Your Progress 2

1) Draw the truth table for the following Boolean functions:

(i) F(A,B,C) = A′.B.C′+A.B.C+A.B.C′+B.C+A.C

(ii) F(A,B,C) = (A+B) . (A′+C′) . (C′+B′)

……

……

2 Simplify the following using algebraic simplification. And draw the logic diagram for the function so obtained

(i) F(A,B) = (A′.B′+B′)′

(ii) F(A,B) = (A.B+A′.B′)′

……

……

3) Simplify the following Boolean functions in SOP and POS forms using K-Maps. Draw the logic diagram for the resultant function.

F (A,B,C,D) = Σ (0,2,5,7,12,13,15)

……

……

3.6 DESIGN OF COMBINATIONAL CIRCUITS

The digital circuits, are constructed with NAND or NOR gates instead of AND–OR–NOT gates as they are U*niversal Gates*. Therefore, any digital circuit can be implemented using these gates. To prove this point in the following diagram AND, OR and NOT gates are implemented using NAND and NOR gates. This is shown in figure 3.11 to 3.13 below.

NOT Operation:

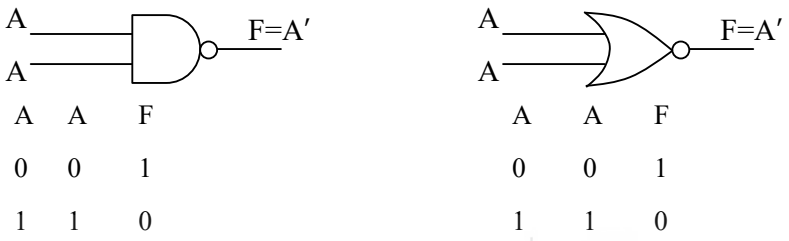


Figure 3.11: NOT Operation using NAND or NOR gates

AND Operation:

Performing AND using NAND gates can be achieved by first performing the NAND or the input followed by inverting the output as shown in Figure 3.12

$F = A.B$

$= ((A.B)')'$

$F = (A \text{ NAND } B)'$

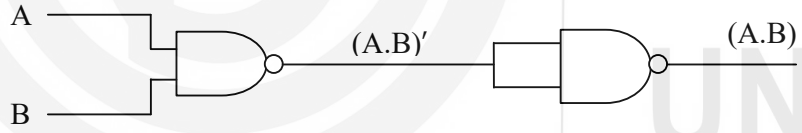


Figure 3.12: Logic circuit of AND Operation using NAND gates

AND operation can also be implemented using NOR gates. The following Boolean expression identifies that first NOR gates are used to invert the A and B input followed by taking NOR of A′ with B′

$F = A.B$

$F = ((A.B)')'$

$= (A'+B')'$

$= A' \text{ NOR } B'$

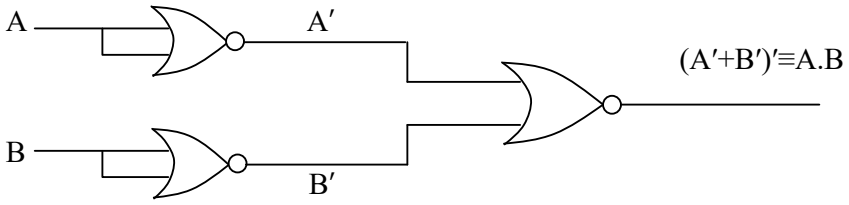


Figure 3.13: Logic circuit of AND Operation using NOR gates

OR Operation:

OR operation can be performed using NAND gate. Please refer to following Boolean expressions:

$F = A+B$

$= ((A+B)')'$

$F = (A'.B')' \Rightarrow A'$ NAND B'

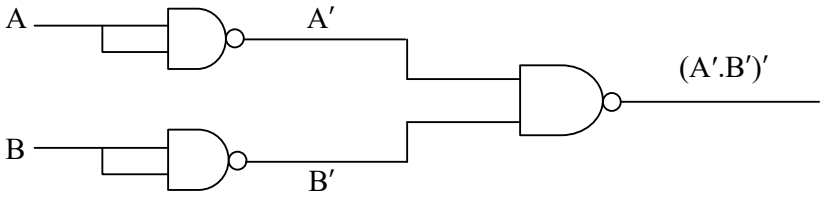


Figure 3.14: Logic circuit of OR Operation using NAND gates

$F=(A+B)$

$F= ((A+B)')'.$

$F = (A \text{ NOR } B)'$

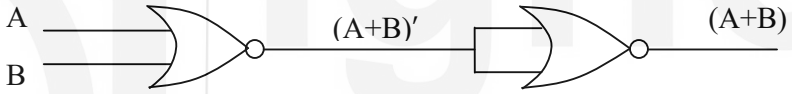
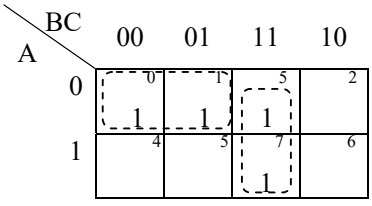


Figure 3.15: Logic circuit of OR Operation using NOR gates

A Boolean function can be implemented using the universal NAND or NOR gates by expressing the function in sum of product form as explained in the following example.

Example: Draw the circuit for $F(A,B,C) = \sum (0,1,3,7)$ using NAND gates.

Solution: Find the optimal Boolean function using K-map in SOP form:



$F(A,B,C) = A'B'+BC$

The AND – OR gate logic circuit for this is:

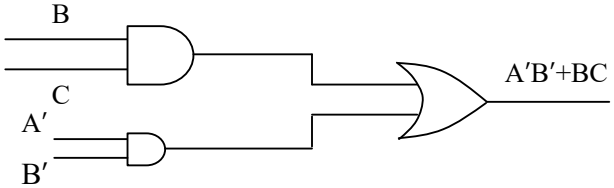


Figure 3.16 Logic Circuit Using AND-OR gate

For NAND gate logic circuit

F(A,B,C) = (A′B′+BC)
= ((A′.B′)′)′+((B.C)′)′
= ((A′. B′)′.(B.C)′)′ (Use of Demorgan's law)
= ((A′ NAND B′).(B NAND C))′
= (A′ NAND B′) NAND (B NAND C)

Thus, the circuit can be made simply by replacing two levels AND-OR circuit by NAND gates:

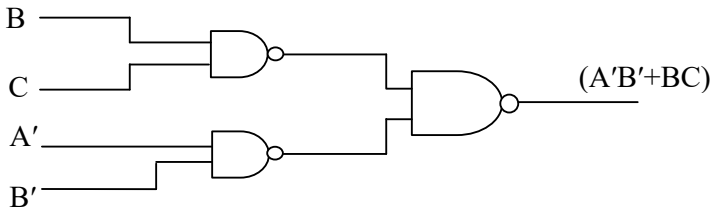


Figure 3.17: Logic Circuit by Replacing AND - OR circuit by NAND gates

A combinational circuit is required to produce a specific set of output for a given step of input. The design of a combinational circuit simply requires the following steps:

Step 1: Make the Truth table for the required design. You must draw the truth table for every output value.

Step 2: Use K-map or any other method to create optimal Boolean function that creates the desired output. One function is designed for each output.

Step 3: Draw the resultant circuit using universal gates.

The next section first design the design of half adder circuit as a combinational circuit. In addition, next section discusses some of the combinational circuits, the design of which is not detailed in this Unit..

3.7 EXAMPLES OF COMBINATIONAL CIRCUITS

In this section, first the combinational circuits design is demonstrated using basic combinational circuits like half and full adders. This is followed by discussion on combinational circuits like decoders, multiplexers etc.

3.7.1 Adders

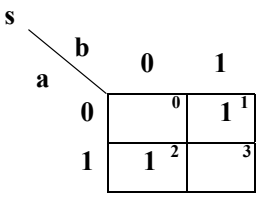
Addition is one of the most common arithmetic operations. In this section two different kinds of addition circuits are designed. The first of the two circuit adds two binary digits and is called a half adder, while the second adds three bits-two addend and one carry bit, and is called a full adder.

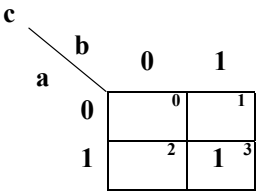
Half Adder:

Let us assume that a half adder circuit is adding two bits a and b to produce one sum bit (s) and one carry bit (c). The following truth table shows this operation. Please note one adding a =1 and b=1 you get a carry as 1 and sum bit as 0 as shown in truth table. The K-maps for the addition is shown in figure 3.18.

a	b	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

(a) Truth table





(b) K- map for sum bit **(c) K-map for carry bit**

Figure 3.18: Truth table and K-maps for half adder

The Boolean expression for them from the k-maps are:

$s = a'b+ab'$ and

$c = a.b$

The logic circuit for the half adder is based on the Boolean expressions are given are shown in Figure 3.19.

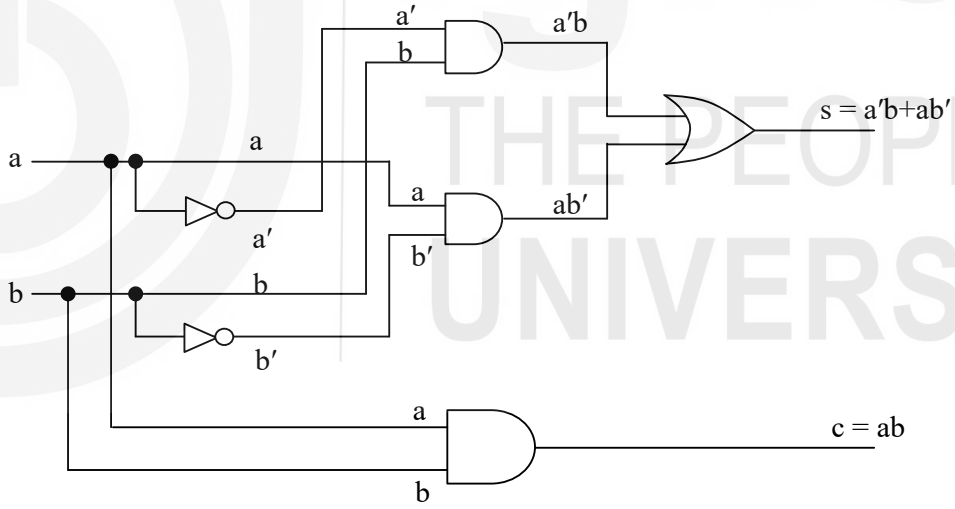


Figure 3.19: The half adder circuit-input addend bits a, b; output sum bit (s) and carry bit (c)

Full Adder:

Full adder is a circuit that adder 3 bits, viz. 2 addend bits and one carry bit. The truth table for full adder is shown in Figure 3.20. Please note that in figure 3.20, c_{in} is carry in bit and c_{out} is carry out bit.

Input					Output	
Decimal equivalent	a	b	c_{in}		Carry out (c_{out})	Sum (s)
0	0	0	0		0	0
1	0	0	1		0	1
2	0	1	0		0	1
3	0	1	1		1	0
4	1	0	0		0	1
5	1	0	1		1	0
6	1	1	0		1	0
7	1	1	1		1	1

Figure 3.20: The Truth Table for Full Adder

Please note that in the truth table, when a = 1, b = 1 and c_{in} =1 , than the output is 11 , which means sum bit (s) is 1 and carry out bit is also 1. The K-map for these are also shown in Figure 3.21



K-map for sum bit

No adjacency

K – Maps for c_{out}

Three adjacencies

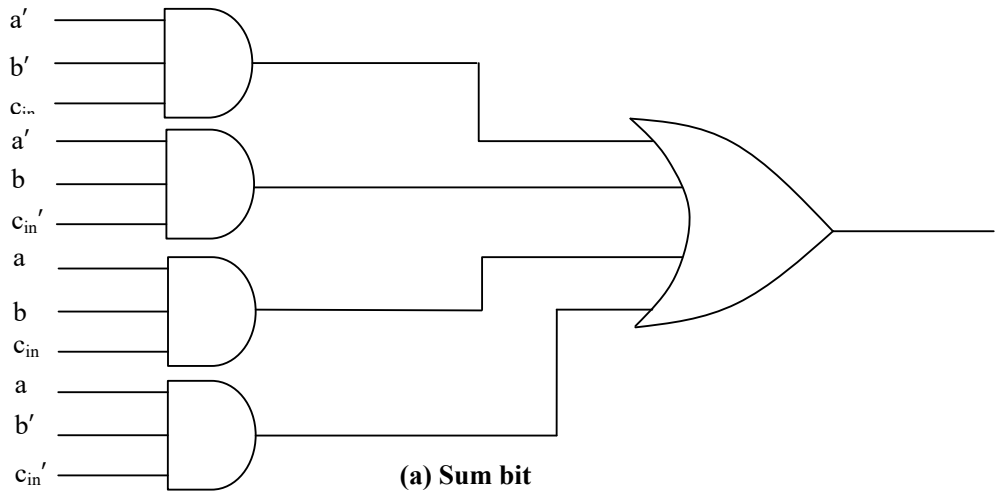
Figure 3.21: The K-maps for Full Adder

The Boolean functions based on the K-maps given in Figure 3.21 are given below:

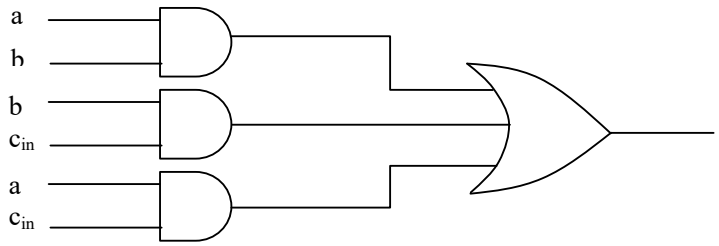
$$s = a'.b'.c_{in} + a'.b.c_{in}' + a\ b\ c_{in} + a\ b'\ c_{in}'$$

$$c_{out} = a\ b + b\ c_{in} + a\ c_{in}$$

Figure 3.22 shows the full adder circuit. Please note that for simplicity the circuits for inverting the input values are not drawn.



(a) Sum bit



(b) Carry Out bit

Figure 3.22: Full Adder

Full adder and half adder only perform bit addition of two operands without or with carry bit respectively. However, binary numbers have several bits e.g. integers can be 4 byte long. How will they be added? This is performed by creating a sequence of full adders, where carry out bit of the lower bit addition is fed as carry in bit of next higher bit addition, as shown in figure 3.23.

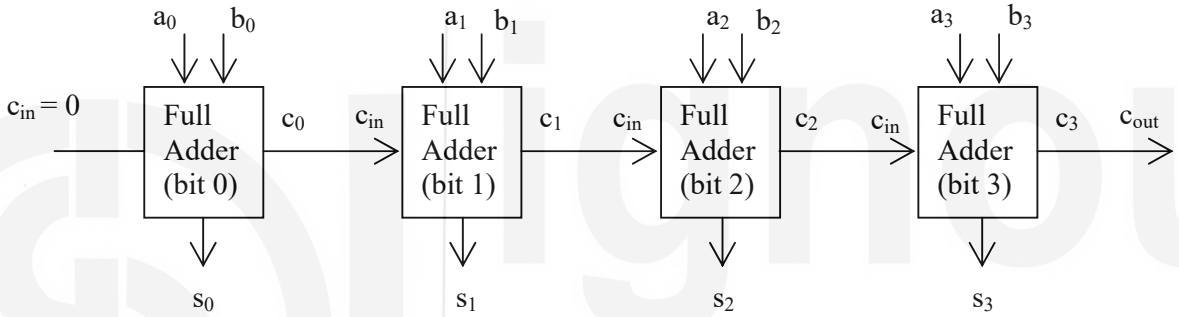


Figure 3.23: Addition of a 4 bit number using 4 full adders

Please note that in the Figure 3.23, the carry out of the previous bit addition is input as carry in bit of the next bit addition. For example, the value of c_0 bit, which is carry out of bit 0 addition, will be available if the full adder of bit 0 has been completed. Drawback of this circuit is the time taken to add the number is large as each full adder will take some signal propagation time, and the addition of the next bit cannot be performed till the c_{in} bit is available. A faster binary adder circuit would predict the carry bit. These are called look-ahead carry adders. It may be noted that carry out of the 0th bit can be computed using the Boolean function $c_0 = a_0.b_0$. The value c_0 is input as c_{in} of full adder of bit 1. The truth table of Figure 3.24 has been drawn for full adder of bit 1. This truth table can be used to design circuit that computes the value of c_1 prior to actual addition.

	Input			**Output**	
Decimal equivalent	**c_0**	**a_1**	**b_1**	**c_1**	**s_1**
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	0	0	1
3	0	1	1	1	0
4	1	0	0	0	1
5	1	0	1	1	0
6	1	1	0	1	0
7	1	1	1	1	1

Figure 3.24: Truth table for Full adder (bit 1)

The k-map for c_1 is shown in Figure 3.25

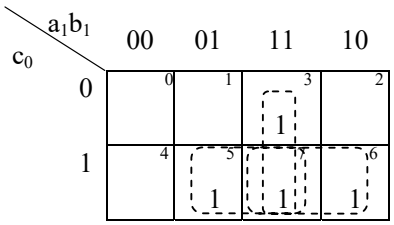


Figure 3.24: K-map for c_1 output of Full adder (bit 1)

There are three adjacencies in the K-map of Figure 3.24. The resultant Boolean function for c_1 would be:

$c_1 = a_1.b_1+c_0.a_1+c_0.b_1$

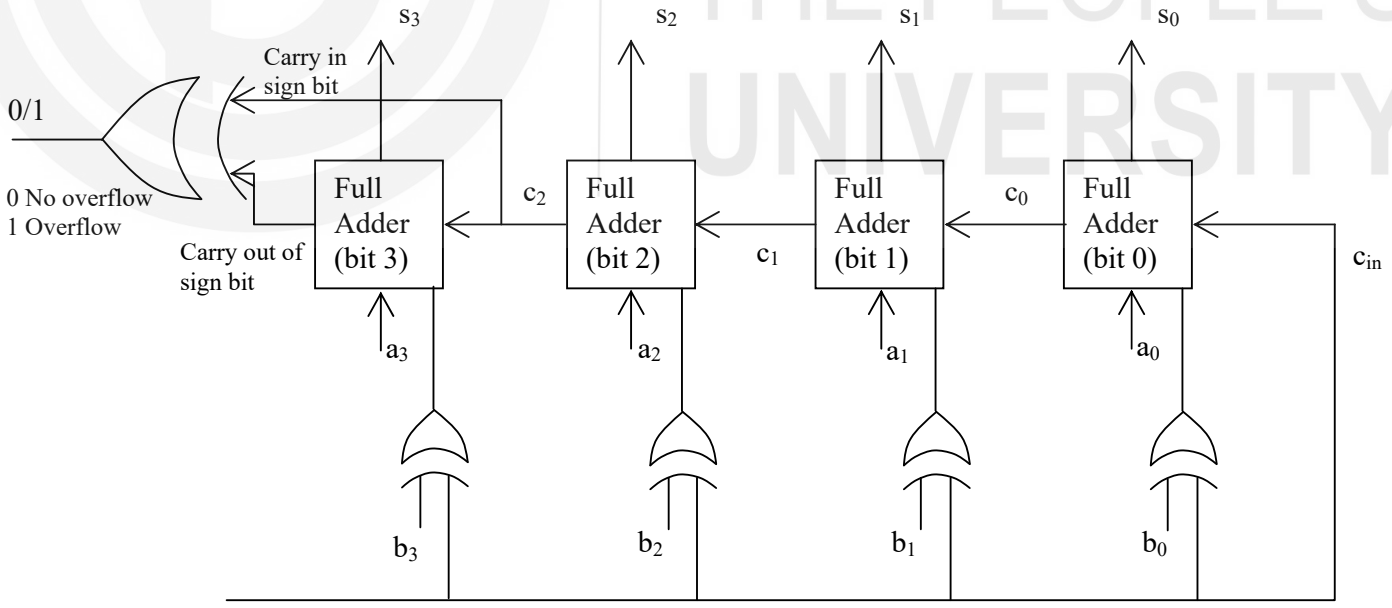
$c_1 = a_1b_1+c_0.(a_1+b_1)$ (Taking c_0 common)

$c_1 = a_1.b_1+a_0.b_0.(a_1+b_1)$ (Replacing c_0 by its equivalent)

Logic circuits can be designed for prediction of carry bits c_0, c_1, etc. and resultant circuits can be implemented along with full adder circuits. You can observe that Boolean expression for higher order carry bits like c_2, c_3 etc. will become more complex, which results in complex logic circuits. Thus, look ahead carry bit adders may be implemented for addition of binary numbers of size 4-8 bits.

Adder-Subtractor Circuit

Adder subtractor circuit is an interesting design, in which a same circuit is used for addition as well as subtraction. This example shows how with some additional logic, you may be able to perform additional operations. ALU is a fine example of extension of such logic. Figure 3.25 shows the circuit of 4 bit adder-subtractor circuit by using full adders.



Mode bit: Addition
Operation mode bit = 0
Subtraction operation mode bit = 1

Figure 3.25: Adder Subtractor circuit using full adders for addition and subtraction of 4-bit 2′s complement numbers

You may please note that the mode bit controls the b input. The following Figure shows the details of operation.

Mode bit = 0		Mode bit = 1	
Input b	Inputs the bits of b input XOR with mode bit value (0)	Input b	Input value after taking XOR of input b and mode bit
0 1	0 XOR 0 = 0 1 XOR 0 = 1	0 1	0 XOR 1 = 1 1 XOR 1 = 0
Thus, when mode bit is 0, the input to full adder is the value of input b.		Thus, when mode bit is 1, the input to adder is the value of 1′s complement of b	
c_{in} = 0 as mode bit = 0 so the circuit adds input a and input b.		c_{in} = 1, so the addition is r = a+b′+1 or r= a+2′s complement of b r = a – b ; subtraction of a and b	

Figure 3.26: Use of Mode bit to control b input in Adder subtractor circuit

Please also note that in 2's complement notation the last bit is treated as sign bit. The overflow condition is checked by finding if the carry into the sign bit and carry out of sign bit are same or not same. In case carry in to the sign bit is not the same as carry out of the sign bit, then overflow is set to 1 (by XOR gate) else overflow is set to 0 (No overflow).

3.7.2 Decoders

Decoder, as the name suggests, decodes the input to one of the output line. Figure 3.27 shows the truth table and logic circuit of a 2 × 4 decoder.

Truth table

Input		Output			
a	b	c	d	e	f
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

The Boolean functions for various output values are

c = a′ b′
d = a′ b
e = a b′
f = a b

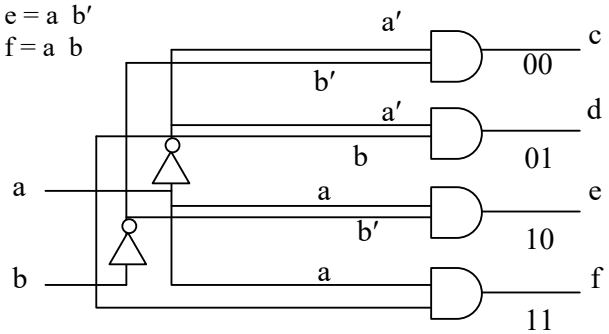


Figure 3.27: 2 × 4 decoder

A decoder line would be selected if it has an output 1. In general, decoder is a very useful circuit for selecting lines and forms the basis of Random Access Memory.

Please note that numbers of output for 2 bit decoder are $2^2 = 4$; hence the name 2×4 decoder. Similarly, the number of output for a 3 bit input would be $2^3 = 8$ and it is called 3×8 decoder.

3.7.3 Multiplexer

A multiplayer allows sharing of a line by multiple inputs. It may be very useful for serialization of data bits over a single output line. The design of a multiplexer is however, different from other combinational circuits as it is the selection lines which control the selection of input line. The following is the truth table of a 4×1 multiplexer. A 4×1 multiplexer selects one of the 4 input lines to be transmitted over a single output. Out of these 4 lines, which will be selected, will be determined by 2 selection lines. How many selection lines will be required for 8×1 multiplexer? Since $2^3 = 8$, so 3 selection lines would be required for 8×1 multiplexer.

Input			
Selection Lines		Input	Output
s_1	s_0		
0	0	I_0	I_0
0	1	I_1	I_1
1	0	I_2	I_2
1	1	I_3	I_3

Please note the values of output can be I_i, where the value of subscript *i* can vary from 0 to 3.

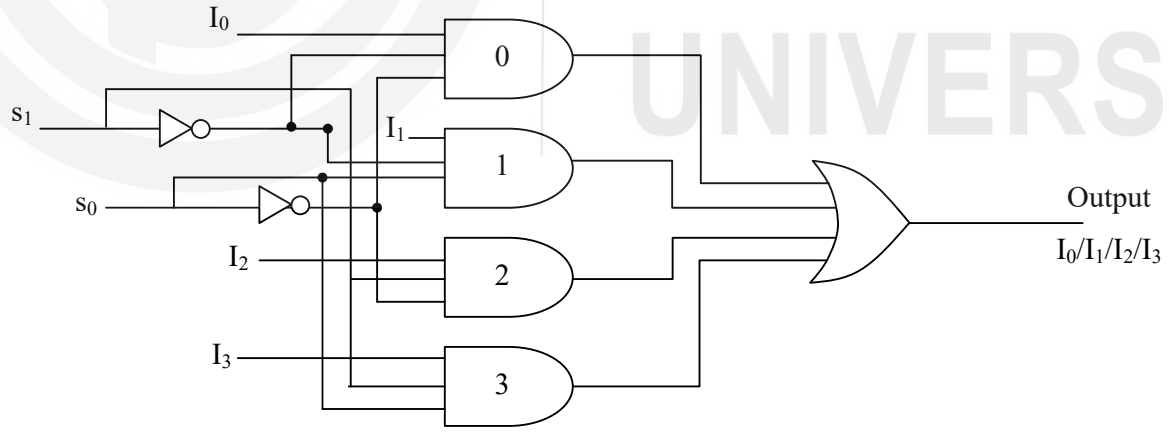


Figure 3.28: 4 × 1 Multiplexer

Please note this is a very important circuit for sharing of an output line across many sources of input.

3.7.4 Encoders

An encoder, in general, is the inverse of a decoder. Based on its input it produces a specific output. For example, the truth table of a 4×2 encoder is shown in Figure 3.29

Input				Output	
I_0	I_1	I_2	I_3	O_1	O_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

Figure 3.29: Truth Table of 4 × 2 encoder

The simple expression for various output can be

$O_1 = I_2 + I_3$

and $O_0 = I_0 + I_1$ Thus, the simple circuit for this encoder is

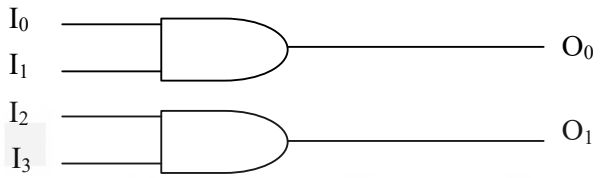


Figure 3.30 Logic Diagram of a simple encoder

3.7.5 Programmable Logic Array

The basic combinational circuits can be implemented using AND-OR-NOT gates. PLAs are circuits which are prefabricated for all possible combination of AND, OR, NOT gates. They can be used to fabricate any kind of logic circuit. They are primarily designed for SOP form of logic circuit.

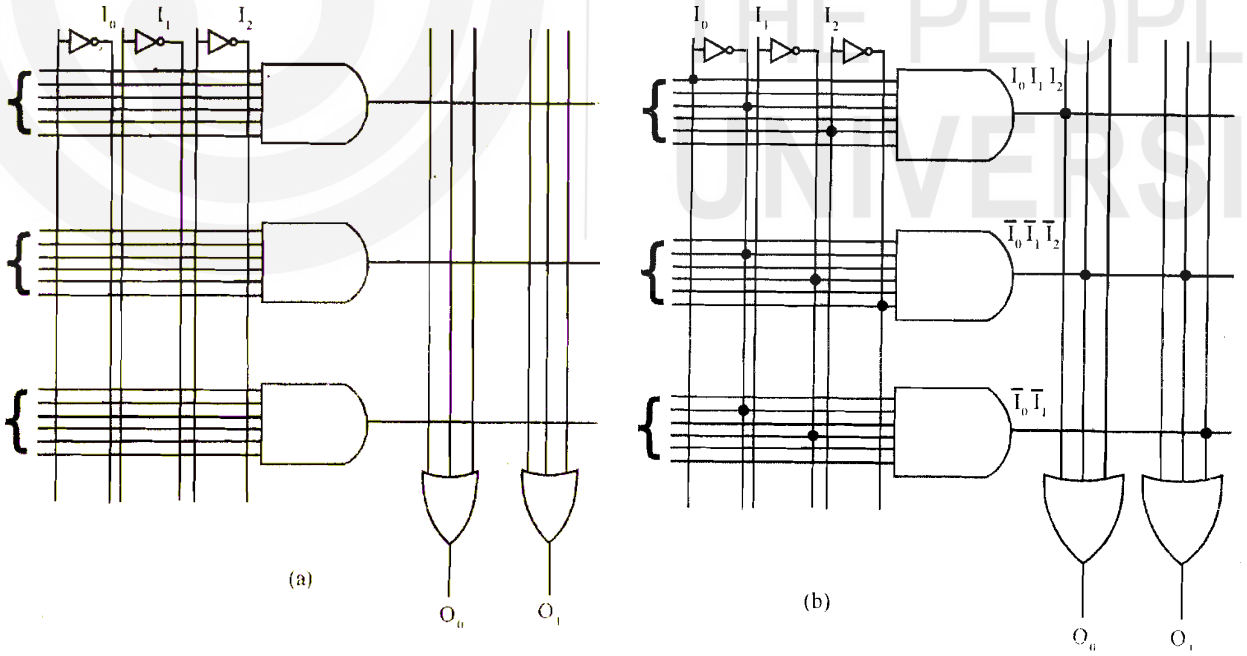


Figure 3.31: Programmable Logic Array

The figure 3.31(a) shows a PLA of 3 inputs and 2 outputs. Figure 3.31(b) shows an implementation of logic function given below using the PLA of figure 3.31 (a).:

$O_0 = I_0 . I_1 . I_2 + I_0' . I_1' . I_2'$

$O_1 = I_0' . I_1' . I_2' + I_0' . I_1'$

3.7.6 Read-only-Memory (ROM)

ROM is an example of use of Programmable Logic Devices (PLD). It stores the binary information using a combinational circuit. The RAM follows the simple sequence. Figure 3.32 shows a ROM of size 4 × 2, which has 4 lines of 2 bits each. Please note the use of 2 × 4 decoder. Also note that wherever the line will be connected an output will appear. These connections are embedded within the hardware. Thus the information of ROM is not lost even after the power failure..

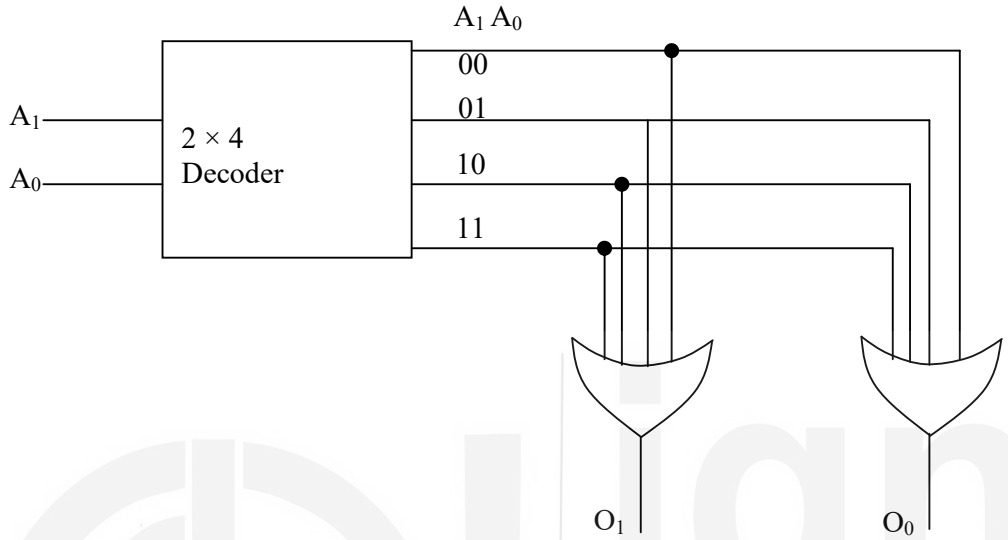


Figure 3.32: ROM Design

Please note that the ROM shown in Figure 3.32 have the following content:

Address Line selection A_1 A_0	ROM content O_1 O_0
0 0	1 0
0 1	0 0
1 0	1 0
1 1	1 0

Please note the number of words in the ROM is 4 which is $2^2 = 4$ as it has 2 address lines. A ROM with 3 address lines will have 8 words. ($2^3 = 8$).

The size of word can be chosen by the designer and, in general, it can be 8, 16, 32 or 64 bit as per the machine firmware designer. The address lines of ROM select any one line as per the address.

Check Your Progress 3

1) Design a combinational circuit, which takes four bit input and produces an output 1 if the input contains three consecutive 1 bits.

……..

……..

2) Draw the logic diagram of the function as above using

(i) AND-OR-NOT gates &

(ii) NAND gate

……..

……

……

3) Consider the circuit of Figure 3.26, what would be the output of the circuit if:

(i) Input a is 1010 and input b is 1100 and mode bit is 0

(ii) Input a is 0010 and input b is 0100 and mode bit is 1

……..

……

……

4) Why is PLA needed?

……

……

……

5) Design a full adder using two half adder circuits.

……

……

……

3.8 SUMMARY

This Unit introduces you to some of the basic concepts relating to computer logic. The Unit first introduces the concept of logic gates, the most fundamental unit of logic circuits. The Unit then explains the process of making simple logic circuits, including combinational circuit. The mathematical foundation of the logic circuit design, the Boolean algebra is also introduced. The Karnaugh's map was used to design simpler circuit. The Unit also explains the desing of different kinds of adders circuit, highlghting , how complex circuit can be desingned using K-map. Finally, the Unit explains some of the most fundamental combinational circuits like decoder, multiplexer, encodes, PLA's etc. It may be noted that the objective of this Unit is not to make you a computer hardware designer, but to introduce you to some of the basic concepts of circuit design.
You can refer to latest trends of design and development including VHDL (a hardware design language) in the further readings.

3. 9 SOLUTIONS/ANSWERS

Check Your Progress 1

1) A logic gate is most fundamental circuit that can be fabricated on a silicon chip. A logic gate operates at signal level to produce simple logic like AND, OR, NOT etc. A Universal gates can be used to implement each and every kind of logic circuit. Two examples of universal gates are NAND and NOR.

2)

I_1	I_2	I_3	$I_1+I_2.I_3$	$(I_1+I_2).(I_1+I_3)$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

3) F = ((A′+B) ′+(A.B′)′)′

= ((A′+B) ′)′ . ((A.B′)′)′ (by Demorgan's Law)

= (A′+B) . (A.B′) (as (a')' = a)

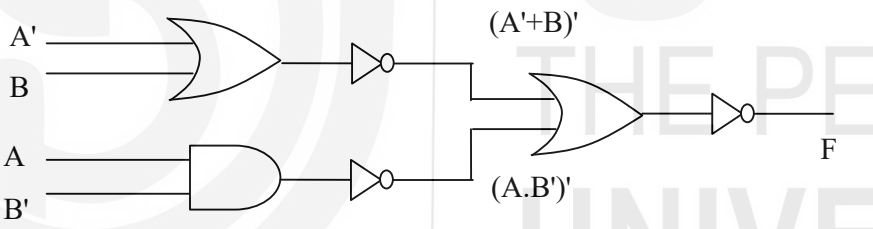
= ((A′+B).A).((A′+B).B')

= ((A'.A)+(B.A)).((A'.B')+B.B'))

= ((0+A.B).(A'.B' +0)

= (A.B.A'.B')=0

4) Draw the logic diagram of the function before simplification.



5) Just the F can be connected to logical 0 input as F=0

Check Your Progress 2

1)

A	B	C	F=A′.B.C′+A.B.C+A.B.C′+B.C+A.C	F= (A+B) . (A′+C′) . (C′+B′)
0	0	0	0	0
0	0	1	0	0
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	1
1	1	1	1	0

2 (i) F(A,B) = (A′.B′+B′)′

= (A'.B')' . (B')' (DeMorgan's Law)

= ((A')'+(B')').B (DeMorgan's Law)

= (A+B).B

= A.B + B.1=B.(A+1)= B

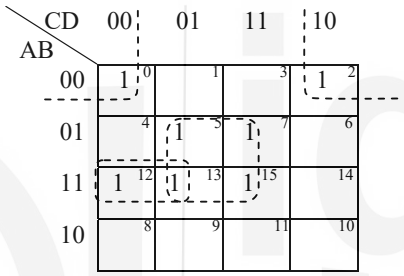
B ——————— F

(ii) F(A,B) = (A.B+A′.B′)′
= (A.B)' . (A'.B')'
= (A'+B').(A+B)
= (A'+B').A + (A'+B').B
= A'.A+B'.A+A'.B+B'.B
= A.B' + A'.B

The logic diagram is same as sum bit of half adder. It is also the A XOR B.

3) Simplify the following boolean functions in SOP and POS forms using K-Maps. Draw the logic diagram for the resultant function.

F (A,B,C,D) = Σ (0,2,5,7,12,13,15)



Three adjacencies

i) Cells 0 and 2: The variables does not change A' B' D'

ii) Cells 12 and 13; The variable does not change A B C'

iii) Cells 5,7,13,15; two variables does not change B D

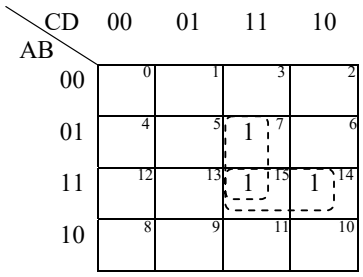
The expression is F=A'.B'.D' + A.B.C' + B.D

Check Your Progress 3

1) The Truth table:

Decimal	A	B	C	D	F
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	0
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	1
15	1	1	1	1	1

The K-map for the Truth table:



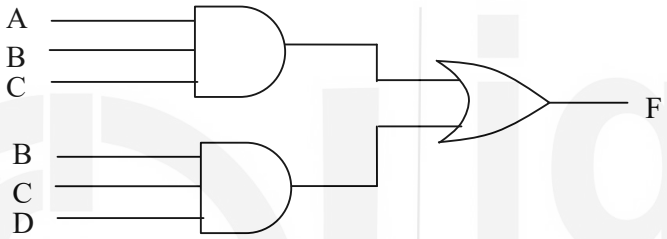
Only two adjacencies: Cells 7 and 15 B C D and

Cells 15 and14 A B C

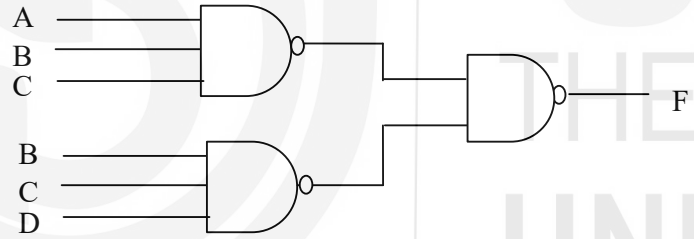
Therefore, the function is: F = A.B.C + B.C.D

2) Logic diagram of the function as above

(i)



(ii)



3) (i) Input a is 1010 and input b is 1100 and mode bit is 0

Bit wise addition will be as follows:

a	1	0	1	0
b	1	1	0	0
c	0	0	0	0
sum bit	0	1	1	0
carry out of sign bit	1			

carry in to sign bit NOT equal to carry out of sign bit,
OVERFLOW

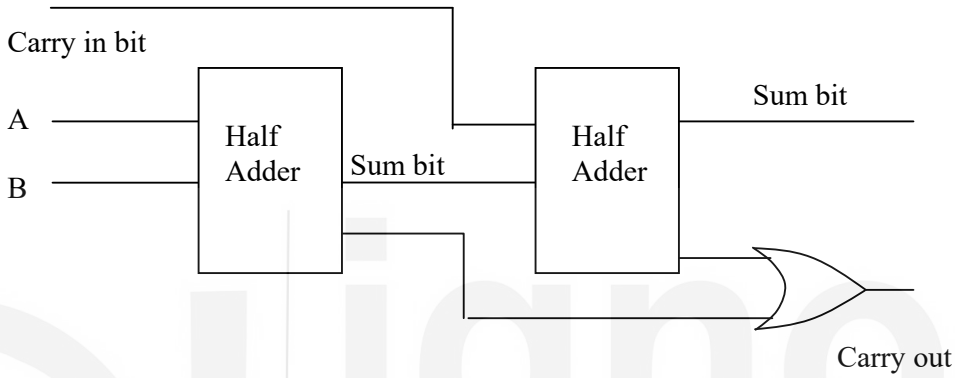
(ii) Input a is 0010 and input b is 0100 and mode bit is 1
Bit wise addition will be as follows:

a	1	0	1	0
b (1's complement)	0	0	1	1
c	0	1	1	1
sum bit	1	1	1	0
carry out of sign bit	0			

carry in to sign bit IS EQUAL to carry out of sign bit,
NO OVERFLOW

4) PLA's can be fabricated as a chip that can be customised as per the need of the SOP logic.

5) A half adder adds two addend bits, whereas a full adder adds the two addends and previous carry bit, therefore, one half adder will be needed to add two addend bits, and second half adder will be needed to sum the sum of first half adder and previous carry bit. The output carry will be set, if any of the two half adder produce the carry out. The following block diagram shows this construction:



UNIT 4 LOGIC CIRCUITS – SEQUENTIAL CIRCUITS

Structure **Page Nos.**

4.0 Introduction
4.1 Objectives
4.2 Sequential Circuits: The Definition
4.3 Latches and flip-flops
4.3.1 Latches
4.3.2 Flip-Flop
4.3.3 Excitation Tables
4.3.4 Master Slave Flip Flops
4.3.5 Edge Triggered Flip-flops
4.4 Sequential Circuit Design
4.5 Examples of Sequential Circuits
4.5.1 Registers
4.5.2 Counters Circuit
4.5.3 Synchronous Counters
4.5.4 Random Access Memory
4.6 Summary
4.7 Solutions/ Answers

4.0 INTRODUCTION

The first Unit of this Block explained the basic structure and process of instruction execution. Unit 2 provided a detailed description of data representation and Unit 3 presented the concepts of basic functional unit of a computer, viz. the logic gates and combinational circuits. In this unit, you will be introduced to one of the most fundamental circuit that can store one bit of data called flip flops. The unit also explains how flip-flops and additional logic circuit can be used to make registers, counters, sequential circuits etc. Finally, the Unit also introduces you to simple design of a sequential circuit.

4.1 OBJECTIVES

After going through this unit you will be able to:

- explain the functioning of flip-flops;
- determine the behaviour of various latches;
- construct excitation table of a flip-flop;
- explain circuits of a computer system like registers, counters etc.

4.2 SEQUENTIAL CIRCUITS: THE DEFINITION

A sequential circuit is an interconnection of combinational circuits and storage elements. The storage elements, called flip-flops, store binary information that indicates the state of sequential circuit at that point of time.

Figure 4.1 highlights that a sequential circuit may involve combinational circuits (which were discussed in Unit 3) the flip-flops (which are discussed in this unit) and a system clock, which is a useful timing device of a computer system.



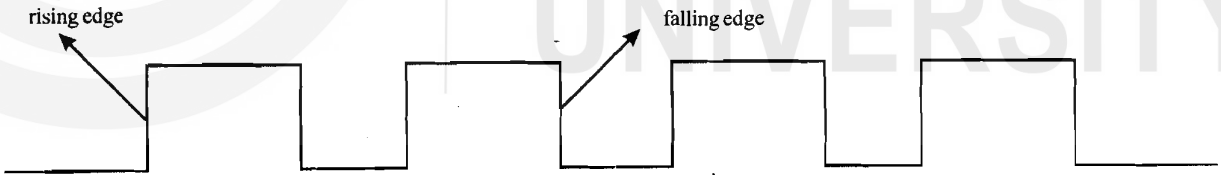
Figure 4.1: Block Diagram of sequential circuits.
(Ref: M. Morris Mano, Charles R. Kime: Logic and compute design fundamental, 2nd Edition, Pearson Education)

The sequential circuits are time dependent. The present state of a combinational circuit is identified by the present output of flip-flop. This output may change over a passage of time and can also be used as one of the input. This change in state can occur either in synchronous or asynchronous manner with respect to system clock.

Synchronous circuits use flip-flops and their state can change only at discrete intervals. Asynchronous sequential circuits are regarded as combinational circuit with feedback path. Such circuits may unstable at times, when the propagation delays of output to input are small. Thus, complex asynchronous circuits are difficult to design.

Clock Pulse and sequential circuits

A sequential circuit uses clock pulse generator, which gives continuous clock pulse to synchronize change in the state of the circuit. Figure 4.2 shows the form of a clock pulse.



Clock pulse

Figure 4.2: Clock signals of clock pulse generator

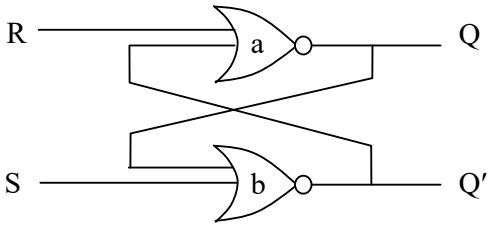
A clock pulse can have two states, viz. 0 or 1, which are also called disabled or active state. Flip-flops are allowed to change their states, in general, with the rising or falling edge of the clock pulse, so as to make stable changes in states of the flip-flops.

4.3 LATCHES AND FLIP-FLOPS

A ***latch*** is a storage element with basic logic circuit, which can store 1-bit of data. It itself is a sequential circuit. Flip-flops are constructed using latches and have more complex timing sequences than latches. Therefore, in order to learn flip-flops, learning the basic concept of latches in very useful, which is discussed next.

4.3.1 Latches

A basic latch can be constructed using either two NOR or two NAND gates. Figure 4.3 (a) shows logic diagram for S-R latch using NOR gates. This latch has two inputs viz. S and R for *S*et and *R*eset respectively; and one output Q. Please note Q′ output is complement of the output Q. This flip flop exhibits two states called SET state (when the flip-flop output Q is1, that is Q′=0) and RESET state or clear state (Q=0; Q′=1).



(a) Logic Diagram

S	R	Q	Q′	Comment
0	0	0/1	0/1	No Change in State
0	1	0	1	Reset State
1	0	1	0	Set State
1	1	-		Undefined Input

(b) Truth Table

Figure 4.3: SR Latch using NOR gates

The following table shows the truth table for NOR gates using in the S-R latch of Figure 4.3 (a).

The Truth tables for NOR gates of Figure 4.3

NOR gate Marked 'a'				***NOR gate Marked 'b'***			
	Input		Output		Input		Output
	R	**Q′**	**Q**		S	Q	Q′
0	0	0	1	0	0	0	1
1	0	1	0	1	0	1	0
2	1	0	0	2	1	0	0
3	1	1	0	3	1	1	0

Let us examine the latch in more details. Assume that initially latch is in clear state, i.e. Q=0 and Q′=1; also assume that both S and R input are 0. The states of the latch will be as follows (refer to the NOR gate truth table given above):

Gate 'a'
Input R Q′ :: 0 1 ⇒ Output (Q) 0
Gate 'b'
Input S Q :: 0 0 ⇒ Output (Q′) 1
⎫ Output of latch stays in CLEAR state

(i) Setting the latch:
Now assume that S is changed to 1and R remains 0 during this time, then the output of Gate 'b' will change first:

S Q :: 1 0 ⇒ Q′ will become 0

Gate 'a' now has the following input:

R Q′ :: 0 0 ⇒ Q will be set to 1.

Gate 'b' now has the following input

S Q :: 1 1 ⇒ Q′ will stay at 0.

⎫ SET state

Thus, Flip-flop will be in SET state.
Finally, after some time S will become 0;

At that time, gate 'a'
R Q′ :: 0 0 Q stays at 1
Gate 'b'
S Q :: 0 1 Q′ stay at 0
⎫ Latch will stay in SET State

(ii) *Reset the latch:*
Now assume that input S remains at 0 and input R is changed to 1, also assume that at this time the latch is in Set state (Q = 1 & Q′ = 0), then the output of Gate 'a' will change as
Gate 'a'

R Q′:: 1 0 ⇒ Q will become 0.
Gate 'b'
S Q :: 0 0 ⇒ Q′ will become 1

Latch is in Reset state.

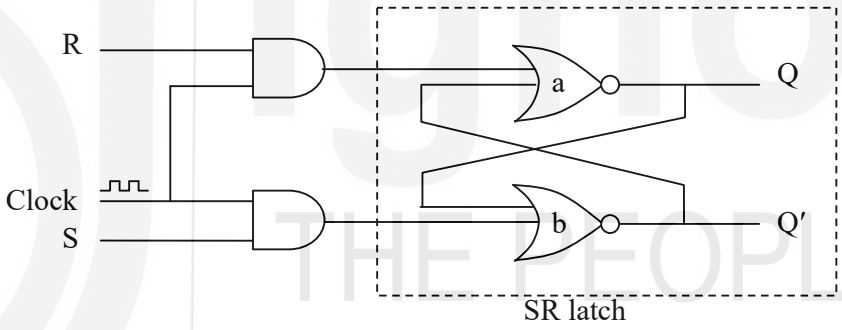
Once again, when S and R both input will become 0, latch will remain in RESET state.

iii) *When both S and R become 1 simultaneously, then?*

A basic S-R latch, in general, changes state at any time, which may result in asynchronous changes in Q output, which can make system unstable. Therefore, latches are constructed with controlled input using clock. This is explained next.

SR latch with Clock

The following diagram shows an SR latch which changes its data only with the occurrence of a clock pulse.



(a) Logic Diagram

Clock(c)	S	R	Present State Q_t before the clock pulse	Next State/Q_{t+1} after occurrence of clock pulse.	Comments
0	Any	Any	0/1	0/1	No change in state
1	0	0	0/1	0/1	No change in state
1	0	1	0/1	0	Reset the latch
1	1	0	0/1	1	Set the flip-flop
1	1	1	0/1	-	Not defined.

(b) Characteristic Table

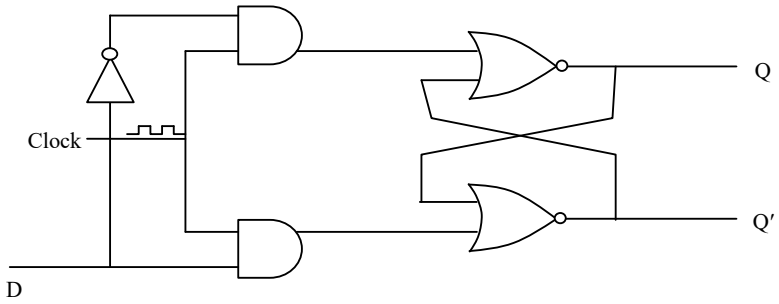
Figure 4.4: R-S latch with clock.

Operations on this clocked SR latch are given below:

1) If no clock signal i.e. clock=0 ⇒ No change in state of latch.
2) Presence of clock signal
   (i) if S=0 and R=0, No change in state/output stays same as earlier state.
   (ii) if S=1, R=0, then next state is the SET state Q=1 & Q′ =0
   (iii) if R=1 S=0, then next state is the RESET state Q=0 & Q′=1
   (iv) if both S and R become 1, then next state/output is not defined.

D Latch

The D (data) latch is modification of RS latch. D latch only uses one input named D, it stores the value of D in the latch, e.g. if the D input is 1, then the next state of latch will also be 1. Figure 4.4 shows the clocked D latch.



(a) Logic Diagram

D	Q_t	Q_{t+1}	When clock pulse occurs
0	0/1	0	Clear the latch
1	0/1	1	Set the latch

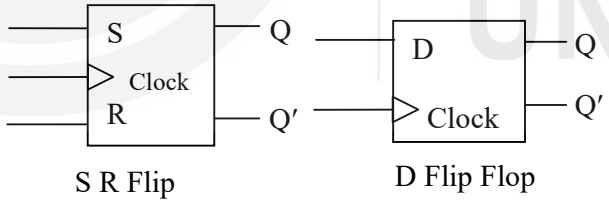
(b) Characteristic Table

Figure 4.5: D latch with clock

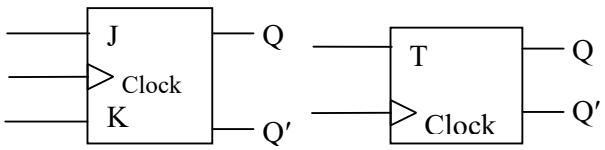
You may please go through the circuit and identify various changes in Q, Q′ with D as shown for SR latch.

4.3.2 Flip-Flops

Latches suffer from the problem due to frequent changes of output, e.g. the output of latch may change depending on the value of R and S input, which may change from 1 to 0 or vice-versa during a single clock pulse. Therefore, they are less suitable for sequential circuits. Flip-flops add more circuitry in latches so that changes in states occur during the rising or falling edge of clock pulse (these are called edge triggered flip-flop). R-S latch with clock can be used with additional circuits to make R-S flip-flop. The flip-flops can also be represented using a block diagram. Figure 4.6 shows the block diagram of basic flip-flops. Please note that in the block diagram the arrow head in front of the clock signal represents that the flip-flop will respond to input during the leading or rising edge (when transition from 0 to 1 takes place) of the clock



S R Flip D Flip Flop



J K Flip T Flip Flop

(Edge trigger is shown by arrow in clock)

Figure 4.6: Graphical Symbols of basic Flip-Flops

JK flip is almost identical to SR flip-flop, except the last combination of J = 1 and K = 1 is used to complement the current state of the flip-flop. T-flip-flop is obtained by joining the J and K input, thus, it shows just two input values. When T = 0, there is no change of state and at T = 1, the current state is complemented. The following figure shows the characteristics table for the basic flip-flops shown in Figure 4.7

SR Flip-flop			
S	R	Q_{t+1}	Comments
0	0	Q_t	No Change in state
0	1	0	Clear state
1	0	1	Set state
1	1	-	Not Defined

JK Flip-Flop			
J	K	Q_{t+1}	Comments
0	0	Q_t	No Change in state
0	1	0	Clear state
1	0	1	Set state
1	1	Q'_t	Complement of Q_t

D Flip-flop		
D	Q_{t+1}	Comments
0	0	Clear State
1	1	Set State

T Flip-flop		
T	Q_{t+1}	Comments
0	Q_t	No Change in state
1	Q'_t	Complement of Q_t

Figure 4.7: Characteristic Table for flip-flops

4.3.3 Excitation Tables

The characteristic tables of flip-flops as shown in Figure 4.7 show how the state of flip-flop will change to the next state based on the present state and input values. The characteristic tables are used for analysis of the sequential circuits. While designing sequential circuits, you need to consider for what transition what possible input combinations would be required. This is done with the help of excitation table. Figure 4.8 shows excitation tables for the basic flip-flops.

Q_t	Q_{t+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

(a) JK Flip flop

Q_t	Q_{t+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

(b) SR Flip flop

Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

(c) D Flip flop

Q_t	Q_{t+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

(d) T Flip flop

X-denotes *DONOT CARE* condition.

Figure 4.8: Excitation Tables for basic flip-flops

Q_t and Q_{t+1} indicate present and next state of a flip flop, respectively. Symbol X in the table means do not care condition i.e. it does not matter whether the input is 0 or 1.

How these excitation tables are created? This is explained with an example of creation of excitation table of JK Flip flop.

a) The state transition from $Q_t = 0$ to $Q_{t+1} = 0$

(i) As both Q_t and Q_{t+1} are 0 it means that there is no change in the state of flip flop, which can be achieved by J=0, K=0;

(ii) Using the input, J=0, K=1, the flip flop can be RESET, i.e. $Q_{t+1} = 0$.

b) The state transition from $Q_t = 0$ to $Q_{t+1} = 1$

(a) Using the input, J=1, K=0, the flip flop is SET, i.e. $Q_{t+1} = 1$

(b) Using the input, J=1, K=1, the flip flop is complemented from Q_t having a value 0 to $Q_{t+1} = 1$

c) State transition from $Q_t = 1$ to $Q_{t+1} = 0$

(a) Using the input, J=0, K=1, flip flop is RESET, i.e. $Q_{t+1} = 0$

(b) Using the input, J=1, K=1, the flip flop is complemented from Q_t having a value 1 to $Q_{t+1} = 0$

d) For state transition from $Q_t = 1$ to $Q_{t+1} = 1$

(a) Using the input, J=0, K=0, no change in flip flop so $Q_{t+1} = 1$

(b) Using the input, J=1, K=0, flip flop is SET, i.e. $Q_{t+1} = 1$

These entire set of input for various transitions can be summarized in the table below:

Present State (Q_t)	**Next State** (Q_{t+1})	**Input J and K**	**Input using DONOT CARE**
0	0	(i) J=0, K=0 (ii) J=0, K=1	**J=0, K=X**
0	1	(i) J=1, K=0 (ii) J=1, K=1	**J=1, K=X**
1	0	(i) J=0, K=1 (ii) J=1, K=1	**J=X, K=1**
1	1	(i) J=0, K=0 (ii) J=1, K=0	**J=X, K=0**

The excitation table has been derived for J-K flip-flop as above. You may draw the excitation table for all other flip-flops using the same method.

Check Your Progress 1

1. What is a sequential circuit? How are sequential circuits different from combinational circuits?

.....

……

……

2. What is a latch? How is different from a flip-flop?

.....

……

……

3. What is an excitation table? Draw the excitation table for SR, D and T flip-flops.

.....

……

4.3.4 Master-Slave Flip-Flop

The master slave flip-flop is constructed using two or more latches. Figure 4.9 shows how two S-R flip-flops can be used to construct a master-slave flip-flop.

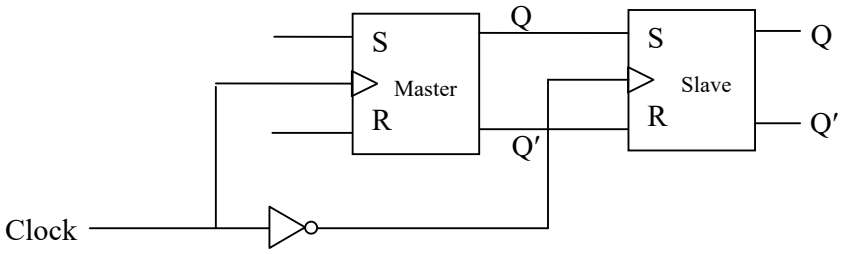


Figure 4.9: Master – Slave flip- flop

You may please note that you can construct a master-slave flip-flop using D or JK flip-flop also. This flip-flop consists of master which changes state when clock pulse occurs. The slave flip flop goes to the state of master flip-flop when the clock signal is 0. (Refer to figure 4.9) This is explained below:

The flip-flop operates is two steps:

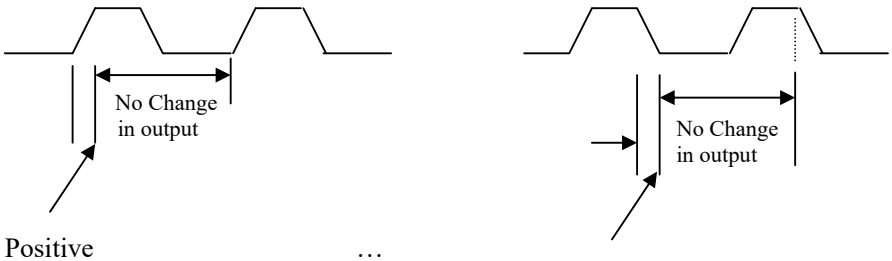
(i) When a clock pulse input is 1: As this time the Master flip-flop, based on the value of S and R, goes to Set or Clear state as the case may be. At this time the slave flip-flop cannot change its state as it receives the inverse of clock pulse. Thus, on the occurrence of clock pulse 'Master' flip-flop goes to the next state (Q_{t+1}), whereas the output from slave flip-flop is the present state (Q_t).

(ii) When the clock pulse input is 0: In this time the input to Master flip-flop will not have any effect on the Master flip-flop output, which has been put in the next state (Q_{t+1}) in the previous step. However, now this Q_{t+1} output of master flip-flop will be applied on the slave flip, which will result in transition of state of slave flip flop to Q_{t+1}. Thus, on completion of a clock cycle master and slave flip-flops both will be in Q_{t+1}. Please note that for slave flip flop only following transitions are possible:

Master output ≡ Slave input				**Slave Output**		
Q	Q′	S	R	Q	Q′	
1	0	1	0	1	0	(Set)
0	1	0	1	0	1	(Reset)

4.3.5 Edge-Triggered flip-flops

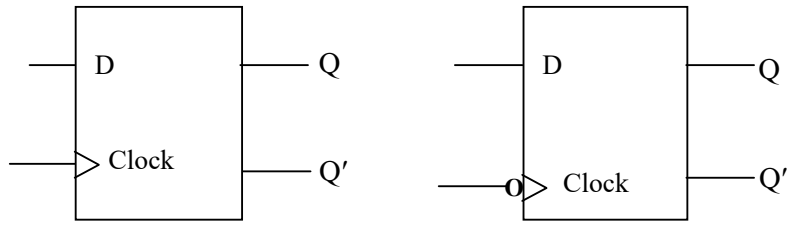
An edge-triggered flip-flop triggers the change either during the rising edge or positive transition (0 to 1 transition) or the falling edge or negative transition of the clock (1to 0 transition). Fig 4.10 shows the clock pulse signal in positive & negative edge-triggered flip-flops.



(a) Positive edge-triggering **(b) Negative edge triggering**

Figure 4.10: Clock Pulse Signal

The following figure shows the block diagram of edge triggered D flip-flop.



(a) Positive edge-triggered D flip-flop **(b) Negative edge–triggered D flip-flop**

Figure 4.11: Edge triggered and master slave D flip-flop

More detailed discussion on these flip-flops are beyond the scope of this unit. You may refer to further readings for the same.

Check Your Progress 2

1. List the advantages of master- slave flip-flop.

   .....

   .....

2. How edge- triggered flip-flops are different to master-slave flip-flops?

   .....

   .....

4.4 SEQUENTIAL CIRCUIT DESIGN

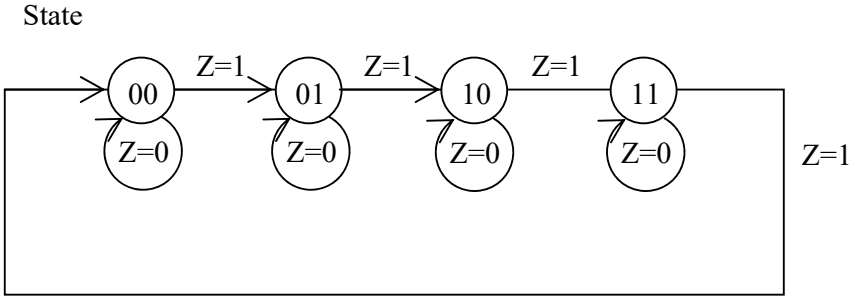
A sequential circuit not only consists of external input and external output, but also an internal state which is characterised by the state of flip flops internal to the circuit. The state of sequential circuit changes as per its design based on some control signal like the clock control. Therefore, design of a sequential circuit is required to address the changes in the internal state of itself. Therefore, in addition to the logic circuit, sequential circuit design requires the information about the changes in state of flip-flops. The process of design of a sequential circuit is explained with the help of an example of design of a 2-bit counter circuit given below.

Example: Design a 2-bit counter circuit.

Solution: A counter is a special circuit which counts the timing sequences. A 2-bit counter will require two flip-flops. The state sequence of 2-bit counter would be 00, 01, 10, 11, 00 and so on. Thus, using a 2-bit counter, you can have 4 distinct internal states of the circuit and counter should move in each transition from one state to next as:

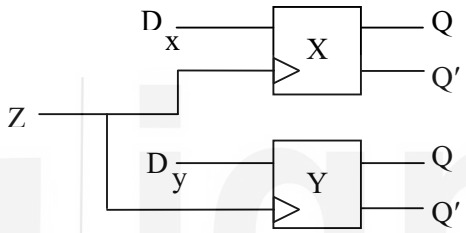
$$00 \longrightarrow 01 \longrightarrow 10 \longrightarrow 11 \longrightarrow 00$$

Assuming that these transitions are triggered by a control signal, say *Z* which can be a clock signal or any other signal generated for this purpose, a state change sequence can be presented as shown in the diagram given below:



This circuit uses two bits to store the state, therefore, requires two flip-flops. The state of the circuit changes to next state, when Z=1, else it stays in the same state. Thus, in this sequential circuit, you require 2 flip-flops and one control signal Z. But, what would be other input and output to this sequential circuit. Well! The other input will be the current states of flip-flops which will govern the next states of flip-flops.

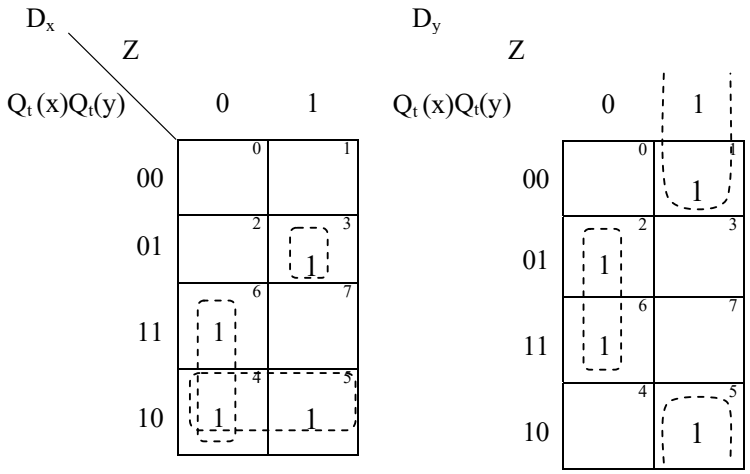
Next, you may take D flip-flop to design the circuit then a Rough design of the circuit would be:



In order to design the logic circuit, which generates the signal D_x and D_y, let us first draw a truth table for flip-flop's X and Y. This truth table is shown in the following table:

	Present States of Flip-Flops			Next State of Flip-Flops		Required value of D_x for transition of X and D_y for the transition of Y	
	Flip-flops		Input	Flip-flops			
	Q_t of X	Q_t of Y	Z	Q_{t+1} of X	Q_{t+1} of Y	D_x	D_y
0	0	0	0	0	0	0	0
1	0	0	1	0	1	0	1
2	0	1	0	0	1	0	1
3	0	1	1	1	0	1	0
4	1	0	0	1	0	1	0
5	1	0	1	1	1	1	1
6	1	1	0	1	1	1	1
7	1	1	1	0	0	0	0

Interestingly, it is the D_x and D_y input that should be generated from the present state and Z input, so that the Next state (Q_{t+1}) of the flip-flops can be derived from the present state of the flip-flop (Q_t). Thus, for the design of counter circuit, you can draw K-map for the design of D_x and D_y with input Q_t (X), Q_t (Y) and Z. TheK-maps for D_x and D_y can be drawn as:



D_x = Terms of (Adjacency of 4,5 + Adjacency of 4,6 + Cell 3)

$D_x = Q_t(X) \cdot Q_t'(Y) + Q_t(X) \cdot Z' + Q_t'(X) \cdot Q_t(Y) \cdot Z$

D_y = Terms of (Adjacency of 2,6 + Adjacency of 1,5)

$D_y = Q_t(Y) \cdot Z' + Q'(Y) \cdot Z$

Thus, the final 2-bit counter circuit will be drawn as shown in Figure 4.12

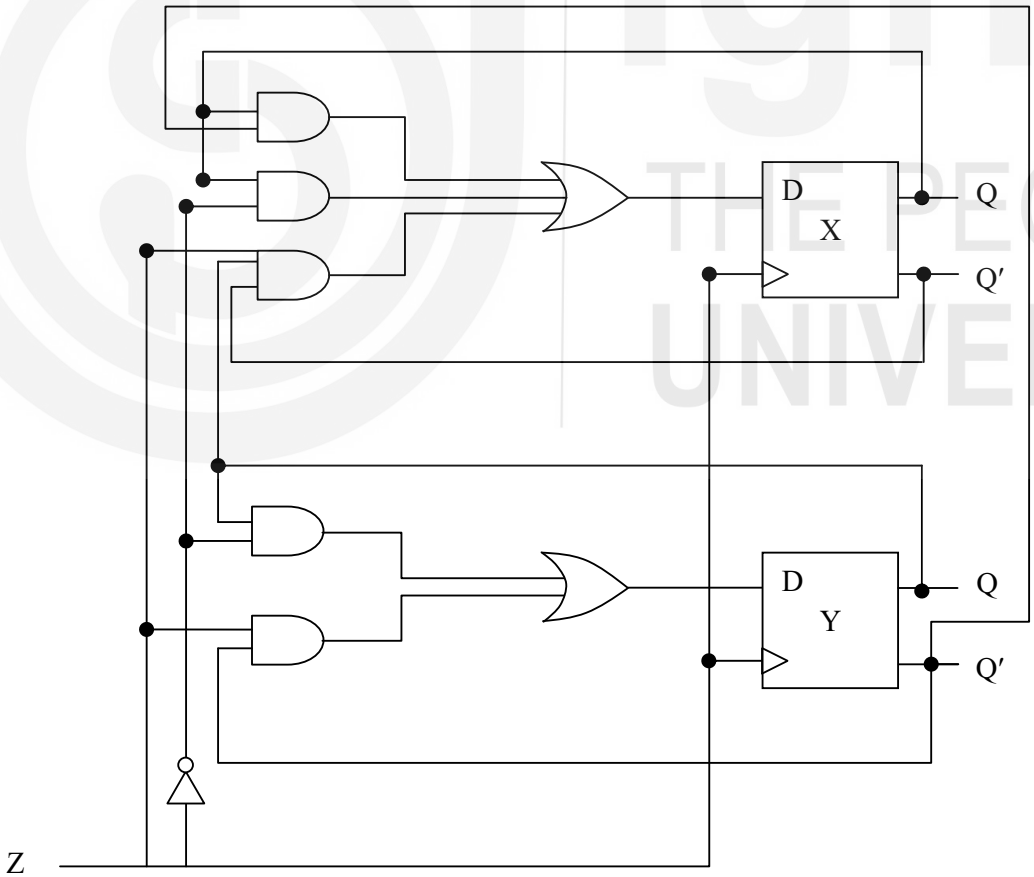


Figure 4.12: 2-bit counter

4.5 EXAMPLES OF SEQUENTIAL CIRCUITS

Let us now explain the basic function of some of the useful examples of sequential circuits like registers, counters etc.

4.5.1 Registers

Registers are the basic storage unit of a computer. Since register temporarily stores certain values, therefore, it requires flip-flops. The size of registers is computed using number of bits it stores. One bit storage requires, at least, one flip-flop. Thus, in general, an n bit register would use n flip-flops. Two common operations on register are:

- To load all bits of a register simultaneously or parallel load.
- Shifting of bits, of register, towards left or right

Figure 4.13 shows a parallel load register..

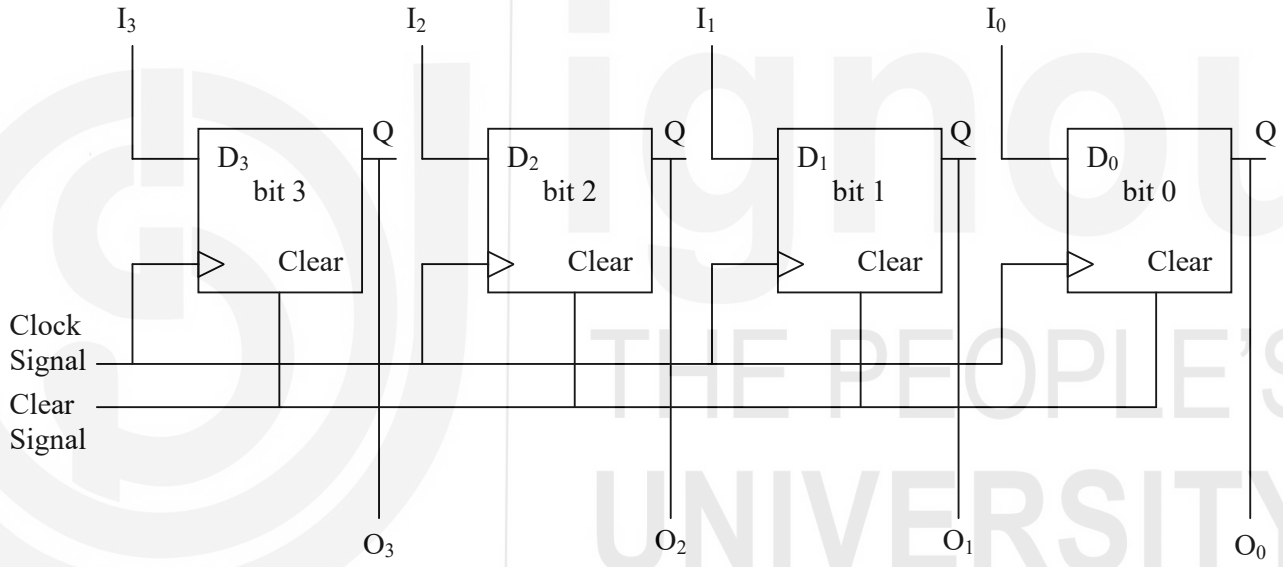


Figure: 4.13 A register with parallel load.

Please note the following point about the register circuit as above:

(1) The 4-bit register is made up of 4 D flip-flops.

(2) Clock signal is applied to all flip-flops simultaneously; therefore, loading operation will load the values I_3, I_2, I_1, and I_0 respectively into the four flip-flops, simultaneously.

(3) Special clear signal is used, which can clear all the bits of the register simultaneously, if needed.

(4) The output of register O_3, O_2, O_1, O_0 can be used for any arithmetic operations. Please note that registers output changes synchronously.

Shift register: Shift operation is very special operation for a computer ALU. A shift register is capable of shifting the content of a register either to left or to the right by one bit at a time. The following figure shows a right shift register, however, you can construct a left shift register in a similar manner.

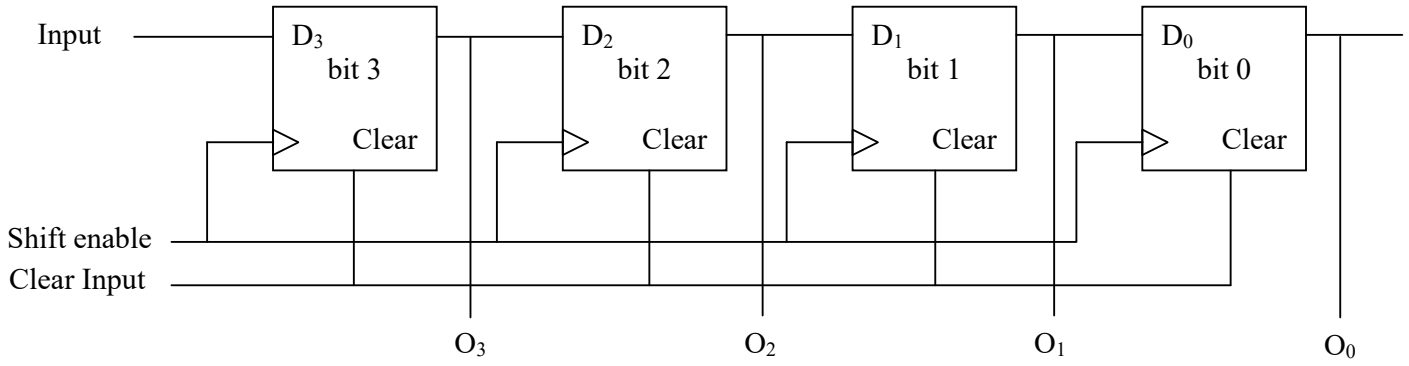


Figure 4.14: 4-bit Right Shift Register

Please note the following points.

- The external input is applied to D_3. The output of D_3 is applied to D_2, and so on.
- The shift enable is applied as clock input. It enables the shift operation.
- For example, assume the shift register had state 1 0 0 1 and input bit is 1, then after the right shift operation the output will change as:

	I	O_3	O_2	O_1	O_0
Before Shift	1	1	0	0	1
After Shift	1	1	1	0	0

A single registers can be included with the facility of left shift, right shift and parallel load. Such a register is called bi-directional shift register with parallel load. You may create its block diagram as an exercise.

4.5.2 Counters Circuit

Counters are sequential circuits, which produce output in a sequence on the occurrence of a transition signal. The counters may be used in keeping sequence such as steps of execution of a single instruction. There are two types of counters- asynchronous and synchronous.

In synchronous counter the flip-flop change their state one by one, while in synchronous counter all flip-flops may change the state simultaneously

Asynchronous Counter: An asynchronous counter is also called a ripple counter as the changes in the state of flip-flop is done one by one like a wave. Figure 4.17 shows a 3-bit ripple counter using T-flip flop (Please note the earlier 2-bit counter was designed using D flip-flop). These counters also have all clear input but for simplicity it is omitted in the figure.

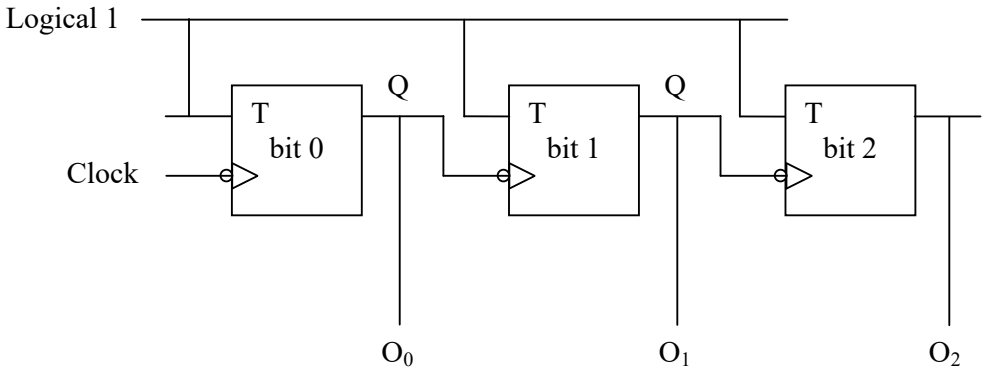


Figure: 4.15: 3-bit ripple counter

Please note the following points in the figure.

(i) The bit 0 will complemented each time a clock pulse occurs as it is connected to clock.

(ii) For bit 1 flip flop the transition will be triggered, if Q_t of bit 0 flip-flop is 1. In that case Q_{t+1} of bit 1 will be complemented. This occurs because the transition signal of (bit 1) flip-flop is connected to Q_t output of bit 0 flip-flop. Similarly, the transition of bit 2 flip flop will occur, if Q_t of bit 1 flip-flop was in state 1.

(iii) The transition is expected to occur with the falling edge (indicated by o before the clock input).

(iv) Please note change in states would be as follows. Assuming initial state to be 0 0 0

bit 0	bit 1	bit 2
0	0	0
1	0	0
0	1	0
1	1	0
0	0	1
1	0	1
0	1	1
1	1	1
0	0	0

} indicates the falling edge.

4.5.3 Synchronous Counter:

The flip-flops of the synchronous counter can change their state simultaneously. A 3-bit synchronous counter with rising edge of clock signal is shown in figure 4.16

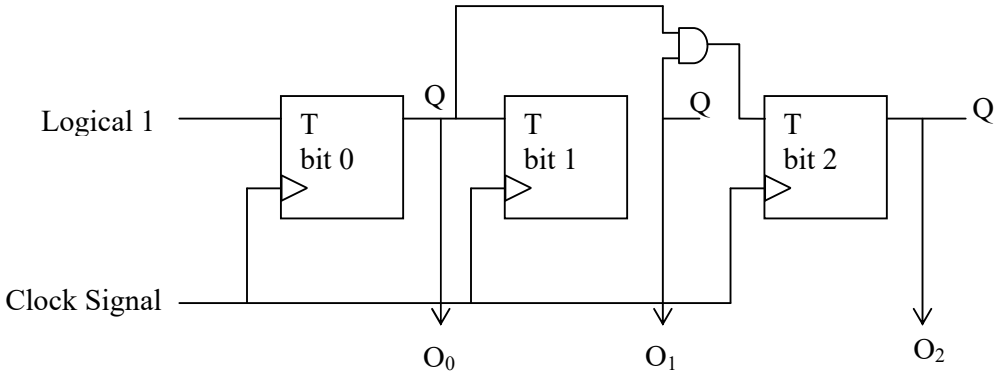


Figure 4.16: Synchronous Counter

Please note in the figure above

(i) bit 0 is complement on occurrence of clock pulse

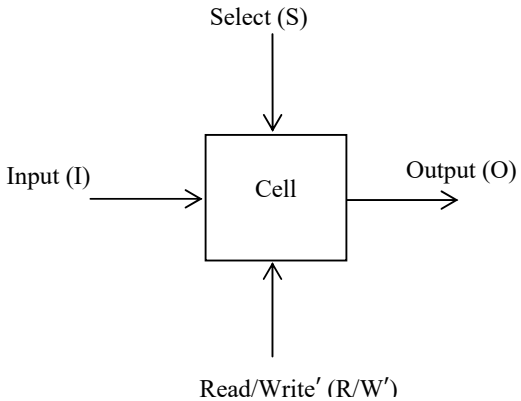
(ii) bit 1 is complemented, if (Q_t of bit 0) was 1.

(iii) bit 2 is complemented, if Q_t of bit 0 and Q_t of bit 1, both are 1.

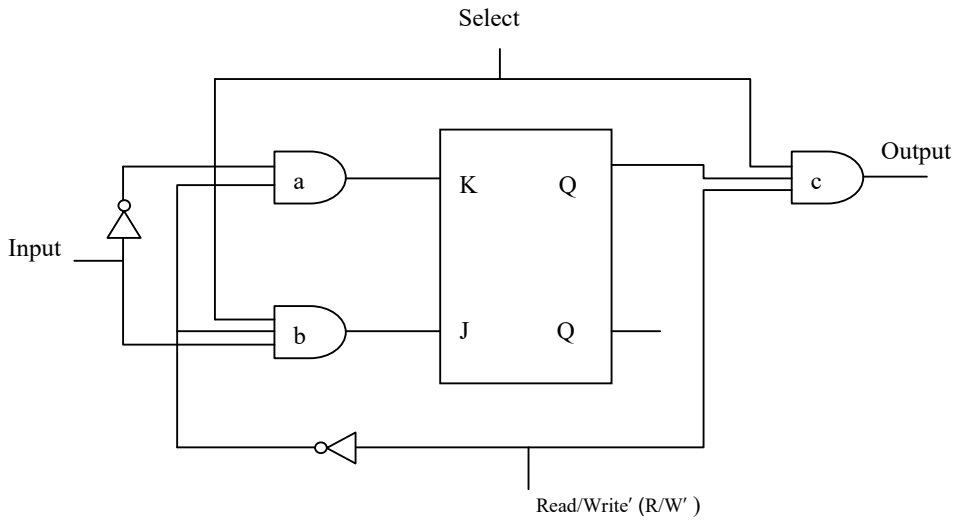
Flip-Flop		
bit 0	bit 1	bit 2
0	0	0
1	0	0
0	1	0
1	1	0
0	0	1
1	0	1
0	1	1
1	1	1
0	0	0

4.5.4 Random Access Memory

In this section, a general configuration for flip-flop based random access memory (RAM) is proposed. A RAM essentially stores bits, therefore, it (especially DRAM technology) may be a sequential circuit. Two basic operations are performed on RAM: *Reading information from RAM,* this operation requires decoding operation, which identifies the cells or lines that are to be read; and *writing to RAM*, which in addition to identifying the cell, also requires changing the state of selected RAM flip-flops based on the input value. The figure 4.21 shows the block diagram and logic diagram and of a RAM cell, which is a single flip-flop.



(a) Block Diagram



(b) Logic Diagram

Figure 4.17: Binary Cell

A RAM cell as shown consists of one flip-flop. The behavior of this cell is exhibited in the following table.

(i) Read/Write′ bit 1 ⇒ Operation is Read

Select bit	Read bit	Q_t	Output of cell (c flip-flop)
0	1	0/1	Not activated
1	1	0/1	Q_t

(ii) Read/Write′ bit 0 ⇒ Write Operation

Assume Input bit to the circuit in Figure 4.17 (b) is I

Select Bit (S)	**Input Bit (I)**	**Q_t of Gate**		**Flip-flop Input**		**Q_{t+1}**	**Comments**
		'a'	**'b'**	**J**	**K**		
0	-	Any	Any	0	0	Q_t	Not selected
1	0	0	1	0	1	0	Clear memory flip-flop
1	1	1	0	1	0	1	Set the memory cell

The write operation as shown in the table above changes the content of memory cell to the value of Input (I), or in other words memory cell has been written into by the value of input (I).

In addition to read/write′ to memory cell, additionally a RAM is to be organized as an array of RAM cells, so as to decode the address of the cells, which is shown in Figure 4.18.

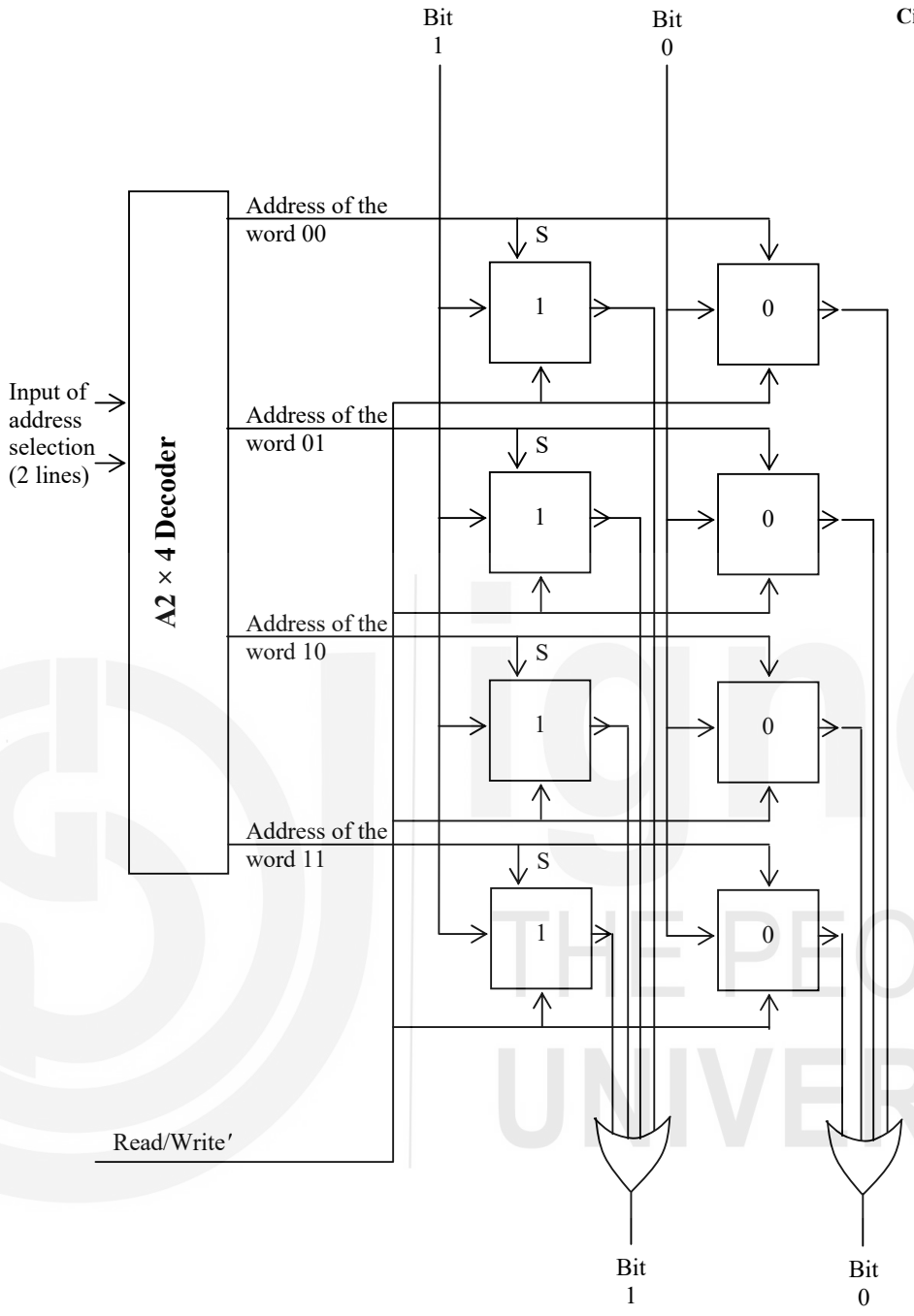


Figure 4.18: Two-dimensional Array based 4 × 2 RAM

The RAM has 4 words, which are decoded by the address decoder. Please note as there are 4 words or lines, therefore, you require 2×4 decoder. This logic can be extended, e.g. a RAM of size 1024×8, would require 10×1024 decoder as $2^{10} = 1024$. So it will have 10 address lines which will decide which word of the RAM array is to be selected.

For this implementation, the number of bits stored in each word would be 2 only, that is why every memory line will have 2 cells. Please note that for a word size of 2 bits, the RAM array would require 2 input and 2 output lines.

For this memory array, in case an address 01 is given as input of address selection bits, it will activate the Select input of cells of address 01 for read or

write operation. Please note that current RAM chip design is not a 2 dimensional design as shown in Figure 4.18. It may follow a different more optimal organization, discussion on which is beyond the scope of this unit.

Check Your Progress 3

1) What are the differences between synchronous & asynchronous counters?

.....

.....

.....

2) Is ripple counter same as shift register?

.....

.....

.....

3) Design a two bit counter, which has the states 00, 01, 10, 00, 01, 10…..

.....

.....

.....

4.6 SUMMARY

This unit introduces you the concepts of sequential circuits which is the foundation of digital design. Flip-flops are also a sequential circuit and the basic storage unit of a computer system. This unit also explains the working of a latch, which is the basic circuit that can be used for storing one bit of information. The sequential circuit can be formed using combinational circuits (discussed in the last unit) and flip flops. The unit also discusses the construction of some of the important sequential circuits like registers, counters, RAM. For more details, the students can refer to further reading.

4.7 SOLUTIONS / ANSWERS

Check Your Progress 1

1. A sequential circuit is designed to process and store data. Therefore, it consists of flip-flops for storing a state representing 0 or 1 and additional combinational circuit that may result in change of state depending on the combinational logic. Example of sequential circuits are - registers, counters etc. The main difference is that a sequential circuit also has a state.

2. Latch is a basic asynchronous sequential circuit designed with feedback to exhibit two different states, viz. 0 or 1. These states can be modified as per the input to latch. Example of latch is SR latch. Latches can change their state at any point of time based on input, whereas flip-flops are designed to change their states at specific time, for example on the occurrence of a clock pulse. Therefore, flip-flops have more complex circuitry than latch.

3. Excitation table are used for analysis and design of sequential circuits. They represent different combination of input to a flip-flop that may cause a specific state transition in the flip-flop. The following are the excitation tables of different flip-flops:

   SR Flip-flop

Present State (Q_t)	Next State (Q_{t+1})	Input S and R	Input using DONOT CARE
0	0	(i) S=0, R=0 (ii) S=0, R=1	**S=0, R=X**
0	1	S=1, R=0	**S=1, R=0**
1	0	S=0, R=1	**S=0, R=1**
1	1	(i) S=0, R=0 (ii) S=1, R=0	**S=X, R=0**

D Flip-flop

Present State (Q_t)	Next State (Q_{t+1})	Input D	Input using DONOT CARE
0	0	D=0	**D=0**
0	1	D=1	**D=1**
1	0	D=0	**D=0**
1	1	D=1	**D=1**

T Flip-flop

Present State (Q_t)	Next State (Q_{t+1})	Input D	Input using DONOT CARE
0	0	T=0	**T=0**
0	1	T=1	**T=1**
1	0	T=1	**T=1**
1	1	T=0	**T=0**

Check Your Progress 2

1. The master-slave flip is a simple structure, which changes its output during the clock pulse, when it is 0, thus, will result in synchronous state transitions of flip-flop.

2. An edge- triggered flip-flop changes it state either during the rising or falling edge of the clock pulse, thus, has a different construction than master-slave slip-flop.

Check Your Progress 3

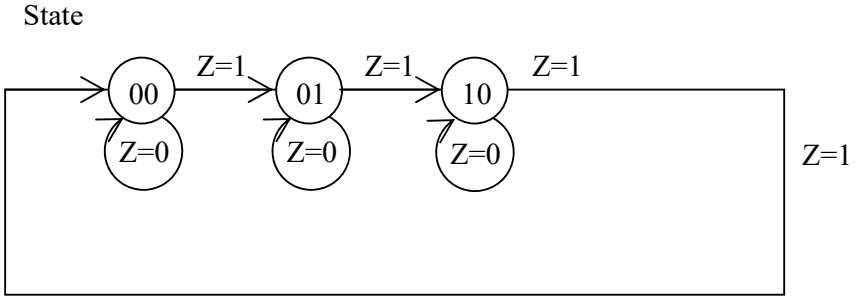
1. All the flip-flops in the synchronous counter may change their state simultaneously, whereas in asynchronous counter, change of state of previous flip-flop may cause that effect to take place.

2. No, shift register causes shifting of state of a flip-flop to next flip-flop, whereas ripple counter is governed by the change of state.

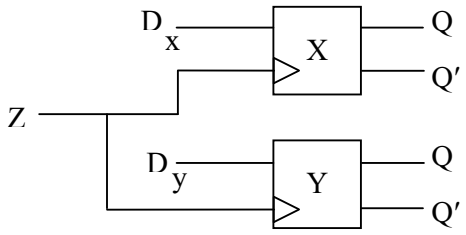
3. The states 00, 01, 10, 00, 01, 10…..

   00 ⟶ 01 ⟶ 10 ⟶ 00 ⟶ 01

   Assuming the control signal, say Z, state transitions are:



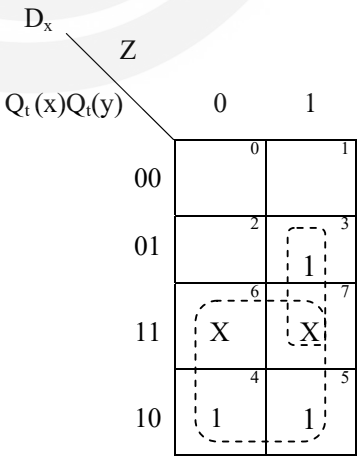
Rough design of the circuit would be:

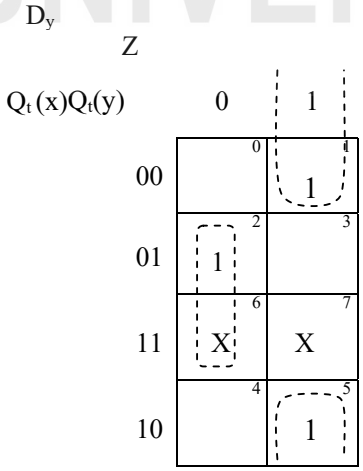


Truth table for flip-flop's X and Y:

	Present States of Flip-Flops			Next State of Flip-Flops		Required value of D_x for transition of X and D_y for the transition of Y	
	Flip-flops		Input	Flip-flops			
	Q_t of X	Q_t of Y	Z	Q_{t+1} of X	Q_{t+1} of Y	D_x	D_y
0	0	0	0	0	0	0	0
1	0	0	1	0	1	0	1
2	0	1	0	0	1	0	1
3	0	1	1	1	0	1	0
4	1	0	0	1	0	1	0
5	1	0	1	0	0	1	1
6	1	1	0	-	-	X	X
7	1	1	1	-	-	X	X

The K-maps for D_x and D_y can be drawn as:





D_x = Terms of (Adjacency of 4,5 + Adjacency of 3,7)

$D_x = Q_t(X) + Q_t(Y) \cdot Z$

D_y = Terms of (Adjacency of 2,6 + Adjacency of 1,5)

$D_y = Q_t(Y) \cdot Z' + Q'(Y) \cdot Z$

Thus, the final counter circuit for the given states would be:

